

TaxDesk Technical Documentation

This document explains the whole project from the ground up. It assumes you can read code but not that you already know databases, web servers, or the engineering ideas behind them. Every concept is defined the first time it appears, then shown in the actual code that uses it.

Everything here describes the repository as it stands today. Where a thing is not built yet, this document says so plainly rather than describing an intention as if it were real.

How to read this. Parts 1 and 2 are the product and the shape of the system. Part 3 is the concept teaching, the longest part. Parts 4 through 6 are the data model, every function explained line by line with its complete code, and traces of what actually happens at runtime. Parts 7 through 9 are the decisions behind the code and the honest gaps. Part 10 is how to run it.

Contents

TaxDesk Technical Documentation

Part 1, The Product

- The problem
- What TaxDesk is
- What is built today
- What is not built yet

Part 2, The Shape Of The System

- Why not just a spreadsheet
- Why the code is in layers
- What each directory is for

Part 3, The Fundamentals

- 3.1 What a relational database is
- 3.2 What SQL is
- 3.3 Primary keys
- 3.4 Foreign keys and referential integrity
- 3.5 Composite keys, surrogate keys, natural keys
- 3.6 Constraints, rules the database enforces
- 3.7 Normalization
- 3.8 Many to many, and the junction table
- 3.9 Soft deletion, keeping history
- 3.10 Parameterized queries and SQL injection
- 3.11 Transactions, and ACID
- 3.12 Idempotency
- 3.13 SQLite, and its three traps
- 3.14 Migrations
- 3.15 HTTP, requests and responses
- 3.16 The post redirect get pattern

- 3.17 Routing, and what a decorator really does
- 3.18 The server and the framework are two different things
- 3.19 Dependency injection
- 3.20 Templates and automatic escaping
- 3.21 Trust boundaries
- 3.22 Testing
- 3.23 Version control, branches, and review

Part 4, The Data Model

CLIENTS

SERVICES

CLIENT_SERVICES

PERIODS

TASKS

documents

SETTINGS

Part 5, The Code, Every Function, Line By Line

- 5.1 The Schema, Statement By Statement
- 5.2 database/migrate.py, The Runner
- 5.3 database/queries.py, Every SQL Statement The App Runs
- 5.4 database/seed.py, The Development Data
- 5.5 app/main.py And app/deps.py, Assembling The Application
- 5.6 The Routes, health.py And onboarding.py
- 5.7 The Templates, What The Browser Receives
- 5.8 The Database Tests
- 5.9 The Application Tests

Part 6, What Actually Happens At Runtime

- Starting the application
- A read, GET /onboarding
- A write, POST /onboarding/confirm

Part 7, Decisions And Rejected Alternatives

- Services as a table, not fixed columns
- Required services live in the runner, not in the schema file
- Settings as one typed row, not a key and value store
- Transactions inside the migration script
- Confirm before create, in onboarding
- Deterministic seed data
- Raw SQL, no ORM

Part 8, What Is Not Built, And Why

Part 9, Honest Limitations

Part 10, Running And Developing

- Running the application
- Development tasks
- Dependencies, and why each exists
- Adding a change

Part 11, Glossary

Part 1, The Product

The problem

A solo Indian tax practitioner files the same four government returns for many clients, every month. Those four are GSTR-3B, GSTR-1, EPF, and ESI. Today the tracking happens in an Excel sheet with hand ticks. Three things go wrong with that.

- ticks get forgotten, and a missed filing means a government penalty with a client's name on it
- the sheet drifts from reality, because nothing stops a wrong or duplicated entry
- the proof files, the challans and acknowledgements, sit in client folders on disk with no connection to the ticks

What TaxDesk is

A small web application that runs on the practitioner's own computer, stores everything in one local file, and answers one question every morning. Which clients still owe GSTR-3B, GSTR-1, EPF, or ESI work this month, and where is the proof for the ones already done.

The phrase for this shape is local first. There is no cloud account, no server rented somewhere, and no copy of client data leaving the machine. The practitioner opens a browser pointed at their own computer.

What is built today

- the database, six tables with the rules enforced inside the database itself
- a migration system, so the structure can change over time without anyone editing a live database by hand
- a development seed, fake data for building against
- the web application skeleton, running on localhost
- onboarding, which turns folders on disk into client records

What is not built yet

Service assignment per client, periods, task generation, the dashboard, document search, and the proof scanner. Part 8 explains why each is absent rather than forgotten.

Part 2, The Shape Of The System

Why not just a spreadsheet

Start from the failure. A spreadsheet cannot refuse a bad entry. You can type the same client twice, tick a filing that does not exist, write "Done" in a column that means something else, or delete a row by accident. Nothing objects.

A database can refuse. That single difference is the reason this project exists as software rather than a better spreadsheet template. Rules that must never break are written into the database, and the database enforces them on every write, no matter which piece of code is doing the writing.

Why the code is in layers

Imagine all the code in one file. A page that draws HTML also runs database commands, and a change to a table breaks the HTML. Worse, a page meant only to display something could accidentally write data, because nothing separates the two jobs.

Splitting the work into layers means each layer has one job and talks only to the layer below it.

```
browser
|      HTTP, requests and responses
web layer  app/routes/, parses input, decides what to show, redirects
|          plain Python function calls
data access database/queries.py, every SQL statement lives here
|          SQL over one connection per request
SQLite     one local file, enforces the data rules itself
|
disk       the practitioner's client folders, read, never written
```

Two rules hold this together.

- routes never contain SQL, they call functions in `database/queries.py`
- the database layer never knows a web application exists, it takes a connection and returns rows

The payoff is that either half can change without touching the other, and each half can be tested on its own.

What each directory is for

```
app/      the web application
  main.py  builds the application, runs migrations at startup
  deps.py  shared pieces, the template engine and the connection
  routes/  one file per group of pages
  templates/ the HTML

database/  everything about stored data
  migrations/ numbered SQL files, the structure
```

migrate.py	opens connections, applies migrations
queries.py	every SQL statement the application runs
seed.py	fake data for development only

docs/	this document and its siblings
tests/	the automated checks

Part 3, The Fundamentals

This part defines every engineering idea the project uses. Each one follows the same shape. What problem it solves, what it actually is, and where it appears in this code.

3.1 What a relational database is

The problem. Facts need somewhere to live that survives the program closing, and those facts have relationships to each other.

What it is. A store of tables. A table is a grid. Each column is one kind of fact, such as a name. Each row is one thing, such as one client. The word relational refers to rows in one table pointing at rows in another.

In TaxDesk. Six tables, described in Part 4. Clients point at the services they need, tasks point at a client and a service and a month.

3.2 What SQL is

The problem. Something must express what you want from the tables.

What it is. A language for talking to a relational database. Four statements carry almost everything in this project.

- SELECT: read rows
- INSERT: add a row
- UPDATE: change existing rows
- DELETE: remove rows, which this project deliberately almost never uses, see 3.12

In TaxDesk. Every SQL statement the running application uses is in `database/queries.py`. The structure statements, the CREATE TABLE ones, are in `database/migrations/`.

3.3 Primary keys

The problem. You need a way to point at exactly one row and never accidentally mean a different one.

What it is. A primary key is a column, or a group of columns, whose value identifies a row uniquely and never repeats in that table.

Why not use the client's name as the key. Names get corrected, respelled, and reused. If everything pointed at the name, then fixing a typo would break every pointer. A number that never changes is safer, so the row keeps a plain integer id whose only job is identity.

In TaxDesk. `CLIENTS.ID`, `SERVICES.ID`, `TASKS.ID` are integer primary keys. `SETTINGS.ID` is an integer too, with a twist explained in 3.6.

3.4 Foreign keys and referential integrity

The problem. A task belongs to a client. Nothing should let a task exist for a client who was never created.

What it is. A foreign key is a column holding another table's primary key. Referential integrity is the guarantee that such a pointer always points at something real. The database refuses writes that would break it.

In TaxDesk, `CLIENT_SERVICES.CLIENT_ID` points at `CLIENTS.ID`, `TASKS.SERVICE_ID` points at `SERVICES.ID`, and so on. A test proves the refusal is real, an attempt to insert a document for a nonexistent client raises an error.

3.5 Composite keys, surrogate keys, natural keys

Three related ideas, easiest side by side.

- natural key: the real world fact already identifies the row. A month is identified by its year and its month number, nothing else is needed
- surrogate key: an invented id with no business meaning, used when the natural identity is awkward to carry around
- composite key: a key made of more than one column

In TaxDesk, all three appear.

- `PERIODS` uses a natural composite key, `(YEAR, MONTH)`. August 2026 can exist only once, and the key says so directly
- `CLIENT_SERVICES` uses a composite key, `(CLIENT_ID, SERVICE_ID)`. The pair is the fact, so the pair is the identity
- `TASKS` uses a surrogate key. Its natural identity is four columns, client and service and year and month, which is clumsy for other tables to reference, so it carries a plain `ID` and keeps the natural identity as a separate rule, see 3.7

3.6 Constraints, rules the database enforces

The problem. A rule written only in application code holds until one code path forgets it. Six months later a different function writes the same table and the rule quietly stops applying.

What it is. A constraint is a rule attached to the table. The database checks it on every write from anyone, forever.

The kinds this project uses.

- NOT NULL: this column must have a value
- UNIQUE: no two rows may repeat this value or combination
- CHECK: the value must satisfy a condition
- PRIMARY KEY: identity, and implicitly unique and not null
- FOREIGN KEY: must point at an existing row

In TaxDesk, the important ones.

- `UNIQUE (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH)` on `TASKS`. This is the single most important line in the schema. It makes a duplicate task physically impossible, so the central promise of the product does not depend on the code being correct
- `FOLDER_PATH TEXT NOT NULL UNIQUE` on `CLIENTS`. Two clients can never claim the same folder
- `CHECK (STATUS IN ('pending', 'done', 'not_applicable'))` on `TASKS`. A typo such as "Done" is rejected at write time
- `CHECK (ID = 1)` on `SETTINGS`. A clever use, explained in Part 4, it caps the whole table at one row

3.7 Normalization

The problem. If the same fact is written in two places, the two copies drift apart, and then nobody knows which is true.

What it is. Organising tables so each fact lives in exactly one place, and everything else points at it.

In TaxDesk. A service name such as GSTR-3B is stored once, as a row in `SERVICES`. Tasks store the service id, never the text. Renaming a service later would touch one row, and nothing could disagree, because nothing else holds a copy.

The test for whether a design is normalized enough. Changing one real world fact should change one row.

3.8 Many to many, and the junction table

The problem. One client needs several services. One service applies to many clients. Neither table can hold that on its own, because a column can hold one value, not a list.

What it is. A third table whose rows are the pairings. It is called a junction table, or a join table.

In TaxDesk. `CLIENT_SERVICES` holds one row per client and service pair, keyed by the pair itself so the same pairing cannot exist twice.

3.9 Soft deletion, keeping history

The problem. When a practitioner stops filing EPF for a client, the past still happened. Tasks were generated, some were completed. Deleting the relationship would erase the explanation for that history.

What it is. Instead of removing the row, a flag marks it inactive. The record survives, the behavior changes.

In TaxDesk. `CLIENT_SERVICES.ACTIVE` is that flag, and the schema file says so in a comment, off is a flag, never a delete. Task generation reads only rows where `ACTIVE = 1`.

3.10 Parameterized queries and SQL injection

The problem. If user input is glued into a SQL string, the input can change the meaning of the command. A field containing `' ; DROP TABLE CLIENTS; --` stops being data and becomes an instruction. That attack is called SQL injection.

What it is. A parameterized query sends the SQL and the values separately. The value is filled in after the command is parsed, so it can never be read as part of the command.

In TaxDesk. Every statement uses `?` placeholders and passes values as a tuple. There is no string built from user input anywhere in `database/queries.py`.

```
conn.execute(
    "INSERT OR IGNORE INTO CLIENTS (NAME, FOLDER_PATH) VALUES (?, ?)",
    (name, folder_path),
)
```

3.11 Transactions, and ACID

The problem. Some changes only make sense together. The migration runner applies a schema file and then records that it did. If the program died between those two steps, the database would be wrong about itself forever.

What it is. A transaction groups writes so that either all of them become permanent or none of them do. Committing makes them permanent. Rolling back discards them.

ACID is the four letter summary of what a database transaction guarantees.

- Atomicity: all of it or none of it
- Consistency: committed data satisfies every constraint
- Isolation: concurrent work does not see each other's half finished changes
- Durability: once committed, it survives a crash

In TaxDesk, two places.

- `database/migrate.py` wraps each migration and its logbook record in one transaction, so the pair cannot separate
- `app/deps.py` commits a web request's writes only if the request finished cleanly, and rolls them back if it raised

3.12 Idempotency

The problem. Real usage is messy. Buttons get double clicked, scripts get rerun, and people forget what they already did.

What it is. An operation is idempotent when doing it twice leaves the same result as doing it once.

In TaxDesk, deliberately, everywhere it matters.

- applying migrations, the logbook makes a rerun do nothing
- the development seed, running it twice produces byte identical state, and a test proves it by comparing every row
- creating clients, `INSERT OR IGNORE` plus the `UNIQUE` rule absorbs a repeat
- task generation, the same shape, proven by the seed

3.13 SQLite, and its three traps

What it is. SQLite is a relational database that runs as a library inside your own program rather than as a separate server. The whole database is one ordinary file. For one office on one machine that is close to ideal, and its weakness, many programs writing at once, is a problem this product does not have.

Three behaviors surprised this project and are worth knowing.

- foreign keys are off by default, per connection. SQLite ships that way for backwards compatibility, and the setting cannot be stored in the file or in the schema. Every single connection must run `PRAGMA foreign_keys = ON`, which is why one function owns opening the database and nothing else may bypass it
- `executescript` commits on its own. It ends any open transaction before running, so calling `.commit()` afterwards does not make the script atomic with anything else. This project therefore writes `BEGIN` and `COMMIT` inside the executed script itself
- there are no real date or length types. `VARCHAR(100)` is accepted and the length is ignored, and dates are text you format yourself. This project stores text in ISO order and says so

3.14 Migrations

The problem. The database file holds real client data and never leaves the practitioner's machine, so it cannot be shipped. The structure, though, must be identical on every machine, and must be able to change over time without anyone editing a live database by hand.

What it is. Each structural change is a numbered SQL file, committed to version control. A runner applies the files a given database has not seen yet, in order, and records each one it applied. Applied files are then frozen forever, because editing one would never reach databases that already ran it. A change is always a new file.

In TaxDesk. `database/migrations/001_schema.sql` and `002_settings.sql`, applied by `initialize()` in `database/migrate.py`, recorded in a table named `schema_applied` that lives inside the database itself so each database knows its own state.

3.15 HTTP, requests and responses

What it is. The language a browser and a server speak. The browser sends a request, the server sends back a response. Nothing is remembered between requests unless something deliberately stores it, which is what the word stateless means.

The pieces used here.

- methods, the verb of the request. GET means read me something, POST means here is data to save
- status codes, the three digit verdict. 200 fine, 302 and 303 go to another page, 400 your input was bad, 404 no such thing, 500 the server broke
- the body, the form fields on the way in, the HTML on the way out

Why the GET and POST distinction matters more than it looks. A browser will repeat a GET freely, on refresh, on back, on a prefetch. If a write hid behind a GET, a refresh would silently write again. So reads are GET, writes are POST, with no exceptions.

3.16 The post redirect get pattern

The problem. After a form POST, the browser is sitting on the result of a write. Pressing refresh resubmits the form and writes twice.

What it is. The server answers a successful POST with a redirect to a normal page. The browser then performs a plain GET, and that GET is what the user is sitting on. Refresh now repeats the harmless read.

In TaxDesk. Every POST route ends with a 303 redirect. The 303 code specifically instructs the browser to use GET for the next request, which is the detail that makes the pattern work.

3.17 Routing, and what a decorator really does

What it is. Routing is the map from a method and a path, such as GET and `/onboarding`, to the function that answers it.

The decorator. A line like `@router.get("/onboarding")` above a function is a decorator. It is not magic. It is a function that receives the function written below it and, in this case, stores an entry in a table saying that GET on that path is handled here. This happens once, when the module is imported, before any request arrives. At request time the framework just looks the path up.

In TaxDesk. Five routes exist today.

```
GET /                redirect to onboarding
GET /health          liveness, returns {"status": "ok"}
GET /onboarding       the root form plus folder discovery
POST /onboarding/root save the root folder
POST /onboarding/confirm create clients from confirmed folders
```

3.18 The server and the framework are two different things

A common confusion worth clearing. FastAPI decides what to answer. Uvicorn is the program that actually listens on a network port, accepts connections, and hands each request to FastAPI. FastAPI cannot listen on its own, and uvicorn does not know what your routes mean. They are separate jobs, and the interface between them is a standard called ASGI.

3.19 Dependency injection

The problem. Every route needs a database connection, opened correctly, closed reliably, committed on success and rolled back on failure. Twenty routes each doing that by hand means twenty chances to get it wrong.

What it is. The route declares what it needs rather than building it. The framework supplies it. That is the whole idea, receive your tools, do not construct them.

In TaxDesk. A route signature says `conn: Connection = Depends(get_db)`. Before the route runs, FastAPI calls `get_db` in `app/deps.py`. That function is a generator, it opens the connection and pauses at `yield`, handing the connection over. When the route finishes, the function resumes and either commits or rolls back, then always closes. One

correct lifecycle, written once, impossible for a route to forget.

3.20 Templates and automatic escaping

The problem. Building HTML by joining strings is unreadable, and dangerous. If a client were named `<script>steal()` `</script>` and that text were pasted into the page, the browser would run it. That attack is called cross site scripting, XSS.

What it is. A template is HTML with labelled blanks. The engine fills the blanks and escapes the values on the way in, turning `<` into `<`, so a value can never turn into code.

In TaxDesk. Jinja2 templates in `app/templates/`. `base.html` is the shared frame, other pages extend it. Escaping is on by default, so safety is the default rather than something to remember.

3.21 Trust boundaries

The problem. Data arriving from outside the program may be shaped however the sender wants, including deliberately hostile shapes.

What it is. A trust boundary is the line where outside data enters. At that line, the program validates rather than assumes. Running on localhost does not remove this, because the browser is still outside the program.

In TaxDesk, the sharpest example. The confirm step of onboarding never trusts submitted folder names. It rescans the real root folder, builds the set of names that genuinely exist, and accepts only names in that set. A crafted submission such as `../evil`, or an absolute path pointing somewhere else entirely, matches nothing and is silently ignored. A test sends exactly those and proves no client is created.

That style is called deriving the valid set from a trusted source rather than validating the untrusted input, and it is the stronger of the two approaches, because it cannot be fooled by an input shape nobody predicted.

3.22 Testing

Why. Manual checking decays. Code changes, and behavior that worked yesterday quietly breaks with nothing to notice.

The kinds used here.

- unit test: exercises one function directly
- integration test: exercises several layers together, for example a real HTTP request that reaches the real database
- fixture: shared setup a test asks for by name, such as a fresh database

A rule this project follows. A test must be able to fail. A test that passes no matter what the code does proves nothing. When the rollback behavior in `get_db` had no test, that behavior was unproven even though it was written and documented, so a test was added that writes and then deliberately raises.

In TaxDesk. 33 tests, all on temporary databases in temporary folders, so the developer's real database and real files are never touched.

tests/test_database.py	8	migrations, constraints, rollback boundary
tests/test_seed.py	5	seed content and repeatability
tests/test_app.py	7	skeleton, health, startup, transactions
tests/test_onboarding.py	13	discovery, confirmation, trust boundary

3.23 Version control, branches, and review

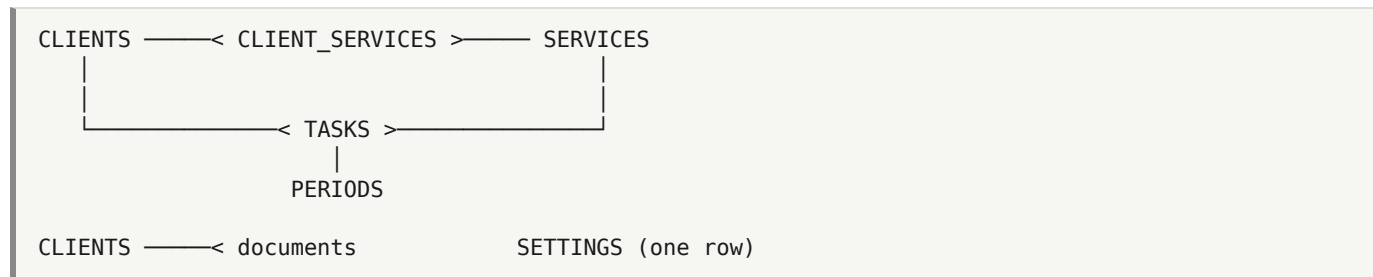
What it is. Git records snapshots of the project called commits. A branch is a line of work separate from the main line. A pull request, PR, proposes merging a branch and is where review happens.

The workflow this project uses. One branch per intent, named for it. Work is committed and pushed there. A PR is opened. Review happens on the PR, findings get fixed on the same branch, and only then does it merge into main. Main therefore only ever contains reviewed code.

This has already paid for itself. Three real defects were caught in review rather than in use, described in Part 7.

Part 4, The Data Model

Six tables. The full definitions are in `database/migrations/001_schema.sql` and `002_settings.sql`.



CLIENTS

Who the practitioner works for. Holds the name and `FOLDER_PATH`, the absolute path of that client's folder on disk. The path is **UNIQUE**, because a folder identifies a client and two clients cannot share one.

SERVICES

One row per filing type. The four rows, GSTR-3B, GSTR-1, EPF, ESI, are inserted by the migration runner rather than by the schema, for a reason explained in Part 7. The name is **UNIQUE**.

This being a table rather than four fixed columns is what makes a fifth filing type a new row someday instead of a structural change.

CLIENT_SERVICES

The junction table from 3.8. One row per client and service pair, keyed by the pair, carrying `ACTIVE` as the soft delete flag from 3.9.

Today nothing in the application writes to this table. Onboarding creates clients and stops. That is the gap the next feature fills.

PERIODS

One row per tracked month, keyed naturally by `(YEAR, MONTH)`, with `STATE` holding open or closed. A closed month is meant to refuse further changes, and the schema stores the state, while the rule that enforces it belongs to the application code that does not exist yet.

TASKS

The heart of the eventual product. One row means one client owes one filing for one month. It carries the status, the due date, and the completion trace, when and how a task was completed.

Its four column `UNIQUE` constraint is the reason duplicate work items cannot exist. Its due date is deliberately nullable, because the real due day for each filing type is not yet known and inventing one would be inventing a business rule.

documents

File links owned by a client. Deliberately minimal. The relationship between a document and a task is unresolved on purpose, because no requirement has yet forced its shape.

SETTINGS

Application configuration as exactly one row.

```
CREATE TABLE SETTINGS (  
  ID INTEGER PRIMARY KEY CHECK (ID = 1),  
  ROOT_FOLDER TEXT NOT NULL  
);
```

The `CHECK (ID = 1)` is worth pausing on. Since the primary key must be unique and this check forces it to be the value 1, the table can physically never hold a second row. No row at all means the application is not configured yet, which is how onboarding knows to ask.

Part 5, The Code, Every Function, Line By Line

This is the heart of the book. Every source file in the project appears here, and inside each one, every function, constant, SQL statement, template, and test, with its complete code quoted exactly as it exists in the repository, followed by a plain English walkthrough. Nothing is elided and nothing is paraphrased, the code printed here is the code that runs.

Each piece follows the same rhythm. First the code itself, then why it exists, then how it works, read top to bottom, with every idiom named and explained the first time it appears. If a line looks mysterious, the paragraphs after the code block are where to look.

One housekeeping note before the chapters. Three files in the project are completely empty, `app/__init__.py`, `app/routes/__init__.py`, and `database/__init__.py`. An `__init__.py` file marks its folder as a Python package, which is what makes an import like `from database import queries` resolve. They are empty because marking the folder is their entire job, and code placed inside them runs on import, which is a place surprises love to hide. They get no chapters because there is nothing more to say about them.

5.1 The Schema, Statement By Statement

Everything TaxDesk knows lives in one SQLite database, and these two files are where that database gets its shape. They are migrations, plain SQL files that the app runs exactly once each, in numbered order, to build or extend the schema. The first file, `001_schema.sql`, creates the six tables that model the product itself, clients, the services they buy, the client to service subscriptions, the months being worked, the tasks that pair a client, a service, and a month, and the documents on disk. The second file, `002_settings.sql`, adds a small configuration table with an unusual constraint that guarantees it can never hold more than one row. Read together they are the contract every other part of the system builds on, so we will walk them line by line and take nothing on faith.

The File Header

Before any table, `001_schema.sql` opens with four comment lines. In SQL a line starting with `--` is a comment, ignored by the database engine but read by every human who opens the file.

```
-- TaxDesk schema.  
-- Foreign key enforcement is per connection in SQLite, a PRAGMA in this  
-- file would not stick. The code that opens the database must run  
-- PRAGMA foreign_keys = ON on every connection.
```

Why it exists. This comment is a warning about a genuine SQLite trap. SQLite ships with foreign key enforcement turned off by default, for historical compatibility, and the switch that turns it on is a PRAGMA, a special SQLite command that changes engine settings. That switch applies only to the connection that ran it, not to the database file itself.

How it works. The first line names the file. The next three explain why you will not find `PRAGMA foreign_keys = ON` anywhere in this migration. If the line were here, it would run once, during migration, on the migration's own connection, and then vanish. Every later connection the app opens would still have enforcement off, and all the FOREIGN KEY clauses you are about to read would be silently ignored. So the comment pushes the responsibility where it actually belongs, into the Python code that opens connections, which must run the PRAGMA every single time. This is a good example of a comment carrying information the code cannot, because the absence of a line is invisible unless someone explains it.

CLIENTS

```
CREATE TABLE CLIENTS(  
    ID INTEGER PRIMARY KEY,  
    NAME TEXT NOT NULL,  
    -- the folder is a client's identity on disk, two clients can never share one  
    FOLDER_PATH TEXT NOT NULL UNIQUE  
);
```

Why it exists. A client is the central noun of TaxDesk, the person or business the practitioner does tax work for. Every service subscription, every task, and every document in the rest of the schema hangs off a row in this table. In TaxDesk a client also has a physical presence, a folder on disk, and this table is where the database record and the folder are tied together.

How it works. `CREATE TABLE CLIENTS(` opens the statement and names the table. Everything up to the closing `);` is the list of columns and constraints.

`ID INTEGER PRIMARY KEY` declares the identifier column, and in SQLite this exact spelling is special. Every ordinary SQLite table secretly has a hidden integer column called the rowid, a unique number the engine assigns to each row. When you declare a single column as `INTEGER PRIMARY KEY`, that column does not sit beside the rowid, it becomes the rowid under a visible name. Two things follow. Inserting a row without supplying an ID makes SQLite pick one automatically, usually one more than the current largest, so you get autoincrementing behavior for free. And lookups by ID are as fast as the engine can go, because you are searching the table's own physical key. `PRIMARY KEY` also means unique and, for this integer form specifically, never NULL. NULL is the database marker for an absent value, and a primary key must always be present.

`NAME TEXT NOT NULL` stores the client's display name. `TEXT` is SQLite's string type. `NOT NULL` is a constraint, a rule the engine enforces on every insert and update, and here it refuses any attempt to create a client with no name. The mistake it blocks is a half filled row that would later render as a blank entry in every list in the app.

The comment above the last column explains a design decision. In TaxDesk a client is discovered from a folder, so the folder path is not incidental data, it is the client's identity on disk. If two client rows could point at the same folder, the app could not tell whose documents live there.

`FOLDER_PATH TEXT NOT NULL UNIQUE` enforces exactly that. `NOT NULL` refuses a client with no folder. `UNIQUE` makes the engine reject any second row carrying a folder path that already exists in the table. The real mistake it blocks is onboarding scanning the same directory twice and creating a duplicate client, a bug that would silently split one client's history across two rows. With the constraint in place the second insert fails loudly instead.

SERVICES

```
CREATE TABLE SERVICES(  
    ID INTEGER PRIMARY KEY,  
    NAME TEXT NOT NULL UNIQUE  
);
```

Why it exists. A service is a kind of recurring tax work, the thing a client subscribes to and a task gets created for. Keeping services in their own table, rather than as free text on each task, means the list of offerings lives in one place and every other table can point at it by number.

How it works. `ID INTEGER PRIMARY KEY` is the same idiom as in `CLIENTS`, the column becomes the table's rowid, autoassigns on insert, and is guaranteed unique and present. `NAME TEXT NOT NULL UNIQUE` holds the service's name and stacks two constraints. `NOT NULL` refuses a nameless service. `UNIQUE` refuses a second service with the same name, which blocks the classic seeding mistake where a setup script runs twice and the database ends up with two rows both called the same thing. With duplicates possible, half the clients might link to one copy and half to the other, and no report would ever add up. The constraint makes that state unrepresentable.

CLIENT_SERVICES

```
CREATE TABLE CLIENT_SERVICES(  
    CLIENT_ID INTEGER NOT NULL,  
    SERVICE_ID INTEGER NOT NULL,  
    -- switching a service off must keep history, off is a flag, never a delete  
    ACTIVE INTEGER NOT NULL DEFAULT 1 CHECK (ACTIVE IN (0, 1)),  
    PRIMARY KEY (CLIENT_ID, SERVICE_ID),  
    FOREIGN KEY (CLIENT_ID) REFERENCES CLIENTS(ID),  
    FOREIGN KEY (SERVICE_ID) REFERENCES SERVICES(ID)  
);
```

Why it exists. One client can subscribe to several services, and one service is sold to many clients. A relationship like that cannot live as a column on either table, it needs a junction table, a table whose rows each record one pairing between a row of one table and a row of another. `CLIENT_SERVICES` is that table, and it also records whether each pairing is currently switched on.

How it works. `CLIENT_ID INTEGER NOT NULL` and `SERVICE_ID INTEGER NOT NULL` are the two halves of the pairing, each an integer that will hold the ID of a row in `CLIENTS` or `SERVICES`. Both are `NOT NULL` because a subscription missing either side means nothing. Declaring `NOT NULL` explicitly here also guards a SQLite quirk worth knowing, in SQLite a `PRIMARY KEY` column is allowed to be `NULL` unless you say otherwise, an old bug preserved for compatibility. The exceptions are the `INTEGER PRIMARY KEY` rowid form and primary key columns in `WITHOUT ROWID` tables, both of which do enforce `NOT NULL`, and this composite key is neither. Spelling out `NOT NULL` closes that door.

The comment before `ACTIVE` states a product rule. When a client stops buying a service, the past tasks done under that subscription must survive. Deleting the row would orphan or cascade away that history, so deactivation is a flag flip, never a delete.

`ACTIVE INTEGER NOT NULL DEFAULT 1 CHECK (ACTIVE IN (0, 1))` packs four ideas into one line. SQLite has no boolean type, so true and false are stored as the integers 1 and 0. `NOT NULL` means the flag must always have a value. `DEFAULT 1` tells the engine what to store when an insert does not mention the column, so a brand new subscription is active without the code having to say so. `CHECK (ACTIVE IN (0, 1))` is a check constraint, an arbitrary condition the engine evaluates on every insert and update, refusing the row when the condition is false. `IN (0, 1)` is true only when the value is exactly 0 or exactly 1. The real mistake this blocks is someone storing 2, or 'yes', or -1 in a column that every query reads as a boolean, values that would make `WHERE ACTIVE = 1` quietly miss rows that were meant to count as active.

`PRIMARY KEY (CLIENT_ID, SERVICE_ID)` is a composite primary key, a primary key made of two columns together. The pair must be unique, so client 3 can subscribe to service 5 exactly once, and a double insert of the same pairing fails. Note that because this is a two column key, not the single `INTEGER PRIMARY KEY` form, it does not become the rowid, the table still keeps its hidden rowid and this key is enforced through an internal index.

The two `FOREIGN KEY` lines are referential constraints. `FOREIGN KEY (CLIENT_ID) REFERENCES CLIENTS(ID)` declares that every value in `CLIENT_ID` must exist as an ID in `CLIENTS`, and `FOREIGN KEY (SERVICE_ID) REFERENCES SERVICES(ID)` does the same for services. When enforcement is on, remember the file header, the engine refuses two kinds of mistakes. It refuses inserting or updating a subscription that points at a client or service ID that does not exist. And because no `ON DELETE` behavior is declared, it also refuses deleting a client or a service while any subscription row still points at it. Both refusals protect the same invariant, no row in this table can ever dangle.

PERIODS

```
CREATE TABLE PERIODS(  
    YEAR INTEGER NOT NULL CHECK (YEAR BETWEEN 2000 AND 2100),  
    MONTH INTEGER NOT NULL CHECK (MONTH BETWEEN 1 AND 12),  
    STATE TEXT NOT NULL DEFAULT 'open' CHECK (STATE IN ('open', 'closed')),  
    PRIMARY KEY (YEAR, MONTH)  
);
```

Why it exists. Tax work is monthly, and TaxDesk treats each month as a first class thing with its own lifecycle rather than a pair of numbers scattered across task rows. A period is one calendar month, identified by year and month, and it is either still being worked or finished. Giving periods their own table lets tasks point at a month that provably exists and lets the app close a month as a single state change.

How it works. `YEAR INTEGER NOT NULL CHECK (YEAR BETWEEN 2000 AND 2100)` stores the year and constrains it to a sane range. `BETWEEN 2000 AND 2100` is true when the value is inside that range, endpoints included. The mistake this blocks is a malformed year sneaking in from a bug or a bad import, the year 24 when someone meant 2024, or the year 20024 from a typo. Either would sort wrongly and create phantom periods no query expects.

`MONTH INTEGER NOT NULL CHECK (MONTH BETWEEN 1 AND 12)` does the same for months. There is no month 0 and no month 13, and this check makes the database itself the last line of defense against off by one bugs in code that builds months from zero based indexes, a very common slip when a language's date library counts months from 0.

`STATE TEXT NOT NULL DEFAULT 'open' CHECK (STATE IN ('open', 'closed'))` gives each period a two value lifecycle. `DEFAULT 'open'` means a newly created period is open without the inserting code saying so, which matches reality, a month starts as work in progress. The `CHECK` restricts the column to exactly the two spellings `'open'` and

'closed', blocking the mistake where one code path writes 'Closed' or 'done' and every query filtering on state splits the data into invisible variants. When a column holds a small fixed vocabulary, a CHECK with IN is the SQLite idiom for what other databases call an enum.

PRIMARY KEY (YEAR, MONTH) is another composite primary key. A given month of a given year can exist only once, so nothing can create March 2026 twice with two different states. The NOT NULL on both columns is again explicit rather than assumed, for the same SQLite primary key quirk described under CLIENT_SERVICES.

TASKS

```
CREATE TABLE TASKS(
  ID INTEGER PRIMARY KEY,
  CLIENT_ID INTEGER NOT NULL,
  SERVICE_ID INTEGER NOT NULL,
  PERIOD_YEAR INTEGER NOT NULL,
  PERIOD_MONTH INTEGER NOT NULL,
  STATUS TEXT NOT NULL DEFAULT 'pending'
    CHECK (STATUS IN ('pending', 'done', 'not_applicable')),
  -- nullable on purpose, real due days are unknown until the practitioner confirms them
  DUE_DATE TEXT,
  COMPLETED_AT TEXT,
  COMPLETION_METHOD TEXT,
  -- the core product rule, one client, one service, one month, at most one task
  UNIQUE (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH),
  FOREIGN KEY (CLIENT_ID) REFERENCES CLIENTS(ID),
  FOREIGN KEY (SERVICE_ID) REFERENCES SERVICES(ID),
  FOREIGN KEY (PERIOD_YEAR, PERIOD_MONTH) REFERENCES PERIODS(YEAR, MONTH)
);
```

Why it exists. A task is the unit of work the whole product revolves around, one client, one service, one month, and whether that work got done. This table joins CLIENTS, SERVICES, and PERIODS into concrete work items and records their outcome. It is the largest statement in the file because it carries the most rules.

How it works. ID INTEGER PRIMARY KEY is the rowid alias idiom a third time, giving each task an autoassigned unique integer identity.

CLIENT_ID INTEGER NOT NULL and SERVICE_ID INTEGER NOT NULL name whose work this is and what kind. PERIOD_YEAR INTEGER NOT NULL and PERIOD_MONTH INTEGER NOT NULL name when. Notice the period is referenced by two columns, not one, because PERIODS itself has a two column primary key. All four are NOT NULL since a task missing any of them is not a task.

STATUS TEXT NOT NULL DEFAULT 'pending' CHECK (STATUS IN ('pending', 'done', 'not_applicable')) is the enum via CHECK idiom again, this time with three states. A new task defaults to 'pending', awaiting work. 'done' means completed. 'not_applicable' records the explicit decision that this client did not need this service this month, which is different from pending and different from done, and having it as a state means that decision survives in the record instead of tasks being deleted to express it. The CHECK blocks any other spelling, so filtering and counting by status stays trustworthy. The declaration wraps onto a second line in the source, that is pure formatting, SQL reads it as one clause.

The comment above `DUE_DATE` flags something readers would otherwise report as a bug. Three columns in a schema this strict suddenly allow `NULL`, and the comment explains that for `DUE_DATE` it is deliberate, the real due day for a task is not known until the practitioner confirms it, so the honest value at creation time is no value at all. Storing a guessed date would be worse than storing nothing.

`DUE_DATE TEXT` carries that confirmed date once it exists. It is `TEXT` because SQLite has no dedicated date type, dates are conventionally stored as text strings. No `NOT NULL` means `NULL` is allowed, and here `NULL` means not yet confirmed.

`COMPLETED_AT TEXT` and `COMPLETION_METHOD TEXT` are also nullable, and for these two the reason needs no comment. A pending task has not been completed, so it cannot have a completion time or a method. These columns are meaningful only once `STATUS` reflects finished work, and until then `NULL` is the correct value.

The next comment announces the most important line in the file. `UNIQUE (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH)` is a table level `UNIQUE` constraint spanning four columns, meaning that exact combination may appear at most once. This is the core product rule enforced as physics rather than convention. Whatever bug appears upstream, a task generator running twice, a retried request, a race between two writers, the database will refuse the second task for the same client, service, and month. Without it, a duplicate task could be marked done while its twin stayed pending, and the practitioner would chase work that was already finished.

The three `FOREIGN KEY` clauses anchor the pointers. `FOREIGN KEY (CLIENT_ID) REFERENCES CLIENTS(ID)` and `FOREIGN KEY (SERVICE_ID) REFERENCES SERVICES(ID)` refuse tasks pointing at nonexistent clients or services, and refuse deleting a client or service that still has tasks. `FOREIGN KEY (PERIOD_YEAR, PERIOD_MONTH) REFERENCES PERIODS(YEAR, MONTH)` is a composite foreign key, two columns referencing a two column key together, and it means a task can only exist for a month that has a row in `PERIODS`. You cannot log work against February 2031 unless February 2031 has been created as a period first, and you cannot delete a period that still has tasks hanging from it.

documents

```
CREATE TABLE documents (  
  id INTEGER PRIMARY KEY,  
  -- every document belongs to a client, search by client depends on this  
  client_id INTEGER NOT NULL,  
  filename TEXT NOT NULL,  
  path TEXT NOT NULL,  
  FOREIGN KEY (client_id) REFERENCES CLIENTS(ID)  
);
```

Why it exists. TaxDesk clients come with paperwork, files sitting in their folders on disk. This table is the database's index of those files, so the app can answer questions like which documents belong to this client without walking the filesystem every time.

How it works. One thing to notice before the columns, this table is written in lowercase while every table above it is uppercase. SQLite treats identifiers case insensitively, so `documents` and `DOCUMENTS` name the same table and nothing behaves differently. It is a style inconsistency in the file, not a semantic one, and it is worth seeing so you do not go hunting for meaning in it.

`id INTEGER PRIMARY KEY` is the rowid alias idiom once more. `client_id INTEGER NOT NULL` links each document to its owner, and the comment above it says why the constraint matters, the app's search by client feature is only as reliable as this column, an orphan document with no client would be invisible to every client scoped query. `NOT NULL` makes such an orphan impossible to insert.

`filename TEXT NOT NULL` stores the file's name and `path TEXT NOT NULL` stores where it lives, both required. Neither carries a `UNIQUE` constraint, so as written the schema permits two rows with the same path, uniqueness of document paths is not something this table enforces.

`FOREIGN KEY (client_id) REFERENCES CLIENTS(ID)` closes the loop. Every document must point at a real client, and a client with documents cannot be deleted out from under them.

That closing `);` is also the end of `001_schema.sql`. You have now read every line of it.

SETTINGS

The second migration file, `002_settings.sql`, is short enough to show whole, its comment block and its one table.

```
-- Application configuration, exactly one row by design. The CHECK on
-- ID makes a second row impossible. Configuration here is a typed
-- declaration of the values the product actually has, adding a future
-- value is a migration adding a column, never a loose key value row.
-- No row at all means the app is not configured yet, onboarding
-- creates the row when the practitioner picks the root folder.
CREATE TABLE SETTINGS (
    ID INTEGER PRIMARY KEY CHECK (ID = 1),
    ROOT_FOLDER TEXT NOT NULL
);
```

Why it exists. The app has configuration that applies to the whole installation, starting with the root folder that onboarding scans for clients. Configuration is not a list of things, it is one thing, so the table is designed so that it physically cannot hold two competing configurations. The comment block spells out three decisions and each deserves attention.

How it works. The first two comment lines state the design goal, exactly one row, and name the mechanism, the `CHECK` on `ID`. The middle lines reject the popular alternative design. Many apps store settings as a key value table, rows like `('root_folder', '/some/path')`, where every value is untyped text and any typo creates a new phantom setting. This comment forbids that pattern for TaxDesk. Here every setting is a real column with a real type and a `NOT NULL` rule if it needs one, so adding a future setting means writing a new migration that adds a column, and the schema always states exactly which settings exist. The last two lines define what an empty table means, the app is not configured yet, and name who fills it, onboarding, when the practitioner picks the root folder. That turns the row's absence into a meaningful state the app can test for, rather than an error.

Now the trick itself. `ID INTEGER PRIMARY KEY CHECK (ID = 1)` combines two constraints that are individually ordinary. `INTEGER PRIMARY KEY` guarantees every row has a distinct ID, no two rows may share one. `CHECK (ID = 1)` guarantees every row's ID is exactly 1, any other value is refused. Put them together and follow the logic. Every row must have a different ID, and every ID must be 1, so there can be at most one row. A second insert either supplies ID 1 and collides with the primary key, or supplies anything else and fails the `CHECK`, or supplies nothing and gets an

autoassigned ID that is not 1 once row 1 exists, failing the CHECK again. Every path to a second row is blocked by the engine itself, no application code needs to remember the rule. This is the standard SQLite idiom for a singleton table, a table meant to hold exactly zero or one row.

`ROOT_FOLDER TEXT NOT NULL` is the first and so far only setting, the path to the folder onboarding scans. `NOT NULL` means you cannot create the settings row without it, so the row existing and the app being configured are the same fact. There is no half configured state where the row exists but the folder is blank.

Why These Files Are Frozen

A migration file is a message to every copy of the database, past and future, and the numbering is what keeps them all in step. The migration runner walks the files in order, applies each one inside a transaction, and records which numbers have already run, so a file that has been applied is never applied again.

That recording is exactly why an applied file must never be edited. Suppose `001_schema.sql` has run against your database, and you then edit it to add a column. Your database has the file marked as done, so the edit never executes there, while a colleague setting up fresh tomorrow gets the new column. Two databases now believe they followed the same file and hold different schemas, and nothing in the system can tell them apart. The bug this creates is the worst kind, it lives in the gap between machines and reproduces on none of them individually.

So the rule is absolute. Once a numbered file has been applied anywhere that matters, it is frozen, history rather than source. Every schema change, adding a column, adding a table, tightening a constraint, is a new file with the next number, `003_something.sql`, which every database can apply from wherever it currently stands. The file you read at the top of this chapter is not the current schema so much as the first entry in an append only log of schemas, and `002_settings.sql` is the second. Reading them in order replays the database's entire history, which is exactly the point.

5.2 database/migrate.py, The Runner

This file is the single door through which the TaxDesk database comes into existence and changes shape. It opens SQLite connections with the right settings, applies numbered `.sql` migration files exactly once each, remembers what it has already applied in a logbook table, and guarantees a handful of reference rows that the product cannot live without. Every other part of the system, the FastAPI routes included, is supposed to get its connection from here rather than calling `sqlite3.connect` on its own. The file is small, under 140 lines, but it makes two subtle promises, that running it twice is always safe, and that a migration and its logbook record land together or not at all. Most of this chapter is about how those two promises are kept.

The Module Docstring

Why it exists. A docstring is a string literal placed at the very top of a module, class, or function, and Python stores it as documentation for that object. This one is the contract for the whole file, and everything below it is an implementation of these sentences. If you read nothing else in the file, these eleven lines tell you how to run it and what it will never do.

How it works.

```
"""Initialize the TaxDesk database from database/migrations/*.sql.
```

```
Usage:
```

```
python3 database/migrate.py [path/to/database.db]
```

```
Migrations are numbered files applied in filename order, each exactly  
once per database. Safe to run any number of times. An existing  
database is never recreated or deleted, the runner records what it  
has applied and skips it on every later run. Applied files are  
frozen, a schema change is always a new file.  
"""
```

The first line names the job, turn the SQL files under `database/migrations/` into a live database. The `Usage` block shows the command line form, and the square brackets around the path mean the argument is optional, a convention borrowed from Unix manual pages. We will see in the `__main__` block exactly what happens when you leave it off.

The final paragraph makes four promises. Migrations run in filename order, which is why the files are numbered. Each file runs exactly once per database, so the runner must remember what it has done. Running the script again is harmless, a property usually called idempotence, meaning an operation you can repeat without changing the result beyond the first run. And applied files are frozen, so if you need to change the schema you write a new file rather than editing an old one. That last rule matters because a file that already ran will be skipped forever, so editing it would silently do nothing on existing databases while doing something on fresh ones, and the two would drift apart.

Imports and the Three Path Constants

Why it exists. The runner needs a regular expression engine, the SQLite driver, access to command line arguments, and a clean way to talk about file paths. It also needs to know where its own files live, no matter what directory you launch it from, and that is what the three constants solve.

How it works.

```
import re  
import sqlite3  
import sys  
from pathlib import Path  
  
DATABASE_DIR = Path(__file__).resolve().parent  
MIGRATIONS_DIR = DATABASE_DIR / "migrations"  
DEFAULT_DB_PATH = DATABASE_DIR / "taxdesk.db"
```

The imports are all standard library. `re` is Python's regular expression module, used later to check migration filenames. `sqlite3` is the built in driver for SQLite, the file based database this project uses. `sys` gives access to `sys.argv`, the list of command line arguments. `Path` from `pathlib` is an object that represents a filesystem path and knows how to join, resolve, and inspect itself, which is far less error prone than gluing strings together.

Now the anchoring trick. `__file__` is a variable Python sets in every module, holding the path of the source file itself, here the path of `migrate.py`. The catch is that this path can be relative to wherever you started Python, so if you launch the script from the project root it might read `database/migrate.py`, and from inside the folder just `migrate.py`. Calling `.resolve()` turns it into an absolute path, one that starts from the filesystem root and is unambiguous. Then

`.parent` chops off the filename, leaving the directory that contains `migrate.py`, which is the `database/` folder. So `DATABASE_DIR` is always the real `database/` directory regardless of the current working directory when the script runs. This is the standard way to make a script location independent.

The next two lines use the `/` operator, which `Path` overloads to mean join a path segment. `MIGRATIONS_DIR` is the `migrations` folder sitting next to this file, the place `initialize` will scan for `.sql` files. `DEFAULT_DB_PATH` is `taxdesk.db` in the same folder, the database file used when the caller does not name one. All three are module level constants, written in capital letters by Python convention to signal that nothing should reassign them.

The `REQUIRED_SERVICES` Constant

Why it exists. Some rows are not user data and not schema, they are reference data the application logic depends on. TaxDesk is a tax filing tool, and a filing must belong to a service such as GSTR-3B. A database without these rows would be structurally valid and functionally useless. This constant is the one place that list lives.

How it works.

```
# Required reference data, not structure and not fake seed data. The
# product is meaningless without these filing types. Ensured on every
# initialize call, so adding a name here reaches databases that were
# initialized before it existed, without touching schema.sql.
REQUIRED_SERVICES = ["GSTR-3B", "GSTR-1", "EPF", "ESI"]
```

The comment is doing real work, so read it slowly. It draws a line between three kinds of database content. Structure is tables and columns, and that belongs in the migration files. Fake seed data is sample rows you might insert to demo an app, and this is explicitly not that. Reference data is the third category, real rows the product requires to function, and those four filing types, two GST return forms and two payroll schemes, are exactly that.

The second half of the comment explains the delivery mechanism. These names are ensured on every `initialize` call, not inserted once by a migration. That choice has a payoff, if a fifth service is added to this list next year, every existing database picks it up the next time `initialize` runs, with no new migration file and no edit to the schema. We will see the insert that makes this safe to repeat when we reach `initialize`, and it is worth noticing now that the comment says ensured, not inserted, which hints that the operation must tolerate the row already existing.

`connect`

Why it exists. SQLite has a few defaults that are wrong for this project, and this function is where they get corrected, once, for everybody. The docstring calls it the only sanctioned way to open the database, which turns a scattering of driver settings into a single enforced policy. Any code that bypasses it silently loses foreign key enforcement, which is exactly the kind of bug that stays invisible until data is already corrupt.

How it works.

```
def connect(db_path: Path = DEFAULT_DB_PATH) -> sqlite3.Connection:
    """Open the database, creating the file when it does not exist.

    In: the database file path, defaults to database/taxdesk.db.
    Out: an open connection with foreign key enforcement on.
```

SQLite keeps foreign keys off per connection unless asked, so this function is the only sanctioned way to open the database. Code that calls `sqlite3.connect` directly loses that enforcement.

Invariant: connections are short lived and scoped to one request or one script run, never shared between requests. `check_same_thread` is off because FastAPI may create and use a request's connection on different thread pool threads, which is safe exactly because of that invariant, one connection never serves two requests at once.

"""

```
conn = sqlite3.connect(db_path, check_same_thread=False)
# Rows readable by column name everywhere, row["NAME"], while index
# access keeps working for existing callers.
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")
return conn
```

Start with the signature. The parameter carries a type annotation, `db_path: Path`, and a default value, `DEFAULT_DB_PATH`, so calling `connect()` with no arguments opens the standard database file. The `-> sqlite3.Connection` part annotates the return type. Annotations are contracts stated at the boundary, they do not change runtime behavior but they tell both the reader and the type checker what goes in and what comes out.

The docstring's `In` and `Out` lines restate that contract in prose, and the line about creating the file is a fact about SQLite itself, `sqlite3.connect` creates an empty database file when the path does not exist yet. That is why a brand new checkout can run this script and end up with a working database.

Now the three settings, which are the whole point of the function.

- `check_same_thread=False`: by default the `sqlite3` driver raises an error if a connection created on one thread is used from another. FastAPI runs synchronous route code on a thread pool, so the thread that creates a request's connection may not be the thread that later uses it. Turning the check off allows that. The docstring is careful to say why this is safe, and this is the invariant it names. Every connection is short lived and belongs to exactly one request or one script run, and it is never shared between two requests. Thread safety problems come from concurrent use of one connection, not from sequential use across threads, so as long as the invariant holds, disabling the check is sound. Break the invariant, say by caching a connection globally, and this setting stops being a convenience and becomes a data race.
- `conn.row_factory = sqlite3.Row`: a row factory is the function the driver uses to build each result row. The default gives you plain tuples, so you read columns by position, `row[0]`, and a reordered `SELECT` silently changes meaning. `sqlite3.Row` returns rows that support access by column name, `row["NAME"]`, while still supporting index access, which is what the inline comment means about existing callers, code written against tuple indexing keeps working unchanged.
- `PRAGMA foreign_keys = ON`: a `PRAGMA` is a SQLite specific command for changing engine settings. A foreign key is a column that must match a row in another table, for example a filing row pointing at the client it belongs to. SQLite parses these constraints but, for historical compatibility, does not enforce them unless each connection asks, and the setting resets with every new connection. This line asks. Without it you could delete a client and leave its filings pointing at nothing, and no error would ever fire.

The function ends by returning the configured connection. Notice what it does not do, it does not open a transaction, run migrations, or close anything. It has one job, produce a correctly configured connection, and the caller owns the connection's lifetime.

apply_one

Why it exists. Applying a migration is really two writes, run the file's SQL and insert a row into the logbook saying it ran. If the first happens without the second, the next run applies the file again and likely explodes on a duplicate table. If the second happens without the first, a migration is marked done that never ran. Both writes must live inside one transaction, and a quirk of the sqlite3 driver makes that harder than it sounds, which is what this function's design exists to defeat.

How it works.

```
def apply_one(conn: sqlite3.Connection, migration: Path) -> None:
    """Apply a single migration file and record it, atomically.

    In: an open connection and the path of one .sql migration file.
    Out: nothing. Either the migration AND its logbook record are both
    committed, or neither is.

    executescrpt() commits on its own, proven by test, so driver level
    commit handling cannot make the pair atomic. Instead the script
    itself carries BEGIN and COMMIT, one real SQLite transaction wraps
    the migration and its record together. Migration files must not
    manage transactions themselves.
    """

    # The filename is spliced into the script because placeholders do
    # not exist inside executescrpt. Names come from our own repo
    # directory, the allowlist check is defense in depth.
    if not re.fullmatch(r"[A-Za-z0-9._-]+", migration.name):
        raise ValueError(f"unsafe migration filename: {migration.name}")

    body = migration.read_text().strip()
    if not body.endswith(";"):
        body += ";"

    script = (
        "BEGIN;\n"
        f"{body}\n"
        f"INSERT INTO schema_applied (filename) VALUES ('{migration.name}');\n"
        "COMMIT;"
    )

    try:
        conn.executescript(script)
    except Exception:
        if conn.in_transaction:
            conn.rollback()
        raise
```

First, the trap the docstring names. A migration file contains many statements, so you cannot feed it to `conn.execute`, which runs one statement at a time. The driver's tool for multi statement SQL is `conn.executescript`. The trap is that `executescript` commits any pending transaction before it starts and effectively autocommits as it goes, so the usual driver pattern, run the work, then call `conn.commit()` or `conn.rollback()`, cannot bind the migration and its logbook insert into one atomic unit. By the time your Python code gets control back, the migration's statements may already be committed on their own. The docstring adds proven by test, meaning this behavior was verified against the real driver rather than assumed from documentation, and that phrase is the receipt.

The workaround is to stop asking the driver for a transaction and ask SQLite directly. `BEGIN` and `COMMIT` are SQL statements that open and close a transaction inside the database engine itself, so the function builds one text script that reads `BEGIN`, then the whole migration, then the logbook insert, then `COMMIT`, and hands the entire thing to `executescript`. Now the atomicity lives in the SQL, where the driver cannot interfere. This is also why the docstring forbids migration files from managing transactions themselves, a stray `COMMIT` inside a migration would close the wrapper's transaction early and split the pair apart.

Walk the body in order. The first statement is a guard clause, a check at the top of a function that rejects bad input immediately so the happy path below stays flat. `re.fullmatch` returns a match only when the entire string fits the pattern, unlike `re.match` which is satisfied by a matching prefix. The pattern `[A-Za-z0-9._-]+` is an allowlist, one or more characters drawn only from letters, digits, dot, underscore, and hyphen. Why does a filename need vetting at all? Look ahead to the script construction, the filename is spliced directly into SQL text inside single quotes. Normally you would pass values through placeholders, the `?` markers that let the driver escape values safely, but as the comment says, placeholders do not exist inside `executescript`, it accepts only raw text. A filename containing a single quote could therefore break out of the string literal and inject SQL. The comment is honest about the threat level, these names come from the project's own repository, not from users, so the check is defense in depth, a second lock on a door that should already be locked. If the check fails, the function raises `ValueError` with an f-string, a string literal prefixed with `f` in which the expression inside `{}` is evaluated and inserted into the text.

Next the file is read. `migration.read_text()` is a `Path` method that opens the file, reads it fully as text, and closes it, all in one call. `.strip()` removes leading and trailing whitespace, including any trailing newline. Then a small normalization, if the body's last character is not a semicolon, one is appended. This matters because the body is about to be embedded between other statements, and SQL statements are separated by semicolons. A migration file whose author forgot the final semicolon would otherwise fuse its last statement with the `INSERT` that follows, and this two line fix removes that whole class of error.

The `script` assignment uses implicit string concatenation, adjacent string literals inside parentheses that Python merges syntactically into one expression, a tidy way to build multi line text. Read the four pieces top to bottom, `BEGIN;` opens the transaction, the migration body runs, the `INSERT INTO schema_applied` records the filename in the logbook, and `COMMIT;` seals all of it as one unit. Two of the pieces are f-strings, so the joining of the fragments is pure syntax while the `{body}` and `{migration.name}` values inside them are filled in at runtime, and only when this line executes does the final script text exist. The logbook insert only names the `filename` column because, as we will see in `initialize`, the table's other column fills itself in with a default timestamp.

Finally the try block. `conn.executescript(script)` sends the whole script to SQLite. If anything inside fails, a broken statement in the migration, a duplicate filename in the logbook, the `except Exception` clause catches it. `conn.in_transaction` is a driver attribute that is true when the connection has an open transaction, which happens here when the script died after `BEGIN` but before `COMMIT`. In that case `conn.rollback()` discards every change since `BEGIN`, so the failed migration leaves no partial work behind, no half created tables and no logbook lie. Then the bare `raise` re-raises the same exception, unchanged, to the caller. This is the disciplined shape of error handling, clean up your own state, then let the failure travel upward loudly. Swallowing the exception here would report a broken migration as success, which is the worst outcome available.

initialize

Why it exists. This is the orchestrator, the function that turns the docstring's promises into behavior. It makes sure the logbook table exists, finds out which migrations have already run, applies the rest in order, and ensures the required service rows. Everything about it is built to be safe to call repeatedly, on a fresh file, on a fully current database, or anywhere in between.

How it works.

```
def initialize(conn: sqlite3.Connection) -> bool:
    """Apply pending migrations in order, then ensure required services.

    In: an open connection from connect().
    Out: True when at least one migration was applied by this call,
    False when the database was already current. Existing data is
    never touched, and the required services are ensured on every
    call, so new entries in REQUIRED_SERVICES reach already
    initialized databases too.
    """
    conn.execute(
        "CREATE TABLE IF NOT EXISTS schema_applied ("
        " filename TEXT PRIMARY KEY,"
        " applied_at TEXT NOT NULL DEFAULT (datetime('now'))"
        ")"
    )

    already_applied = {
        row[0]
        for row in conn.execute("SELECT filename FROM schema_applied")
    }

    applied = False
    for migration in sorted(MIGRATIONS_DIR.glob("*.sql")):
        if migration.name in already_applied:
            continue

        apply_one(conn, migration)
        applied = True

    for name in REQUIRED_SERVICES:
        conn.execute(
            "INSERT OR IGNORE INTO SERVICES (NAME) VALUES (?)",
            (name,),
        )
    conn.commit()

    return applied
```

The first statement creates the logbook. `schema_applied` is a table with two columns, `filename`, which is the primary key so the same file can never be recorded twice, and `applied_at`, a timestamp that defaults to `datetime('now')`, meaning SQLite fills in the current time when a row is inserted without one. That default is why `apply_one` could insert only the filename. The `IF NOT EXISTS` clause makes the statement a no-op when the table already exists, which is the first of several idempotence tricks in this function, and it also solves a bootstrap puzzle, the table that tracks migrations cannot itself be created by a migration, because you would need the table to know whether that migration ran.

Next the function loads the logbook into memory. The expression inside the braces is a set comprehension, a one expression loop that builds a set, which is a collection with no duplicates and fast membership testing. Iterating over `conn.execute(...)` yields the result rows one at a time, and `row[0]` takes the first column of each, the filename, using the index access that `sqlite3.Row` still supports. The result is `already_applied`, the set of filenames that have ever been applied to this database, so the loop below can check membership with a quick `in` instead of querying the database per file.

Then the migration loop. `MIGRATIONS_DIR.glob("*.sql")` is a `Path` method that finds files matching a pattern, here every file ending in `.sql` inside the migrations folder, and `glob` is the old Unix name for this kind of wildcard matching. Its order is not guaranteed, so the call is wrapped in `sorted()`, which orders the paths by name, and since migration files are numbered, name order is application order. This one word, `sorted`, is what implements the docstring's promise that migrations run in filename order. Inside the loop, a guard with `continue` skips any file whose name is already in the logbook, jumping straight to the next iteration. Files that survive the guard go to `apply_one`, which runs and records them atomically as we saw, and the `applied` flag flips to `True` to remember that this call did real work. If `apply_one` raises, the exception flies out of `initialize` too, so a broken migration stops the run rather than being skipped.

Now look at where the services loop sits, after the migration loop, outside it, not inside. That placement is the mechanism behind the docstring and the constant's comment. If the inserts lived inside the migration loop, they would only run when some migration was pending, and on an already current database the loop body never executes. Sitting outside, they run on every single call, current database or not, and that is what makes a new name added to `REQUIRED_SERVICES` reach databases initialized long before the name existed. Inside the loop, `INSERT OR IGNORE` is SQLite's way of saying insert this row, and if a constraint such as a unique name rejects it, do nothing instead of raising. That is what turns insert into ensure, running it against a database that already has the row is a silent no-op. The value arrives through a `?` placeholder with the arguments passed as `(name,)`, a tuple containing one element, written with the trailing comma that distinguishes a one element tuple from a parenthesized expression. Placeholders work here because this is `execute`, not `executescript`, so the safe path exists and the code takes it. After the loop, `conn.commit()` writes the inserts to disk, and this commit belongs to the service inserts, since each migration already committed itself inside its own script.

The return value is the `applied` boolean, and its meaning is precise. `True` means this call applied at least one migration, `False` means the database was already current. It does not mean success or failure, failure is an exception. The distinction exists so a caller can react to the difference, and the `__main__` block below uses it to print an honest message.

The main Block

Why it exists. The module is both a library and a command. The functions above can be imported by the application, and this block is what runs when someone types `python3 database/migrate.py` in a terminal, wiring the command line to those same functions with no duplicated logic.

How it works.

```
if __name__ == "__main__":
    db_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_DB_PATH
    conn = connect(db_path)
    try:
        applied = initialize(conn)
        print("schema applied" if applied else "already initialized, nothing to do")
```

```
finally:
    conn.close()
```

The guard itself first. Python sets the variable `__name__` to the string `"__main__"` only in the file that was launched directly, while an imported module sees its own module name instead. So this block runs for `python3 database/migrate.py` and stays inert when the application does `from database.migrate import connect`. This is the standard Python idiom for a file that is both importable and executable.

The first line inside picks the database path with a conditional expression, Python's one line if else that selects between two values. `sys.argv` is the list of command line arguments, where index 0 is the script's own name and index 1 is the first real argument. When the user supplied a path, `len(sys.argv) > 1` is true and that argument is wrapped in `Path`. When they did not, the fallback is `DEFAULT_DB_PATH`, which is the optional bracket in the docstring's usage line made real. Being able to pass a path is what lets tests run migrations against a throwaway file without touching the real `taxdesk.db`.

Then the two functions we already know, `connect` opens the configured connection and `initialize` does the work, inside a try block. The `print` uses the same conditional expression idiom to translate the boolean return into one of two honest messages, `schema applied` when this run changed something, `already initialized`, `nothing to do` when it did not. That second message is the docstring's safe to run any number of times promise, visible in the terminal.

The `finally` clause runs whether `initialize` succeeded or raised, and it closes the connection either way. This is the manual spelling of the cleanup a context manager, the `with` statement that guarantees teardown, would give you, and it honors the invariant from `connect`, connections are short lived and scoped to one script run. Note what is deliberately absent, there is no `except`. A failed migration escapes as a raw traceback and a nonzero exit code, which is exactly what you want from a command line tool, scripts and humans alike can see that something went wrong and read precisely what.

5.3 database/queries.py, Every SQL Statement The App Runs

This file is the app's vocabulary of feature SQL, collected in one place as four small Python functions. Route handlers never write feature SQL themselves, they call these functions, with two small exceptions that live elsewhere and do no feature work. The health route in `app/routes/health.py` runs a bare `SELECT 1` as a connectivity probe, and `database/migrate.py` owns the schema side, where it switches on `PRAGMA foreign_keys`, creates and reads its `schema_applied` logbook, executes the migration `.sql` files, and ensures the required service rows with its own `INSERT OR IGNORE`. So if you want to know what the database can be asked to do on behalf of a feature, this file is the answer. Two functions manage the single settings row that stores the root client folder, and two manage the `CLIENTS` table that the onboarding flow fills. Every function takes an open `sqlite3.Connection` as its first argument, because opening and closing connections is someone else's job, handled once per request elsewhere in the app. Keeping the feature SQL here means a reviewer can audit every statement in under a minute, and a route handler stays free of database details.

The Module Docstring and the Single Import

Why it exists. The docstring at the top of a module is the contract for the whole file, and this one states two rules that shape everything below it. First, feature SQL lives here. The docstring puts it more absolutely than the codebase does, since the health route's `SELECT 1` probe and the migration runner's own statements sit outside this file, so read the rule as

covering the SQL that features run. Second, the file only contains functions that a shipped feature actually calls, so you will not find speculative helpers waiting for a future that may never come.

```
"""Every SQL statement the application runs, as small typed functions.

Routes call these and never write SQL themselves. Only the functions a
shipped feature genuinely needs live here, nothing speculative.
"""

from sqlite3 import Connection, Row
```

How it works. The triple quoted string on the first line is a module docstring, a plain string that Python attaches to the module as documentation rather than executing as code. It promises small typed functions, and every function below honors that with full type annotations on parameters and return values.

The single import line pulls two names out of Python's standard `sqlite3` library. `Connection` is the type of an open database connection, the object you call `.execute()` on to run SQL. `Row` is a special row type that behaves like a tuple but also lets you look up columns by name, and we will see it in action shortly. Importing just these two names, instead of the whole module, keeps the file honest about exactly which pieces of `sqlite3` it depends on. Notice there is no `import sqlite3` and no code that opens a database file here. This module never creates connections, it only receives them.

get_root_folder, Reading the One Settings Row

Why it exists. The app stores exactly one piece of configuration, the path to the root folder where client subfolders live. Onboarding needs to ask a simple question at startup, has the user configured that folder yet. This function answers it, and its return value doubles as the answer, a string means configured, `None` means not yet.

```
def get_root_folder(conn: Connection) -> str | None:
    """Read the configured root client folder.

    In: an open connection.
    Out: the stored path, or None when onboarding never saved one,
    which is the app's definition of not configured yet.
    """
    row = conn.execute("SELECT ROOT_FOLDER FROM SETTINGS WHERE ID = 1").fetchone()
    return row["ROOT_FOLDER"] if row else None
```

How it works. Start with the signature. The parameter `conn: Connection` says the caller must hand in an already open connection. The return annotation `str | None` uses Python's union syntax, the vertical bar means the function returns either a string or `None`, and the docstring tells you which case means what. Absence of a row is not an error here, it is a meaningful state, the app has never been set up.

The body is two lines. The first line chains three operations. `conn.execute(...)` sends the SQL text to SQLite and returns a cursor, an object that holds the results of a query. `.fetchone()` asks that cursor for the first result row, and it returns either one row or `None` if the query matched nothing. The SQL itself selects one column, `ROOT_FOLDER`, from the `SETTINGS` table, filtered to `WHERE ID = 1`. The settings table is designed to hold at most a single row whose ID is always 1, which is a common trick for storing app wide configuration in a relational database, you pin the row to a known key so there is exactly one place to look.

The second line is a conditional expression, Python's one line if. Read it right to left, if `row` is truthy, meaning a row came back, return `row["ROOT_FOLDER"]`, otherwise return `None`.

Now the part that trips up newcomers, why does `row["ROOT_FOLDER"]` work at all. By default, `sqlite3` returns each row as a plain tuple, so you would have to write `row[0]` and remember which position holds which column. `sqlite3.Row` is the standard library's answer to that. It is a row class that still supports numeric indexing but also supports lookup by column name, so `row["ROOT_FOLDER"]` fetches the column called `ROOT_FOLDER` regardless of its position in the `SELECT` list. A connection produces `Row` objects once its `row_factory` attribute has been set to `sqlite3.Row`, an assignment that can happen at any point in the connection's life and affects every row fetched after it. That setup does not happen in this file, it happens in `connect()` in `database/migrate.py`, which sets the attribute right after opening each connection, and the type annotations here, like the `list[Row]` you will meet in `list_clients`, are written on the assumption that every connection arrived through that door. Name based access is worth the setup, because if someone later reorders columns in the `SELECT`, code that reads by name keeps working while code that reads by position silently grabs the wrong value.

set_root_folder, an Upsert Explained Clause by Clause

Why it exists. Saving the root folder has two possible situations. The first time the user completes onboarding, no settings row exists and one must be inserted. Every later time, the row already exists and must be updated in place. Writing that as two statements plus an if would work, but SQLite offers a single statement that handles both, and this function uses it.

```
def set_root_folder(conn: Connection, root_folder: str) -> None:
    """Save or replace the root client folder, the single settings row.

    In: an open connection and a validated directory path.
    Out: nothing, the one allowed row is created or updated in place.
    """
    conn.execute(
        "INSERT INTO SETTINGS (ID, ROOT_FOLDER) VALUES (1, ?)"
        " ON CONFLICT(ID) DO UPDATE SET ROOT_FOLDER = excluded.ROOT_FOLDER",
        (root_folder,),
    )
```

How it works. The signature takes the connection and the new path, and returns `None`, annotated explicitly so the reader knows there is no value to collect. The docstring notes the path is already validated, meaning the route that calls this has checked the directory exists before the value ever reaches SQL. This function trusts its caller on that point and does one job only.

The body is a single `conn.execute` call, but the SQL inside is the most interesting statement in the file, an upsert. Upsert is a blended word, update plus insert, and it names a statement that inserts a row when the key is new and updates the existing row when the key is already taken. Take it clause by clause.

- `INSERT INTO SETTINGS (ID, ROOT_FOLDER) VALUES (1, ?)`: this is the optimistic half. It tries to insert a brand new row with ID 1 and the given path. On a fresh database this succeeds and the statement is done.
- `ON CONFLICT(ID)`: this clause tells SQLite what to do if the insert collides with an existing row. A conflict happens when the new row would violate a uniqueness rule, here the primary key on ID, because a row with ID 1 already exists. The column in parentheses names which uniqueness rule we expect to trip, so SQLite knows this handler applies to collisions on ID specifically.

- `DO UPDATE SET ROOT_FOLDER = excluded.ROOT_FOLDER`: instead of failing with an error, SQLite falls back to updating the row it collided with. The SET part says which column to change and what to change it to.
- `excluded.ROOT_FOLDER`: this is the piece that looks like magic until someone names it. During an upsert, SQLite gives you a phantom table called `excluded` that holds the row you tried to insert, the one that got excluded because of the conflict. So `excluded.ROOT_FOLDER` means the `ROOT_FOLDER` value from my attempted insert, in other words the new path the user just chose. The clause reads as, take the value I tried to insert and write it over the old one.

The net effect is exactly what the docstring promises, the one allowed row is created if missing and overwritten if present, in one atomic statement with no race between checking and writing.

Two Python details in this call are worth naming. The SQL is written as two string literals sitting next to each other, one per line. Python joins adjacent string literals into a single string at compile time, which is a clean way to break a long statement across lines without concatenation operators. The space at the start of the second literal, before `ON CONFLICT`, keeps the joined statement readable. Without it the two lines would fuse into `...VALUES (1, ?)ON CONFLICT...`, which SQLite would in fact still parse, because the closing parenthesis is a token on its own, but which reads badly to any human scanning the statement. Putting the space in every time is the safer habit anyway, since between two words a missing space really would glue them into one broken keyword. The second detail is `(root_folder,)`, a tuple containing one element. The trailing comma is what makes it a tuple, `(root_folder)` without the comma is just a parenthesized string, and `execute` requires a sequence of parameters, one value for each `?` in the SQL.

list_clients, Fetching Every Client Row

Why it exists. During onboarding the app shows the user the folders inside the root directory, and it needs to mark which of them are already registered as clients. To do that it needs the full client list, including each client's folder path for the comparison. This function fetches that list in a stable order.

```
def list_clients(conn: Connection) -> list[Row]:
    """List every client, for marking folders as already added.

    In: an open connection.
    Out: client rows with ID, NAME, FOLDER_PATH, ordered by name.
    """
    return conn.execute(
        "SELECT ID, NAME, FOLDER_PATH FROM CLIENTS ORDER BY NAME",
    ).fetchall()
```

How it works. The return annotation is `list[Row]`, a list of the same `sqlite3.Row` objects we met earlier, so a caller can write `client["FOLDER_PATH"]` on each element rather than counting column positions.

The SQL selects three named columns, `ID`, `NAME`, and `FOLDER_PATH`, from the `CLIENTS` table. Naming the columns explicitly, instead of writing `SELECT *`, is a deliberate habit. It documents in the query itself exactly which fields the callers rely on, and it keeps the result stable even if the table later grows extra columns. `ORDER BY NAME` sorts the rows alphabetically by client name, so any screen that displays the list gets a predictable, human friendly order for free rather than whatever order the storage engine happens to return.

Where `get_root_folder` used `.fetchone()` to grab a single row, this function ends with `.fetchall()`, which drains the cursor and returns every matching row as a list. An empty table produces an empty list, not `None`, so callers can loop over the result without a guard. Note also the small formatting choice, the SQL string sits on its own line inside the `execute` call with a trailing comma after it, matching the vertical style used across this codebase.

`create_client`, `INSERT OR IGNORE` and the `UNIQUE` Constraint

Why it exists. When the user confirms a folder should become a client, the app inserts a row into `CLIENTS`. But the same folder might be submitted twice, a double click, a page refresh, a repeated request. The app treats registering an already registered folder as a harmless repeat, not an error, and this function encodes that policy directly in the SQL.

```
def create_client(conn: Connection, name: str, folder_path: str) -> None:
    """Create a client unless its folder is already registered.

    In: an open connection, the client name, and its absolute folder
    path.
    Out: nothing. The UNIQUE rule on FOLDER_PATH is the final guard,
    OR IGNORE turns a duplicate into a quiet no-op instead of an error.
    """
    conn.execute(
        "INSERT OR IGNORE INTO CLIENTS (NAME, FOLDER_PATH) VALUES (?, ?)",
        (name, folder_path),
    )
```

How it works. The signature takes the connection, the client's display name, and the absolute path of its folder, and returns nothing. The docstring is unusually load bearing here, it explains the whole duplicate strategy in two sentences, so let us unpack it.

The `CLIENTS` table carries a `UNIQUE` constraint on its `FOLDER_PATH` column. A `UNIQUE` constraint is a rule the database itself enforces, no two rows may ever hold the same value in that column, and any insert that would break the rule is rejected at the database level. That makes the constraint the final guard the docstring mentions. Even if two requests race each other, even if some future code path forgets to check first, the database physically cannot end up with two clients pointing at the same folder.

A plain `INSERT` that hits a `UNIQUE` violation raises an error, which in Python surfaces as an `sqlite3.IntegrityError` exception. That would force every caller to wrap this in `try` and `except` for a situation the app does not consider a failure. `INSERT OR IGNORE` changes the outcome. The `OR IGNORE` conflict clause tells SQLite, if this insert violates a constraint, skip it silently and report success. The row simply is not inserted, no exception is raised, and execution continues. The docstring's phrase for this is a quiet no-op, an operation that completes without doing anything.

The two mechanisms cooperate as a pair. The `UNIQUE` constraint supplies the guarantee, duplicates are impossible. `OR IGNORE` supplies the manners, hitting the guarantee is not treated as a crash. Neither works alone, `OR IGNORE` with no constraint would happily insert duplicates because nothing ever conflicts, and the constraint with a plain `INSERT` would turn every repeat submission into a 500 error. It is worth contrasting this with `set_root_folder`. There, a conflict meant replace the old value with the new one, so the upsert form with `ON CONFLICT DO UPDATE` was right. Here, a conflict means the client already exists and the first registration wins, so nothing should change and `IGNORE` is right. SQLite offers both behaviors, and this file uses each where its meaning fits.

Finally, notice `(name, folder_path)`, a two element tuple supplying one value per `?`, in the same left to right order as the placeholders appear in the SQL.

Why Every Value Travels Through a ? Placeholder

Why it exists. Look back over all four functions and you will find one pattern with no exceptions. Every value that came from outside, a folder path, a client name, arrives in the SQL through a `?` placeholder and a separate parameter tuple. Not one function builds SQL by gluing strings together with an f-string or the `+` operator. An f-string, for reference, is Python's syntax for embedding expressions inside a string literal, like `f"WHERE NAME = '{name}'"`, and it is exactly the tool you must never use for SQL.

How it works. With a placeholder, the SQL text and the data travel to SQLite separately. `conn.execute` sends the statement with its `?` marks as a fixed template, and sends the tuple of values alongside it. SQLite binds each value into its slot as pure data, after the statement's structure is already parsed and locked. Whatever the value contains, quotes, spaces, even text that looks like SQL commands, it can only ever be a value. It has no way to alter what the statement does.

String building does the opposite. If `folder_path` were spliced into the SQL with an f-string, then a path containing a single quote would break the statement's syntax, and a path crafted to contain SQL of its own would run as part of the statement. That failure mode is called SQL injection, one of the oldest and most damaging classes of security bug, where attacker supplied text is executed as commands because the code mixed data into the command channel. Folder names on a user's disk are outside input like any other, so they get no more trust than a web form would.

The placeholder discipline is also why this file is safe to audit quickly. Every statement here is a fixed string literal, visible in full at rest, with the variable parts marked by `?`. None of these functions ever assembles SQL at runtime, so the feature SQL the app runs is exactly the SQL you can read on the page. Four functions, four statements, and together with the health route's `SELECT 1` probe and the migration runner's own statements, that is the whole conversation between the running application and its database. The development seed script, `database/seed.py`, runs inserts of its own, but it is a command line tool for filling a dev database, the app never calls it.

5.4 database/seed.py, The Development Data

Every screen in TaxDesk is built to show clients, subscriptions, and monthly tasks, and none of those screens can be developed against an empty database. This module fills a fresh development database with a small, fixed cast of fake clients so that every page has something real looking to render. It is a command line tool, run once with `python3 -m database.seed`, and it leans on the migration module from the previous section to create the schema before it writes a single row. The whole file is written to be idempotent, meaning you can run it any number of times and the database ends up in exactly the same state as after the first run. That one property shapes nearly every SQL statement in the file, so keep it in mind as we walk through.

The module docstring

Why it exists. A module docstring is a string placed at the very top of a Python file, before any code, and Python stores it as documentation for the whole module. Here it answers the three questions anyone opening the file will ask, what does this do, how do I run it, and is it safe to run again. It also carries a warning that matters in this project, since TaxDesk is

destined for a real accounting office where fake data must never appear.

How it works.

```
"""Fill a development database with sample TaxDesk data.

Usage:
    python3 -m database.seed [path/to/database.db]

Development tool only, never run on a real office machine. Rerunning
is safe, every write is INSERT OR IGNORE or a deterministic UPDATE,
so a second run changes nothing.
"""
```

The first line is the one sentence summary, the convention for docstrings. The `Usage` section shows the module being run with `python3 -m database.seed`, where the `-m` flag tells Python to run a module by its import name rather than by a file path, which keeps the `database` package imports working. The square brackets around `path/to/database.db` are the traditional way of writing an optional command line argument, and we will see in `main` how that optional path is picked up. The last paragraph states the safety contract. `INSERT OR IGNORE` is a SQLite form of insert that silently does nothing when the row would violate a uniqueness rule, and a deterministic `UPDATE` is one that always sets the same rows to the same values. Together those two guarantee the rerun safety the docstring promises.

The imports

Why it exists. The seed script needs SQLite itself, access to command line arguments, a clean way to handle file paths, and three things from the sibling migration module so it never duplicates connection or schema logic.

How it works.

```
import sqlite3
import sys
from pathlib import Path

from database.migrate import DEFAULT_DB_PATH, connect, initialize
```

`sqlite3` is Python's standard library driver for SQLite databases, used here mostly for its `Connection` type in the annotations. `sys` gives access to `sys.argv`, the list of command line arguments. `Path` from `pathlib` is Python's object for representing a filesystem path, nicer to work with than a raw string. The last line imports from `database.migrate`, the module covered earlier in this part of the book. `DEFAULT_DB_PATH` is the standard location of the database file, `connect` opens a connection with the project's settings applied, and `initialize` creates the schema by running the migrations. Reusing those three means the seed script can be pointed at a file that does not exist yet and still work.

SEED_CLIENTS, the sample cast

Why it exists. The five fake clients are the raw material for everything else the script does. They are defined as data at the top of the file rather than buried inside a function, so anyone can see the whole cast at a glance and adjust it without touching logic.

How it works.

```
# Each entry is (name, folder_path, subscribed service names).
SEED_CLIENTS: list[tuple[str, str, list[str]]] = [
    ("Aster Traders", "D:/Clients/Aster Traders", ["GSTR-3B", "GSTR-1", "EPF", "ESI"]),
    ("Bhima Textiles", "D:/Clients/Bhima Textiles", ["GSTR-3B", "GSTR-1"]),
    ("Cauvery Mills", "D:/Clients/Cauvery Mills", ["GSTR-3B", "EPF"]),
    ("Deccan Services", "D:/Clients/Deccan Services", ["ESI"]),
    ("Everest Agencies", "D:/Clients/Everest Agencies", ["GSTR-1", "EPF", "ESI"]),
]
```

The comment above the constant states the tuple shape, and the type annotation restates it in a form the type checker can verify. A tuple is an ordered, fixed length grouping of values, and `list[tuple[str, str, list[str]]]` reads as a list of tuples where each tuple holds a string, another string, and a list of strings. In order those are the client's display name, the Windows style folder path where that client's documents live, and the names of the services the client subscribes to. Notice the paths start with `D:/`, a deliberate nod to the real office machines this system will eventually run on. The service names, `GSTR-3B`, `GSTR-1`, `EPF`, and `ESI`, must match rows in the `SERVICES` table, because the script will look them up by name shortly. Those rows do not come from any migration file. They are inserted by `initialize` in `database/migrate.py`, which loops over its `REQUIRED_SERVICES` list with `INSERT OR IGNORE` on every call, and that module's own comment stresses the reference data lives outside the frozen migration files on purpose, so a service name added later still reaches databases that were initialized before it existed. The subscriptions are deliberately varied. One client takes all four services, one takes a single service, and the rest sit in between, which gives the future pages a realistic spread to display.

SEED_PERIOD and SEED_COMPLETED_AT

Why it exists. Tasks in TaxDesk belong to a period, one calendar month, so the seed needs to pick a month to generate tasks for. It also needs a timestamp for the one task it will mark as done, and that timestamp is the most interesting line in this part of the file.

How it works.

```
SEED_PERIOD = (2026, 8)
```

```
# Fixed on purpose, seed data is openly fake and must be deterministic,
# running the seed twice has to produce byte identical database state.
SEED_COMPLETED_AT = "2026-08-05 10:00:00"
```

`SEED_PERIOD` is a plain tuple of year and month, August 2026, and later functions unpack it with `year, month = SEED_PERIOD`. The more instructive constant is `SEED_COMPLETED_AT`. The natural instinct when marking something as completed is to stamp it with the current time, something like `datetime.now()`. This script refuses that instinct on purpose, and the comment explains why. If the timestamp came from the clock, every run of the seed would write a slightly different value, and the promise that rerunning changes nothing would quietly break. A test comparing two seeded databases would fail on a single timestamp column. By pinning the value to a fixed string, the script stays deterministic, meaning the same inputs always produce the same output, down to the byte. There is a second, softer reason hiding in the comment as well. A completion time of exactly ten in the morning on the fifth is obviously staged, which is a feature for fake data, since nobody will ever mistake it for a real filing.

service_id, name to id lookup

Why it exists. The `SEED_CLIENTS` constant refers to services by human readable name, but the database links tables together by numeric id. This tiny helper bridges the two, translating a name like `GSTR-3B` into the row id the foreign key columns expect.

How it works.

```
def service_id(conn: sqlite3.Connection, name: str) -> int:
    """Look up a service's id by its name.

    In: an open connection and a service name like 'GSTR-3B'.
    Out: the service's row id. Raises if the name does not exist,
    which would mean initialization never ran.
    """
    row = conn.execute(
        "SELECT ID FROM SERVICES WHERE NAME = ?",
        (name,),
    ).fetchone()
    return row[0]
```

The signature is fully annotated, taking an open `sqlite3.Connection` and a `str`, and returning an `int`. The body is a single query. The `?` in the SQL is a placeholder for a parameterized query, the technique where the driver fills values into the statement itself instead of you gluing strings together, which protects against SQL injection and quoting bugs. The value for the placeholder arrives as `(name,)`, a one element tuple, and the trailing comma is what makes it a tuple rather than a plain parenthesized string. `fetchone` asks the cursor for the first result row, or `None` when there is no match. Because every connection in this project comes from `connect` in `database/migrate.py`, which sets `conn.row_factory = sqlite3.Row`, that row arrives as a `sqlite3.Row` object rather than a plain tuple. A `sqlite3.Row` is a tuple like row object, index access such as `row[0]` works exactly as it would on a tuple, and it also allows access by column name. The `None` case is handled by not handling it. If the name is missing, `row[0]` raises a `TypeError`, and the docstring says exactly what that crash means, initialization never ran, so the `SERVICES` table was never filled. For a development tool, a loud crash pointing at a broken setup is more useful than a polite error message, so the function deliberately keeps no safety net.

seed_clients, clients and their subscriptions

Why it exists. This function turns the `SEED_CLIENTS` constant into rows in two tables, `CLIENTS` for the clients themselves and `CLIENT_SERVICES` for which client subscribes to which service. It also plants one deliberately deactivated subscription, giving the rest of the system an off state to render and to prove it skips.

How it works.

```
def seed_clients(conn: sqlite3.Connection) -> None:
    """Insert the sample clients and their service subscriptions.

    In: an open connection on an initialized database.
    Out: nothing, existing rows are left untouched on rerun.
    """
    for name, folder_path, services in SEED_CLIENTS:
        conn.execute(
            "INSERT OR IGNORE INTO CLIENTS (NAME, FOLDER_PATH) VALUES (?, ?)",
```

```

        (name, folder_path),
    )

    client_id = conn.execute(
        "SELECT ID FROM CLIENTS WHERE FOLDER_PATH = ?",
        (folder_path,),
    ).fetchone()[0]

    for service_name in services:
        conn.execute(
            "INSERT OR IGNORE INTO CLIENT_SERVICES (CLIENT_ID, SERVICE_ID)"
            " VALUES (?, ?)",
            (client_id, service_id(conn, service_name)),
        )

    # One switched off subscription, so pages can show that state and
    # generation can prove it skips inactive rows.
    conn.execute(
        "UPDATE CLIENT_SERVICES SET ACTIVE = 0"
        " WHERE CLIENT_ID = (SELECT ID FROM CLIENTS WHERE NAME = 'Bhima Textiles')"
```

```

        " AND SERVICE_ID = (SELECT ID FROM SERVICES WHERE NAME = 'GST-1')",
    )

```

The `for` line uses tuple unpacking, pulling each three part entry of `SEED_CLIENTS` straight into three named variables, `name`, `folder_path`, and `services`, so the loop body never has to index into a tuple by position.

The first statement inserts the client. `INSERT OR IGNORE` is SQLite's way of saying insert this row, but if a uniqueness rule would be violated, do nothing and do not raise an error. On the first run the row goes in, on every later run the existing row blocks it and the statement is a silent no op. That is what makes this function rerunnable.

The second statement immediately reads the client's id back with a `SELECT` by `FOLDER_PATH`. You might wonder why the code does not use `lastrowid`, the id of the most recently inserted row. The answer is the `OR IGNORE` above it. On a rerun nothing is inserted, so there is no fresh row id to read, but the `SELECT` finds the existing row either way. Looking the id up by the same unique column the insert relied on works identically on the first run and the fiftieth. The `.fetchone()[0]` chain grabs the first result row and then its first column in one expression.

The inner loop walks the client's subscribed service names and inserts one `CLIENT_SERVICES` row per subscription, again with `INSERT OR IGNORE` so reruns pass through harmlessly. Notice the SQL string is written as two adjacent string literals, which Python joins into one string at compile time, a common way to keep long SQL readable, and notice the leading space on the second piece, without which the words `SERVICE_ID)VALUES` would collide. The values are the client id we just looked up and a call to `service_id`, the helper from the previous section, translating the service name into its id inline.

After the loop, one final `UPDATE` runs outside it, and the comment carries its reason. The statement flips the `ACTIVE` flag to `0` on exactly one subscription, Bhima Textiles' GST-1. Rather than carrying ids around, it locates the row with two subselects, a subselect being a small query nested in parentheses inside a larger one, one resolving the client name to an id and one resolving the service name to an id. The seeded world now contains a switched off subscription, which future pages can display and, more importantly, which the task generation in the next function must be seen to skip.

seed_period_and_tasks, the INSERT fed by a SELECT

Why it exists. With clients and subscriptions in place, the seed creates the sample month and then generates one task per active subscription for that month. This is a rehearsal of the real monthly workflow the office will eventually run, a new period opens and the system fans it out into a task per client per service.

How it works.

```
def seed_period_and_tasks(conn: sqlite3.Connection) -> None:
    """Create the sample month and generate its tasks.

    In: an open connection with clients and subscriptions present.
    Out: nothing. One task per active subscription, the UNIQUE rule
    on TASKS absorbs reruns, generating twice adds nothing.
    """
    year, month = SEED_PERIOD
    conn.execute(
        "INSERT OR IGNORE INTO PERIODS (YEAR, MONTH) VALUES (?, ?)",
        (year, month),
    )

    conn.execute(
        "INSERT OR IGNORE INTO TASKS (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH) "
        " SELECT CLIENT_ID, SERVICE_ID, ?, ? FROM CLIENT_SERVICES WHERE ACTIVE = 1",
        (year, month),
    )
```

The first line unpacks `SEED_PERIOD` into `year` and `month`. The first `execute` inserts the period row, August 2026, with the same `INSERT OR IGNORE` idiom you have now seen twice, so a rerun leaves the existing period alone.

The second `execute` is the heart of the function and deserves a slow reading, because it uses a pattern you will meet constantly in SQL work, an `INSERT` fed by a `SELECT`. In a normal insert, the `VALUES` clause supplies one row that you typed out yourself. Here there is no `VALUES` clause at all. Instead, a `SELECT` query runs and every row it produces becomes a row to insert. The database copies data from one table into another in a single statement, without the rows ever passing through Python.

Read the statement clause by clause.

- `INSERT OR IGNORE INTO TASKS (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH)`: the destination, naming the four columns each incoming row will fill, with `OR IGNORE` guarding against duplicates as always.
- `SELECT CLIENT_ID, SERVICE_ID, ?, ?`: the source row shape. The first two items come from the table being read, and the two `?` placeholders are constants, the year and month, stamped identically onto every produced row. A select list can mix column references and literal values freely, and the four items line up by position with the four columns named in the insert.
- `FROM CLIENT_SERVICES`: the source table, the subscriptions that `seed_clients` just wrote.
- `WHERE ACTIVE = 1`: the filter, and the reason the previous function bothered to switch one subscription off.

That `WHERE ACTIVE = 1` clause is doing quiet but important work. A subscription being switched off means the client no longer files that return, so no task should be generated for it. Because `seed_clients` deactivated Bhima Textiles' `GSTR-1` subscription, this statement generates one task fewer than the raw subscription count, and that missing task is the proof that deactivation actually means something. If someone later broke the filter, the seed's summary output would show the wrong pending count for `GSTR-1`, so the fake data doubles as a lightweight test of the generation rule.

The docstring's last sentence explains why `OR IGNORE` is enough for rerun safety here. The `TASKS` table carries a `UNIQUE` rule across client, service, and period, so generating the same month twice produces rows that all collide with existing ones and are all ignored.

mark_sample_statuses, two tasks made interesting

Why it exists. Freshly generated tasks all share the same status, pending, and a page that only ever shows pending rows cannot be tested against the other states. This function hand picks two tasks and pushes them into the two other states the system knows, done and not applicable, so future screens have every state on display.

How it works.

```
def mark_sample_statuses(conn: sqlite3.Connection) -> None:
    """Flip two tasks into interesting states for future pages.

    In: an open connection with generated tasks.
    Out: nothing, one task done with a completion trace, one not
    applicable. Reruns set the same rows to the same values.
    """
    conn.execute(
        "UPDATE TASKS SET STATUS = 'done',"
        " COMPLETED_AT = ?, COMPLETION_METHOD = 'manual'"
        " WHERE CLIENT_ID = (SELECT ID FROM CLIENTS WHERE NAME = 'Aster Traders')"
        " AND SERVICE_ID = (SELECT ID FROM SERVICES WHERE NAME = 'GSTR-3B')",
        (SEED_COMPLETED_AT,),
    )
    conn.execute(
        "UPDATE TASKS SET STATUS = 'not_applicable'"
        " WHERE CLIENT_ID = (SELECT ID FROM CLIENTS WHERE NAME = 'Deccan Services')"
        " AND SERVICE_ID = (SELECT ID FROM SERVICES WHERE NAME = 'ESI')",
    )
```

The first `UPDATE` marks Aster Traders' `GSTR-3B` task as done. It does not stop at the status column, it also writes a complete trace of the completion, `COMPLETED_AT` set to the fixed `SEED_COMPLETED_AT` timestamp through a placeholder, and `COMPLETION_METHOD` set to the literal `'manual'`, as if a person had ticked the task off by hand. Filling all three columns matters because any page that displays a done task will want to show when and how it was completed, and the seed should never present a half filled row.

Look closely at the `WHERE` clause, because it repeats the targeting idiom introduced at the end of `seed_clients`. The task row is not addressed by a numeric id, since ids are assigned by the database and could differ between environments. Instead the clause carries two subselects, each a nested query in parentheses that produces a single value for the outer comparison. The first resolves the client name Aster Traders to its `CLIENTS` id, the second resolves the service name `GSTR-3B` to its `SERVICES` id, and every task row matching both ids is touched. Note that nothing in the clause filters on the period, so a task for the same client and service in a different month would be updated too. That is harmless here only because the seed creates a single period, August 2026, which is what makes the two ids enough to land on exactly one row. Targeting by name keeps the function readable and correct regardless of what ids the inserts happened to hand out, and because names resolve to the same rows every time, reruns set the same rows to the same values, exactly as the docstring promises.

The second `UPDATE` uses the same shape with less to say. Deccan Services' `ESI` task becomes `not_applicable`, the state for a filing that simply does not apply this month, and since that state carries no completion trace, only the status column changes. Between the two statements, and the untouched pending majority around them, every task status the schema knows now exists in the seeded database.

print_summary, the proof of life

Why it exists. A script that ends in silence leaves you wondering whether it did anything. This function prints a small report of pending task counts per service, so one glance at the terminal confirms the seed worked and the numbers make sense.

How it works.

```
def print_summary(conn: sqlite3.Connection) -> None:
    """Print pending counts per service, the seed's proof of life.

    In: an open connection after seeding.
    Out: nothing returned, prints to the terminal.
    """
    year, month = SEED_PERIOD
    rows = conn.execute(
        "SELECT s.NAME, COUNT(*) FROM TASKS t"
        " JOIN SERVICES s ON s.ID = t.SERVICE_ID"
        " WHERE t.PERIOD_YEAR = ? AND t.PERIOD_MONTH = ? AND t.STATUS = 'pending'"
        " GROUP BY s.NAME ORDER BY s.NAME",
        (year, month),
    ).fetchall()

    print(f"pending tasks for {month} / {year}")
    total = 0
    for name, count in rows:
        print(f"  {name}: {count}")
        total += count
    print(f"  total pending: {total}")
```

After unpacking the period, the function runs one aggregate query. The `FROM TASKS t` clause gives the table a short alias, `t`, a nickname used to qualify column names in the rest of the statement, and `JOIN SERVICES s ON s.ID = t.SERVICE_ID` connects each task to its service row so the report can show service names instead of bare ids. A `JOIN` pairs rows from two tables wherever the `ON` condition holds. The `WHERE` clause narrows the tasks to the seeded month and to status pending, so completed and not applicable tasks stay out of the count. `GROUP BY s.NAME` collapses the matching tasks into one row per service name, and `COUNT(*)` in the select list counts how many tasks fell into each group. `ORDER BY s.NAME` sorts the report alphabetically so the output is stable from run to run. `fetchall` pulls every result row into a Python list at once, fine here because there are at most a handful of services.

The printing half is plain Python. The header line uses an f-string, a string literal prefixed with `f` in which expressions inside curly braces are evaluated and spliced into the text. The loop unpacks each result row into `name` and `count`, prints an indented line per service, and accumulates a running `total`, which the final line prints so you do not have to add the numbers in your head. Run against the seeded data, this is where the effect of the deactivated subscription and the two status flips becomes visible as slightly uneven counts across services.

main, the whole seed in order

Why it exists. Someone has to decide which database file to use, open it, run the seeding steps in a working order, commit, and clean up even if something breaks along the way. `main` is that conductor, and it is the only function in the file with any control flow beyond a loop.

How it works.

```
def main() -> None:
    """Run the whole seed, initializing first so a new file works.

    In: an optional database path from the command line.
    Out: nothing, the database is filled and a summary printed.
    """
    db_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_DB_PATH
    conn = connect(db_path)

    try:
        initialize(conn)

        seed_clients(conn)
        seed_period_and_tasks(conn)
        mark_sample_statuses(conn)
        conn.commit()

        print_summary(conn)
    finally:
        conn.close()
```

The first line resolves the database path with a conditional expression, Python's one line if else. `sys.argv` is the list of command line arguments, where index zero is the program itself, so `sys.argv[1]` is the optional path the docstring's usage line advertised. If the user supplied one it is wrapped in a `Path`, otherwise `DEFAULT_DB_PATH` from the migration module is used. Then `connect`, also borrowed from the migration module, opens the connection with the project's standard settings, so the seed can never accidentally open a database configured differently from the rest of the system.

Everything after that lives inside a `try` with a `finally`. A `finally` block runs no matter how the `try` block ends, whether it finishes cleanly or a statement inside it raises, which makes it the right home for cleanup. Here the cleanup is `conn.close()`, releasing the database connection. If any seeding step crashes, the file handle still gets closed instead of dangling until the process dies.

The order of the calls inside the `try` is the dependency chain of the whole module, and reading it top to bottom retells the chapter. `initialize` runs first, applying the migrations and ensuring the `SERVICES` rows, so pointing the seed at a brand new file just works, the schema and the reference data exist before any seed function needs them. `seed_clients` comes next because everything else hangs off clients and their subscriptions, including the one deactivated row. `seed_period_and_tasks` follows, since its `INSERT` fed by a `SELECT` reads the very `CLIENT_SERVICES` rows the previous call wrote. `mark_sample_statuses` must come after generation, as it updates task rows that would otherwise not exist yet. Then `conn.commit()` makes all of those writes permanent, since with this connection setup changes are not durable until committed. Only after the commit does `print_summary` run, so the report reads settled, committed state rather than an in flight transaction. Note what is absent from the `finally` block, there is no rollback logic, and none is needed for a development tool, an uncommitted transaction is simply discarded when the connection closes.

The entry point guard

Why it exists. The file should run its seed when executed as a script but stay quiet when another module imports it, for example a test that wants to call `seed_clients` on its own.

How it works.

```
if __name__ == "__main__":  
    main()
```

`__name__` is a variable Python sets in every module. When a file is executed directly, including through `python3 -m`, its `__name__` is the string `"__main__"`, but when the file is imported its `__name__` is its module path instead. This guard therefore calls `main` only in the executed case. It is the standard closing line of any Python script that is also importable, and with it the seed module is both a tool you run and a library you can borrow pieces from.

5.5 app/main.py And app/deps.py, Assembling The Application

Every FastAPI project has a moment where the pieces stop being separate files and become one running program. In TaxDesk that moment lives in two small files. `app/main.py` builds the application, wires the database migrations into startup, and plugs in the route modules, while `app/deps.py` holds the shared pieces those routes lean on, a template engine and a database dependency that hands each request its own connection. Together they are the assembly point, everything downstream of chapter 4's migration code and upstream of every route you will meet later. They are short, but almost every line encodes a decision, so we will walk both files completely.

The Module Docstring And Its Security Note

Why it exists. The first thing in `app/main.py` is not code but a contract with whoever runs the program. It tells you the one command that starts the app and, more importantly, states the security model in plain words so nobody weakens it by accident. TaxDesk handles tax documents, which is sensitive data, so the boundary deserves to be written at the top of the entry point where it cannot be missed.

```
"""TaxDesk application entry point.  
  
Run locally, and only locally:  
  
    venv/bin/uvicorn app.main:app  
  
uvicorn binds 127.0.0.1 by default, never pass a host flag. The  
security boundary of this local first app is that it listens on  
localhost only.  
"""
```

How it works. A module docstring is a string placed as the very first statement in a file. Python stores it as documentation for the whole module, and tools like `help()` display it. This one does three jobs. First it names the file's role, the application entry point, meaning the file a server loads to get the app. Second it gives the exact run command, `venv/bin/uvicorn app.main:app`, using the virtual environment's own uvicorn binary so the right installed packages are used. Third it explains the security note. uvicorn is the server program that receives HTTP requests and passes them to the FastAPI app, and by default it binds to `127.0.0.1`, the loopback address that only programs on the same machine

can reach. Binding means choosing which network address the server listens on. Passing a host flag such as `--host 0.0.0.0` would open the app to the whole network, so the docstring forbids it outright. That single default is the entire security boundary of this local first app, there is no login screen, so the boundary must hold.

The Imports

Why it exists. The import block is the file's shopping list, and reading it tells you in advance everything `main.py` is about to do. It pulls in typing and context manager tools for the startup hook, the FastAPI class itself, the two route modules, and the database layer from chapter 4.

```
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI

from app.routes.health import router as health_router
from app.routes.onboarding import router as onboarding_router
from database.migrate import DEFAULT_DB_PATH, connect, initialize
```

How it works. The imports come in three groups separated by blank lines, which is the conventional Python ordering, standard library first, third party packages second, this project's own modules last. `AsyncIterator` is a type from the standard library used to annotate the return value of the lifespan function you will see next. `asynccontextmanager` is a decorator, a function you place above another function with an `@` sign to wrap it in extra behavior, and this one wraps an async generator function so that calling it produces an async context manager, an object usable with `async with` that runs setup code on entry and teardown code on exit. `Path` is the standard library's object for filesystem paths, safer than raw strings because it knows how to join and resolve paths on any operating system. `FastAPI` is the class the whole application is an instance of. The next two lines import a `router` object from each route module, renaming them with `as` so the two do not collide, since both modules call their export `router`. A router is FastAPI's container for a group of related endpoints, defined in its own file and plugged into the app later. The last line imports three things from the migration module, `DEFAULT_DB_PATH`, the production database file location, `connect`, the single sanctioned way to open a SQLite connection in this project, and `initialize`, the function that applies any pending migrations.

Lifespan, Migrations Before The First Request

Why it exists. A web app backed by a database has a chicken and egg problem, requests need the tables to exist, but something has to create the tables before the first request arrives. FastAPI's answer is the lifespan hook, a function the framework runs around the serving period, and TaxDesk uses it to apply migrations exactly once at startup. This guarantees no request can ever see a database that is missing a table or behind on schema changes.

```
@asynccontextmanager
async def lifespan(application: FastAPI) -> AsyncIterator[None]:
    """Run migrations once before the first request is served.

    In: the FastAPI app, whose state carries the database path.
    Out: yields while the app serves. The startup connection is
    closed immediately, requests always get their own.
    """
    conn = connect(application.state.db_path)
    try:
```

```

        initialize(conn)
    finally:
        conn.close()
    yield

```

How it works. Start with the decorator. `@asynccontextmanager` turns the async generator function below it into a factory, calling `lifespan(...)` now returns an async context manager built around one run of the function's body. A context manager is any object that runs code when you enter a block and more code when you leave it, the pattern behind `with open(...)`, and the async variant does the same for code that may need to await. FastAPI expects its lifespan in exactly this shape, it calls the factory once and enters the resulting context manager around the serving period. Everything before the `yield` runs at startup, before the server accepts its first request. The `yield` then hands control back to FastAPI, which serves requests for as long as the process lives. Everything after the `yield`, here nothing, would run at shutdown.

The signature takes the FastAPI `application` and is annotated as returning `AsyncIterator[None]`, which is the honest type of an async generator that yields once and yields no useful value. The docstring follows the project's In and Out convention, naming what comes in and what goes out.

The body is four lines of real work. `conn = connect(application.state.db_path)` opens a database connection. Note where the path comes from, `application.state` is a small attribute bag FastAPI provides on every app for exactly this kind of shared value, and `create_app` below stores the database path there. The lifespan function reads it back instead of importing a global path, which means the same lifespan code works for the production database and for a temporary test database without any changes.

The `try` and `finally` around `initialize(conn)` is a cleanup guarantee. `initialize` applies every migration that has not run yet, and whether it succeeds or raises, the `finally` block always executes, so `conn.close()` always runs. If a migration fails, the exception still propagates and the app refuses to start, which is the right outcome, but the connection does not leak.

Finally, notice that `yield` sits after the `finally` block, not inside it. The startup connection lives only for the few milliseconds migrations take, then closes before the app serves anything. As the docstring says, requests always get their own connection, and that is `get_db`'s job in `deps.py`.

create_app, The Application Factory

Why it exists. Instead of building the app at import time with hardcoded settings, TaxDesk builds it inside a function. This pattern is called an application factory, a function that constructs and returns a fresh app each time it is called. The payoff is testability, a test can call `create_app` with a temporary database path and get a fully wired app that touches no real data, while production calls it with the default.

```

def create_app(db_path: Path = DEFAULT_DB_PATH) -> FastAPI:
    """Build the application around one database path.

    In: the database file path, tests pass a temp path, production
    uses the default.
    Out: a FastAPI app with migrations wired into startup and the
    routes plugged in.
    """
    application = FastAPI(lifespan=lifespan)

```

```
application.state.db_path = db_path
application.include_router(health_router)
application.include_router(onboarding_router)
return application
```

How it works. The signature is the whole design. `db_path: Path = DEFAULT_DB_PATH` declares one parameter with a type annotation and a default value. Call `create_app()` with no arguments and you get the production database. Call `create_app(tmp_path / "test.db")` from a test and every part of the app, migrations included, operates on that throwaway file. This is dependency injection done by hand, the one value that varies between environments enters through the front door as a parameter rather than being baked into the code. The return annotation `-> FastAPI` states the contract at the boundary.

The body reads top to bottom as an assembly sequence. `FastAPI(lifespan=lifespan)` constructs the app and registers the lifespan function from the previous section, so the migration hook is now wired to run at startup. `application.state.db_path = db_path` stores the path on the app's state bag, which is what makes it reachable later from two places you have already seen or soon will, `application.state.db_path` inside `lifespan` and `request.app.state.db_path` inside `get_db`. The path is set once, here, and every other piece of code reads it rather than deciding it.

The two `include_router` calls plug the route modules in. Each router was defined in its own file with its own endpoints, the health check in `app/routes/health.py` and the onboarding flow in `app/routes/onboarding.py`, and including them merges their endpoints into this app. The final line returns the finished application to whoever asked for it.

`app = create_app()` And How uvicorn Finds It

Why it exists. A factory function is only useful if something calls it. This single line calls it at import time and binds the result to a module level name, which is precisely the name the run command points at.

```
app = create_app()
```

How it works. When Python imports `app/main.py`, every top level statement executes, so this line runs `create_app()` with no arguments, producing the production app with the default database path, and stores it in the variable `app`. Now look back at the run command from the docstring, `venv/bin/uvicorn app.main:app`. That argument is an import string with two halves split by the colon. The left half, `app.main`, is a module path, uvicorn imports the `main` module inside the `app` package exactly as `import app.main` would. The right half, `app`, is an attribute name, uvicorn looks up that attribute on the freshly imported module and expects to find an application it can serve. The lookup lands on the object this line created. So the flow at launch is uvicorn imports the module, the import executes `app = create_app()`, uvicorn grabs `app`, starts serving, and FastAPI runs `lifespan` before the first request, which applies migrations. One command, and the whole chain from chapter 4's migration runner to a listening server fires in order. Tests skip this line's result entirely, they call `create_app` themselves with their own path, which is exactly why the factory and the module level instance are separate things.

`deps.py`, The Shared Pieces

Why it exists. Route modules across the project need the same two things, a way to render HTML templates and a way to talk to the database. Duplicating that in every route file would scatter the rules, so `deps.py` centralizes both. Its docstring also restates a project law, every connection goes through the one sanctioned `connect()` function, never around it.

```
"""Shared pieces route modules need, the database dependency and the
template engine. Every route gets its own short lived connection
through the single sanctioned connect(), and never anything else."""

from collections.abc import Iterator
from pathlib import Path
from sqlite3 import Connection

from fastapi import Request
from fastapi.templating import Jinja2Templates

from database.migrate import connect
```

How it works. The docstring names the two exports and the discipline behind the database one, short lived connections from a single source. The imports again follow the three group convention. `Iterator` is the typing name for something you can loop over one value at a time, used to annotate `get_db`'s return type. `Path` you have met. `Connection` is the type of an open SQLite database handle from the standard library's `sqlite3` module, imported so the annotation can name what `get_db` yields. From FastAPI come `Request`, the object representing the current HTTP request, and `Jinja2Templates`, FastAPI's wrapper around the Jinja2 template engine. Jinja2 is a library that fills HTML files containing placeholders with real values at request time, which is how this server side rendered app produces its pages. Last comes `connect` from the migration module, the sanctioned way to open the database, honoring the docstring's rule at the import level, the only piece of `sqlite3` taken here is the `Connection` type for annotations, never the module's own way of opening one.

The templates Object

Why it exists. Every route that returns an HTML page needs a configured template engine, and the engine needs to know one thing, which directory holds the template files. Creating it once at module level means every route imports the same instance and the directory is decided in exactly one place.

```
templates = Jinja2Templates(directory=str(Path(__file__).resolve().parent / "templates"))
```

How it works. Read the expression inside out. `__file__` is a variable Python sets in every module to the path of the module's own source file, here the path to `deps.py`. `Path(__file__)` wraps it in a `Path` object, and `.resolve()` converts it to an absolute path with any symlinks and relative segments cleaned up, so the result does not depend on which directory the server was started from. `.parent` steps up one level to the directory containing `deps.py`, which is the `app/` package. The `/` after it is not division, `Path` overloads the division operator to mean join a path segment, so `parent / "templates"` is the `app/templates` directory. `str(...)` converts the `Path` back to a plain string. This is a choice, not a requirement of the API, `Jinja2Templates` also accepts `Path` objects directly, and the code simply normalizes the value to a string rather than relying on that support. The finished object, bound to the module level name `templates`, is what routes use to render pages, and because the location is computed relative to this file, the app finds its templates no matter where it is launched from.

get_db, One Connection Per Request

Why it exists. Sharing one database connection across all requests would be a correctness trap, one request's half finished writes could be committed by another, and an error in one place could poison everyone. TaxDesk instead gives each request its own connection with a strict lifecycle, open at the start, commit on success, roll back on failure, close no matter what. `get_db` is that lifecycle written as a FastAPI dependency, a function the framework calls for you and whose result it hands to any route that declares it as a parameter.

```
def get_db(request: Request) -> Iterator[Connection]:
    """Hand one request one database connection with honest teardown.

    In: the current request, whose app state carries the database
    path, set once in create_app.
    Out: yields an open connection for exactly this request. A route
    that finishes cleanly gets its writes committed, a route that
    raises gets them rolled back, and the connection always closes.
    """
    conn = connect(request.app.state.db_path)
    try:
        yield conn
        conn.commit()
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()
```

How it works. Because the body contains `yield`, this is a generator function. A generator is a function that can pause. Calling it does not run the body, it returns a generator object, and each time someone asks that object for the next value, the body runs until it hits a `yield`, hands out the yielded value, and freezes in place with all its local variables intact. Asking again resumes it right after the `yield`. That pause is the trick that lets one function wrap code that runs somewhere else entirely. The return annotation `Iterator[Connection]` says it honestly, this function produces an iterator whose values are connections, and it will yield exactly one.

FastAPI knows this pattern. When a route declares a parameter like `db: Connection = Depends(get_db)`, the framework calls `get_db`, runs it up to the `yield`, passes the yielded connection into the route, and after the route finishes, resumes the generator so the code after `yield` can clean up. Setup before the `yield`, the route in the middle, teardown after, the same shape as a context manager.

Line by line. The parameter `request: Request` is itself filled in by FastAPI, which knows how to supply the current request to any dependency that asks for it. `conn = connect(request.app.state.db_path)` opens the connection, and trace where that path came from, `request.app` is the FastAPI application handling this request, `.state.db_path` is the value `create_app` stored during assembly. One assignment in the factory, read here on every request, no globals in between.

Now the `try` block, and this is where the three exits live. `yield conn` pauses the generator and gives the route its connection. Everything the route does with the database happens while this function is frozen on that line. What happens next depends on how the route ends, and there are exactly three possibilities.

- clean return: the route finishes without error, FastAPI resumes the generator, execution continues past the `yield` to `conn.commit()`, which makes the request's writes permanent, then the `finally` closes the connection. A commit ends a database transaction by durably saving all of its changes at once.
- route raises: the route hits an exception, and FastAPI does not just abandon the generator, it throws that same exception into the generator at the paused `yield`, as if the `yield` line itself had raised. Control jumps over `conn.commit()`, so nothing is committed, and lands in `except BaseException`. `conn.rollback()` undoes every write the request made, restoring the database to its state before the request began. The bare `raise` then re raises the same exception unchanged, so FastAPI still sees the error and returns a proper error response, the rollback did not swallow it. On the way out, `finally` closes the connection.
- generator closed early: the framework can also stop the generator without any route error, most concretely when the client disconnects mid request and there is no one left to answer. Closing a paused generator raises `GeneratorExit` inside it at the `yield`. That too lands in the `except BaseException` handler, gets a rollback, is re raised by the bare `raise`, and passes through `finally` to close the connection. Python does not demand that re raise, a closing generator is allowed to simply return, what it forbids is yielding another value during the close. Re raising is still the right move here, it keeps this one handler's behavior identical for every kind of exit, roll back, pass the signal on, close.

That third exit is exactly why the handler says `except BaseException` and not `except Exception`, and the distinction is worth slowing down for. Python's exception classes form a tree. `Exception` is the base of ordinary errors, the kind your own code raises and catches. Above it sits `BaseException`, which also covers the exceptional exits that are not ordinary errors, `GeneratorExit` when a generator is being shut down, `KeyboardInterrupt` when someone presses Ctrl+C, `SystemExit` when the process is asked to stop. `GeneratorExit` deliberately inherits from `BaseException` directly, not from `Exception`, precisely so that careless `except Exception` handlers do not trap it. If this function caught only `Exception`, then a client disconnect that closed the generator would sail past the handler, `conn.rollback()` would never run, and the request's unfinished writes would be left to whatever the closing connection does with them instead of being rolled back on purpose. Catching `BaseException` guarantees the rollback runs on every abnormal exit, whatever its species, and the immediate `raise` guarantees nothing is swallowed. This is the rare situation where the usually suspicious `except BaseException` is exactly right, because the handler does not handle the exception at all, it only inserts a rollback into its flight path and lets it continue.

The `finally: conn.close()` line is the floor under all three exits. A `finally` block runs no matter how the `try` block ends, normal completion, ordinary exception, or `GeneratorExit`, so the connection is closed on every path through this function. That is the docstring's promise of honest teardown made literal, commit on the clean path, rollback on every failure path, close always. Every route in the chapters ahead gets its database access through this one function, so when you read a route later and see a `Connection` parameter appear as if by magic, this is the machinery behind it.

5.6 The Routes, `health.py` And `onboarding.py`

This chapter reads the two route files that exist so far, `app/routes/health.py` and `app/routes/onboarding.py`. A route file is where HTTP meets the application, each function in it answers one URL, and FastAPI wires the two together through decorators. The health file is tiny on purpose, it is the proof that the whole stack works and the template every later route copies. The onboarding file is the first real feature, it lets the practitioner point TaxDesk at a folder on disk, shows the subfolders it finds there, and turns the ticked ones into client rows in the database. Between those two steps sits

an explicit confirmation, discovery never creates anything by itself. Everything else in the system, settings queries, client queries, the database connection, arrives here through imports, so these files stay thin coordinators rather than places where logic lives.

health.py, The Module Header

Why it exists. Every route file needs the same scaffolding, a docstring stating its contract, its imports, and a router object for FastAPI to collect. The header of `health.py` is the smallest possible version of that scaffolding, which is exactly why the project keeps it as the pattern for future route files.

```
"""The health route, the skeleton's only endpoint and the template
every future route file follows, parse nothing, call through the
dependency, return a plain dict."""

from sqlite3 import Connection

from fastapi import APIRouter, Depends

from app.deps import get_db

router = APIRouter()
```

How it works. The module docstring, the triple quoted string at the top of the file, states the file's job and names the pattern to copy, parse nothing, call through the dependency, return a plain dict. `Connection` is imported from Python's standard `sqlite3` module purely as a type, it lets the function signature below say what kind of object it expects. From FastAPI come two names. `APIRouter` is a small container that collects route functions so the main application can mount them all in one call. `Depends` is FastAPI's dependency injection marker, dependency injection means a function declares what it needs and the framework supplies it, instead of the function fetching it for itself. `get_db` is the project's own dependency, defined in `app/deps.py`, which hands each request a database connection scoped to that request. The last line creates the module's router. This `router` variable is what the application imports and includes, every decorator in the file registers its function on it.

The health Function

Why it exists. When something is wrong with a deployed service, the first question is whether the server is up at all and whether it can still reach its database. The health endpoint answers both questions with one cheap request, and by living behind the same `get_db` dependency as every real route, it exercises the same spine a real request would travel.

```
@router.get("/health")
def health(conn: Connection = Depends(get_db)) -> dict[str, str]:
    """Report that the whole spine is alive, server to database.

    In: nothing from the caller.
    Out: exactly {"status": "ok"}, and nothing else, no paths, no
    versions, no internals. The SELECT proves SQLite is reachable,
    its result is deliberately not exposed.
    """
    conn.execute("SELECT 1")
    return {"status": "ok"}
```

How it works. The first line is a decorator, a decorator is a function that wraps another function, and the `@` syntax applies it to the definition directly below. `@router.get("/health")` registers `health` on the router as the handler for HTTP GET requests to the path `/health`. The registration happens once, at import time, when Python executes the module. From then on, FastAPI calls `health` whenever a matching request arrives.

The signature declares one parameter, `conn`, typed as a `Connection`, with `Depends(get_db)` as its default value. That default is a signal to FastAPI rather than a real default, before calling `health`, the framework runs `get_db` and passes its result in as `conn`. The return annotation `dict[str, str]` promises a dictionary whose keys and values are both strings.

The docstring spells out the contract, and it is worth pausing on the "Out" line, the response is exactly `{"status": "ok"}` and nothing more. Health endpoints are often reachable without authentication, so leaking paths, versions, or internals through one would hand information to strangers.

The body is two lines. `conn.execute("SELECT 1")` runs the simplest possible SQL statement against SQLite. The point is the round trip itself, if SQLite cannot be reached, say the database file is locked by another process, opening the connection or running this statement raises and the request fails, which is precisely the signal a health check exists to give. A merely missing file would not trip it, since `connect` quietly creates an empty database file when the path does not exist, so what the probe really proves is the spine itself, a connection was opened and a statement executed end to end. The result of the query is thrown away on purpose. Finally the function returns a plain dict, and FastAPI serializes it to JSON for the response body.

onboarding.py, The Module Header

Why it exists. Onboarding is the flow where the practitioner's existing folder layout becomes TaxDesk's client list. The header states the module's two safety rules up front, creation only happens after explicit confirmation, and this module never writes to the filesystem. Everything that follows in the file honors those two lines.

```
"""Onboarding, the practitioner's folders become clients, with an
explicit confirmation between discovery and creation. The filesystem
is read only here, this module never writes to disk."""

from pathlib import Path
from sqlite3 import Connection

from fastapi import APIRouter, Depends, Request
from fastapi.responses import HTMLResponse, RedirectResponse, Response

from app.deps import get_db, templates
from database import queries

router = APIRouter()
```

How it works. After the docstring come the imports, grouped by origin. `Path` from `pathlib` is the standard library's object for working with filesystem paths, and `Connection` again types the database parameter. From FastAPI, this file needs more than `health` did. `Request` is the object representing the incoming HTTP request, the handlers here use it to read submitted forms and query parameters. From `fastapi.responses` come three response types, `HTMLResponse` to mark a route as returning a web page, `RedirectResponse` to send the browser to a different URL, and `Response` as the general return type annotation covering both. From the project's own `app.deps` come `get_db`, the same connection

dependency health used, and `templates`, the configured template engine that turns HTML files plus data into rendered pages. `queries` from the `database` package holds every SQL operation, so no SQL string appears anywhere in this file. The final line creates this module's own router, separate from the health router, and every decorator below registers on it.

The `candidate_folders` Helper

Why it exists. Two different routes need the same answer to the same question, which folders inside the configured root could become clients. Discovery needs it to draw the checklist, and confirmation needs it to decide which submitted names are real. Centralizing the scan in one helper guarantees both routes see the exact same list, which is what makes the trust check in `confirm_clients` sound.

```
def candidate_folders(root_folder: str) -> list[Path]:
    """List the immediate subfolders of the root, nothing deeper.

    In: the configured root folder path as a string.
    Out: the immediate child directories, hidden dot folders and
    plain files excluded, sorted by name. Empty when the root has
    stopped existing or cannot be read, a directory can stat as real
    while a permission or a removable drive still blocks listing it.
    """
    root = Path(root_folder)
    if not root.is_dir():
        return []

    try:
        children = list(root.iterdir())
    except OSError:
        return []

    return sorted(
        (child for child in children
         if child.is_dir() and not child.name.startswith(".")),
        key=lambda child: child.name.lower(),
    )
```

How it works. Notice first what is missing, there is no decorator. This is a plain helper, called by route functions, never reached by a URL of its own.

The body opens by wrapping the incoming string in a `Path` object, which gives access to filesystem methods like `is_dir` and `iterdir`. Then comes the first guard, `if not root.is_dir(): return []`. This is an early return, the function handles the bad case immediately and exits, instead of nesting the happy path inside an `if`. A saved root can stop being a directory at any time, the folder may have been renamed, deleted, or lived on a drive that was unplugged, and in every such case the honest answer is an empty list, which the caller renders as zero candidates.

The next block guards a subtler failure. `root.iterdir()` yields the entries inside a directory, and wrapping `list(...)` around it forces the whole listing to happen right here, inside the `try`. The reason for the `try` is in the docstring, a directory can pass the `is_dir` check yet still refuse to be listed. The classic case is permissions, the operating system lets you see that the folder exists but denies you the right to open it, and other cases like a removable drive going away mid request behave the same. All of those failures surface as `OSError`, Python's exception family for operating system level failures, and the `except` turns them into the same calm empty list rather than a crash page.

The return statement filters and orders in one expression. The first argument to `sorted` is a generator expression, a generator is like a list comprehension but written with parentheses, it produces items one at a time on demand instead of building a full list in memory first. This one walks `children` and keeps only entries that are directories and whose names do not start with a dot. Dot folders are the Unix convention for hidden housekeeping directories, things like `.git`, and no practitioner means those as clients. Plain files are dropped by the `is_dir` test in the same breath.

The `key` argument tells `sorted` what to compare. `lambda` is Python's syntax for a small unnamed function written inline, and this one maps each child to its name in lowercase. Without it, Python would sort by raw character codes, where every uppercase letter comes before every lowercase letter, so `Zeta` would sort ahead of `alpha`. Lowercasing the key gives the alphabetical order a human expects, while the returned paths keep their original spelling.

The `discovery_context` Helper

Why it exists. The onboarding page needs more than a folder list, it needs to know which folders are already clients so they render disabled, how many genuinely new ones remain, and what the submit button should say. Both `onboarding_page` and the error path in `save_root` render that same panel, so the shaping lives in one helper that returns a ready dictionary of template variables.

```
def discovery_context(conn: Connection, root_folder: str | None) -> dict:
    """Build the candidate list and the button label for a root.

    In: an open connection and the currently configured root, or None.
    Out: candidates is None when no root is configured, otherwise one
    entry per immediate subfolder, existing clients marked so they
    render disabled and already ticked out of the count.
    """
    if root_folder is None:
        return {"candidates": None, "new_count": 0, "submit_label": "Add 0 clients"}

    existing_paths = {client["FOLDER_PATH"] for client in queries.list_clients(conn)}
    candidates = [
        {
            "name": folder.name,
            "existing": str(folder.resolve()) in existing_paths,
        }
        for folder in candidate_folders(root_folder)
    ]
    new_count = sum(1 for candidate in candidates if not candidate["existing"])

    return {
        "candidates": candidates,
        "new_count": new_count,
        "submit_label": f"Add {new_count} client{' ' if new_count == 1 else 's'}",
    }
```

How it works. The signature accepts the connection and the root, and the type `str | None` says the root may be absent, which is the state before the practitioner has saved anything. The first lines handle exactly that with an early return. Note the distinction it encodes, `candidates` is `None`, which the template can read as "no root configured yet, do not draw the panel at all", a different situation from an empty list, which would mean "a root exists but holds no folders".

The next line builds `existing_paths` with a set comprehension. A set comprehension looks like a list comprehension but uses braces and produces a set, a collection optimized for fast membership tests with `in`. It asks `queries.list_clients` for every client row in the database and collects each row's `FOLDER_PATH` column. The result is the set of folder paths TaxDesk already knows as clients.

Then a list comprehension builds one small dict per candidate folder, reusing `candidate_folders` for the scan. The interesting line is the `existing` flag, `str(folder.resolve()) in existing_paths`. The `resolve` method turns a path into its absolute, canonical form, symbolic links followed and relative pieces flattened, and the comparison matters because the database stores resolved paths, as we will see in `confirm_clients`, which writes `str(folder.resolve())`. Comparing resolved string to resolved string means the same real folder always matches its own row, however the root happened to be spelled when it was scanned. A folder marked `existing` renders disabled and unticked, per the docstring, so the practitioner cannot accidentally submit it again.

`new_count` counts the candidates whose `existing` flag is false, using `sum` over a generator that yields `1` for each new candidate, a common Python idiom for counting matches without building a list.

The final return packs the three template variables. The `submit_label` line is an f-string, a string literal prefixed with `f` in which the parts inside braces are evaluated as Python expressions and spliced into the text. The inner expression `' ' if new_count == 1 else 's'` is a conditional expression that pluralizes the label, so the button reads "Add 1 client" but "Add 2 clients", a small courtesy that spares the template from doing grammar.

The home Route

Why it exists. Someone who types the bare address of the application should land somewhere useful, and right now onboarding is the only page there is. Rather than duplicate the page under two URLs, the root path forwards to the real one.

```
@router.get("/")
def home() -> HttpResponseRedirect:
    """Send the visitor to the only page that exists so far.

    In: nothing.
    Out: a redirect to onboarding.
    """
    return HttpResponseRedirect("/onboarding", status_code=302)
```

How it works. The decorator `@router.get("/")` registers this function, at import time, as the handler for GET requests to the site root. The function takes no parameters, it does not even need the database. Its single line returns a `RedirectResponse`, which tells the browser to go fetch a different URL instead of rendering a body here. Status code 302 means "found elsewhere, for now", the standard temporary redirect. Temporary is the right choice because this forwarding is an artifact of the application being young, once a real home page exists, `/` will serve it directly, and a permanent redirect code would have taught browsers to keep skipping it.

The onboarding_page Route

Why it exists. This is the page the whole flow lives on. It shows the form for entering a root folder, and once a root is saved, it also shows the discovery panel listing that root's subfolders as candidate clients. One route serves both states, the template decides what to draw from the variables it receives.

```

@router.get("/onboarding", response_class=HTMLResponse)
def onboarding_page(
    request: Request,
    conn: Connection = Depends(get_db),
) -> Response:
    """Show the root form, and discovery once a root is saved.

    In: nothing from the URL.
    Out: the rendered page, its candidate list marking folders that
    are already clients, new folders ticked by default.
    """
    root_folder = queries.get_root_folder(conn)

    return templates.TemplateResponse(
        request,
        "onboarding.html",
        {
            "root_folder": root_folder,
            "configured_root": root_folder,
            "error": request.query_params.get("error"),
            **discovery_context(conn, root_folder),
        },
    )

```

How it works. The decorator registers the function for GET requests to `/onboarding`, and the extra argument `response_class=HTMLResponse` tells FastAPI this route returns an HTML page rather than JSON, which sets the correct content type on the response and documents the route honestly. The signature asks for two things, the `Request` object, which FastAPI supplies automatically when a parameter is typed as `Request`, and the database connection through the familiar `Depends(get_db)`.

The body's first beat fetches the currently configured root from the settings table via `queries.get_root_folder`. This is `None` on a fresh install, and everything downstream is built to handle that.

The second beat renders the template. `templates.TemplateResponse` takes the request, the template file name `onboarding.html`, and a dictionary of variables the template can use. Two of those variables carry the same value here, and the distinction is the key to understanding `save_root` later. `root_folder` is what the text input displays, while `configured_root` is what is actually saved in the database. On a plain page view they are identical, on a failed save they will diverge. The `error` variable is read from the URL's query parameters, the part after `?` in an address, with `.get("error")` returning `None` when no such parameter exists, so the template shows an error banner only when one was passed.

The last entry uses `**`, the dictionary unpacking operator, which spreads all key and value pairs from one dict into another. Here it merges everything `discovery_context` built, `candidates`, `new_count`, and `submit_label`, into the template variables without naming them one by one. The result is a single dictionary holding both the form state and the discovery panel state, handed to the template in one call.

The `save_root` Route

Why it exists. The practitioner types a path and submits it, and this route is the gate that decides whether that path becomes the configured root. It validates before it writes, keeps the settings untouched on a bad submission, and re-renders the form in a way that helps the practitioner fix a typo rather than retype from scratch.

```

@router.post("/onboarding/root")
async def save_root(
    request: Request,
    conn: Connection = Depends(get_db),
) -> Response:
    """Validate and save the root folder, the single settings row.

    In: the submitted form with the root_folder text.
    Out: 303 back to onboarding on success. An invalid path renders
    the form again with an error and SETTINGS stays untouched.
    """
    form = await request.form()
    submitted = str(form.get("root_folder", "")).strip()

    if not submitted or not Path(submitted).is_dir():
        # Echo back exactly what was typed, so a typo can be corrected
        # in place, and keep showing the still valid saved root's own
        # discovery panel, a rejected new path never touched SETTINGS.
        configured_root = queries.get_root_folder(conn)
        return templates.TemplateResponse(
            request,
            "onboarding.html",
            {
                "root_folder": submitted,
                "configured_root": configured_root,
                "error": "That path does not exist or is not a folder. Nothing was saved.",
                **discovery_context(conn, configured_root),
            },
            status_code=400,
        )

    queries.set_root_folder(conn, str(Path(submitted).resolve()))
    return RedirectResponse("/onboarding", status_code=303)

```

How it works. The decorator registers this handler for POST requests to `/onboarding/root`. POST is the method browsers use to submit forms, and using a distinct path for the submission keeps the page URL, `/onboarding`, purely a GET address. Note the `async def`. Reading a request body is an operation the server may have to wait on, the bytes arrive over the network, so FastAPI exposes it as an awaitable, and any handler that needs it must be declared `async`.

The first line of the body is exactly that wait, `form = await request.form()`. The `await` keyword pauses this handler until the form body has fully arrived and been parsed, while the server remains free to serve other requests in the meantime. The next line extracts the field. `form.get("root_folder", "")` fetches the input named `root_folder`, falling back to an empty string if the browser sent no such field at all, `str(...)` pins the type down, and `.strip()` trims surrounding whitespace so a stray space cannot make a valid path fail or an all whitespace entry pass.

Then the validation, a single condition with two arms. `not submitted` catches the empty submission, and `not Path(submitted).is_dir()` catches text that is not an existing directory on this machine. Either failure takes the error branch. The comment inside that branch carries the design intent, and the code below it does exactly what the comment says. The route reads the currently saved root fresh from the database, then re-renders the same `onboarding.html` template, but now the two look-alike variables from `onboarding_page` finally split. `root_folder`, the value shown in the text input, is set to `submitted`, the exact rejected text, so the practitioner sees their typo sitting in the box and can fix the one wrong character instead of retyping the whole path. `configured_root`, and the discovery panel built by `discovery_context(conn, configured_root)`, stay pinned to the saved root, because a rejected submission never

touched settings and the previously valid root is still doing its job. The `error` string tells the practitioner plainly what happened and reassures them that nothing was saved. The response carries `status_code=400`, the HTTP code for a bad request, so the failure is honest at the protocol level too, a monitoring tool or a test sees a 400, never a fake success.

The happy path is two lines at the bottom, flat and unindented, the early return having taken the failure out of the way. `queries.set_root_folder` writes the root into the settings table, and the value written is `str(Path(submitted).resolve())`, the canonical absolute form of the path rather than whatever relative or link laden spelling was typed. Storing resolved paths is what makes the equality checks in `discovery_context` reliable.

The final line redirects with `status_code=303`. Code 303, See Other, tells the browser to follow up with a GET to the given URL. This is the redirect after POST pattern, after a successful form submission you send the browser to a fresh GET of the results page, so that reloading the page repeats a harmless GET instead of resubmitting the form, and the browser's history holds a page, never a pending submission.

The `confirm_clients` Route

Why it exists. This is the second half of the two step promise in the module docstring, discovery showed candidates, and only an explicit confirmation creates clients. It is also the file's one true trust boundary, the incoming form data is a list of names sent by a browser, and a browser can be made to send anything. The route's job is to accept only names that correspond to real folders inside the real root, and to drop everything else without ceremony.

```
@router.post("/onboarding/confirm")
async def confirm_clients(
    request: Request,
    conn: Connection = Depends(get_db),
) -> Response:
    """Create a client for each confirmed folder, and only for those.

    In: the submitted form with ticked folder names, untrusted.
    Out: 303 back to onboarding. The accepted set is derived from a
    fresh scan of the real root, so crafted names like ../evil or
    absolute paths never match and die silently. Existing clients are
    never touched, the UNIQUE rule absorbs any duplicate.
    """

    root_folder = queries.get_root_folder(conn)
    if root_folder is None:
        return RedirectResponse("/onboarding", status_code=303)

    actual_folders = {
        folder.name: folder for folder in candidate_folders(root_folder)
    }

    form = await request.form()
    for name in form.getlist("folders"):
        folder = actual_folders.get(str(name))
        if folder is None:
            continue

        queries.create_client(
            conn,
            name=folder.name,
            folder_path=str(folder.resolve()),
        )
```

```
return RedirectResponse("/onboarding", status_code=303)
```

How it works. The decorator registers the handler for POST requests to `/onboarding/confirm`, and like `save_root` it is `async` because it must await the form body. Walk the body in order, because the order is the security argument.

First, the guard. The route fetches the configured root, and if none is saved it redirects straight back to onboarding. A confirmation with no root is a request that should not exist, someone posting to this URL out of sequence, and the early return ends it before any form data is even read.

Second, and before touching anything the browser sent, the route builds its source of truth. The dict comprehension, a comprehension that builds a dictionary, maps each folder's name to its `Path` object, and crucially it builds this map from `candidate_folders(root_folder)`, a fresh scan of the real disk at this very moment. The route does not trust the list it rendered on the page earlier, folders may have been created or deleted since, and it certainly does not trust the submission. What exists on disk right now, inside the root, non hidden, a directory, is the entire universe of acceptable names.

Only third does the untrusted data enter. `await request.form()` parses the submission, and then comes `form.getlist("folders")` where `save_root` used `form.get`. The difference matters. A form can carry several fields with the same name, which is exactly what a group of checkboxes produces, one `folders` entry per ticked box. `form.get` would return only the last of them, silently losing every other tick. `form.getlist` returns all values for the name as a list, an empty list when none were ticked, so the loop naturally does nothing on an empty confirmation.

Inside the loop, each submitted name is looked up in `actual_folders` with `.get`, which returns `None` for a missing key instead of raising. Here the trust boundary does its work. A crafted submission like `../evil`, an absolute path, or the name of a folder that never existed simply is not a key in the map built from disk, so the lookup yields `None` and `continue` skips to the next name. No error, no log line, no acknowledgment, the docstring's phrase is that such names die silently, and silence is deliberate, an attacker probing the endpoint learns nothing about what exists.

A name that survives the lookup is, by construction, a real immediate subfolder of the configured root, and only then does the route write. `queries.create_client` receives the name and the resolved absolute path, and notice that both values come from the `folder` object found on disk, never from the submitted string, so even the spelling that gets stored is the filesystem's own. The docstring notes the last line of defense, if a client for that path already exists, the database's UNIQUE constraint, a rule declared on the table that forbids two rows with the same value, absorbs the duplicate rather than creating a second client.

The route ends with the same 303 redirect back to `/onboarding` that `save_root` used, redirect after POST again. The browser lands on a fresh GET of the page, where the newly created clients now show up through `discovery_context` as existing, disabled, and out of the count, which is the practitioner's confirmation that the click worked.

5.7 The Templates, What The Browser Receives

Everything the previous sections built, the settings row, the folder scan, the candidate list, ends its journey as HTML in someone's browser, and these two files decide what that HTML looks like. TaxDesk uses Jinja templates, which are HTML files with small placeholders and control tags that the server fills in with real values before sending the page.

`base.html` is the shared shell that every page of the app will sit inside, and `onboarding.html` is the one real page the app has so far, the screen where the practitioner names a root folder and confirms which subfolders become clients. The route functions in `app/routes/onboarding.py` gather the values, and these templates turn those values into markup. Reading them closely also shows you every state the onboarding screen can be in, because the template is where those states become visible.

Before the files themselves, one line from `app/deps.py` explains how the server finds them.

```
templates = Jinja2Templates(directory=str(Path(__file__).resolve().parent / "templates"))
```

`Jinja2Templates` is the helper that FastAPI ships for rendering Jinja files, and this line points it at the `app/templates` directory, the folder that holds the two files below. When a route calls `templates.TemplateResponse(request, "onboarding.html", {...})`, the helper loads the named file from that directory, feeds it the dictionary of values, and returns the finished HTML as the response body.

base.html

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>TaxDesk</title>
</head>
<body>
  <h1>TaxDesk</h1>
  {% block content %}{% endblock %}
</body>
</html>
```

Why it exists. Every page in a web app repeats the same skeleton, the doctype, the head, the site title, and copying that skeleton into every page file means every future change happens in many places. Jinja solves this with template inheritance, where one base file owns the skeleton and each page file fills in only its own middle. This file is that base. It is deliberately tiny right now, because the app has exactly one page, but it is the seam where a navigation bar or a stylesheet will later be added once, for every page at the same time.

How it works. Read it top to bottom.

The first line, `<!doctype html>`, tells the browser to interpret the page using the modern HTML standard rather than an old compatibility mode. It is not a tag, it is a declaration, and it must come first.

`<html>` opens the document, and everything else nests inside it until the closing `</html>` on the last line.

The `<head>` section holds information about the page rather than visible content. `<meta charset="utf-8">` declares the text encoding, so the browser decodes bytes into characters correctly, which matters the moment a client folder is named with an accent or a non Latin script. `<title>TaxDesk</title>` sets the text shown in the browser tab.

The `<body>` section holds the visible page. `<h1>TaxDesk</h1>` is the top level heading, so every page of the app opens with the product name.

The line `{% block content %}{% endblock %}` is the Jinja part. Anything wrapped in `{% ... %}` is a Jinja control tag, an instruction to the template engine rather than text for the browser. The `block` tag carves out a named hole in the template, here a hole called `content`, and the matching `endblock` closes it. In the base file the hole is empty, there is nothing between the two tags, so rendering `base.html` alone would produce a page with just the heading. A child template that extends this file provides its own `block content`, and Jinja pastes the child's version into this hole. That is the entire inheritance mechanism, the base declares the hole, the child fills it.

onboarding.html

```
{% extends "base.html" %}
{% block content %}
<h2>Onboarding</h2>

{% if error %}
<p><strong>{{ error }}</strong></p>
{% endif %}

<form method="post" action="/onboarding/root">
  <label>
    Root client folder:
    <input type="text" name="root_folder" value="{{ root_folder or ' ' }}" size="60">
  </label>
  <button type="submit">Save root folder</button>
</form>

{% if configured_root and candidates is not none %}
  {% if candidates %}
    <h3>Select the folders that represent clients.</h3>
    <p>Current root: {{ configured_root }}</p>
    <p>
      We found {{ candidates | length }} folder{{ ' ' if candidates | length == 1 else 's' }}.
      {{ new_count }} new client folder{{ ' is' if new_count == 1 else 's are' }} selected.
    </p>
    <form method="post" action="/onboarding/confirm">
      {% for candidate in candidates %}
        <p>
          <label>
            <input type="checkbox" name="folders" value="{{ candidate['name'] }}"
              {% if candidate['existing'] %}disabled{% else %}checked{% endif %}>
            {{ candidate['name'] }}
            {% if candidate['existing'] %}(already added){% endif %}
          </label>
        </p>
      {% endfor %}
      <button type="submit">{{ submit_label }}</button>
    </form>
  {% else %}
    <p>No folders found inside the root. Create your client folders on
    disk, then come back here, this page never creates folders itself.</p>
  {% endif %}
{% endif %}
{% endblock %}
```

Why it exists. This is the whole onboarding screen, the root folder form and, once a root is saved, the confirmation list of candidate folders. One template covers every state of the screen, from the very first visit with nothing configured through an error, a populated candidate list, and an empty root, by branching on the values the routes pass in. Keeping all states in

one file means the reader, and the practitioner, always land on the same URL and the page simply shows more as more becomes known.

How it works. The rest of this section walks the file piece by piece. Each piece is quoted exactly as it appears above, so you can match it back to the full listing.

The extends line and the block

```
{% extends "base.html" %}
{% block content %}
<h2>Onboarding</h2>
```

The `extends` tag declares that this template is a child of `base.html`. Jinja does not strictly require it to come first, the docs only say it should be the first tag in the template, and this file follows that convention by opening with it. Rendering `onboarding.html` therefore really renders `base.html`, with this file's blocks pasted into the base's matching holes. `{% block content %}` opens the child's version of the `content` block you saw in the base, and everything down to the `{% endblock %}` on the last line of the file becomes the body of the page, sitting under the base's `<h1>TaxDesk</h1>`. The `<h2>Onboarding</h2>` heading is plain HTML, the page's own title one level below the site heading.

The error message

```
{% if error %}
<p><strong>{{ error }}</strong></p>
{% endif %}
```

The `if` tag renders its contents only when the condition is truthy, and `endif` closes it. Truthiness works as in Python, so this paragraph appears only when `error` is a non empty string, and disappears entirely when the routes pass `None`.

Inside the paragraph you meet the second kind of Jinja marker. Double braces, `{{ ... }}`, output a value into the page, where `{% ... %}` only controls flow and prints nothing itself. Output through double braces is automatically escaped in this app, because the environment that `Jinja2Templates` builds switches escaping on unconditionally, for every template it renders. Escaping means characters with meaning in HTML, such as `<`, `>`, and `&`, are converted to harmless entities like `<` before they reach the browser. That matters right here, because one of the two sources of `error` is the URL itself. In `onboarding_page` the value comes from the query string, the part of a URL after the question mark.

```
"error": request.query_params.get("error"),
```

Anyone can craft a link to this page with `?error=<script>...` in it, and without escaping that script would run in the visitor's browser. With escaping it renders as inert text. The other source is the failed save in `save_root`, where the route renders this same template directly with a fixed message and status code 400.

```
"error": "That path does not exist or is not a folder. Nothing was saved.",
```

The root folder form

```
<form method="post" action="/onboarding/root">
  <label>
    Root client folder:
    <input type="text" name="root_folder" value="{{ root_folder or ' ' }}" size="60">
  </label>
  <button type="submit">Save root folder</button>
</form>
```

A `<form>` groups inputs and a submit button, and its two attributes say what happens on submit. `method="post"` makes the browser send an HTTP POST, the verb for requests that change state, and `action="/onboarding/root"` is the URL it posts to, which is exactly the path the `save_root` route handles. The `<label>` wraps both its caption text and the input, a standard HTML pattern that makes clicking the words focus the text box.

The input line carries three attributes worth reading slowly. `type="text"` makes it a plain text box. `name="root_folder"` is the key the browser uses when it packages the form data, and it is the same key the route reads with `form.get("root_folder", "")`, so the template and the route agree on the field's name by this string. `value="{{ root_folder or ' ' }}"` prefills the box, and the `or ' '` is a small Jinja idiom you should memorize. Jinja's `or` works like Python's, it yields the left side when that is truthy and the right side otherwise. On the very first visit no root exists yet, `queries.get_root_folder(conn)` returns `None`, and Jinja would print `None` literally, putting the four letter word `None` inside the text box. The `or ' '` swaps `None` for an empty string, so the box starts blank. `size="60"` just makes the box sixty characters wide, roomy enough for a filesystem path.

Where the prefill value comes from depends on which route rendered the page. The GET handler passes the saved root so a returning visitor sees their current setting.

```
"root_folder": root_folder,
```

The failed save passes back exactly what was typed, so a typo can be fixed in place instead of retyped from scratch.

```
"root_folder": submitted,
```

The `<button type="submit">` triggers the form submission when clicked, and its caption is fixed text, Save root folder.

The discovery guard

```
{% if configured_root and candidates is not none %}
```

This single line decides whether the lower half of the page, the discovery panel, exists at all. It combines two conditions with `and`. The first, `configured_root`, is truthy only when a root path is actually saved in settings. The second uses a Jinja test, `candidates is not none`, where `is` applies a named test to a value and `none` is Jinja's lowercase spelling of Python's `None`.

The distinction between `None` and an empty list is deliberate and comes straight from `discovery_context` in the route module. When no root is configured it returns `None` for candidates, which means there was nothing to scan.

```
if root_folder is None:
    return {"candidates": None, "new_count": 0, "submit_label": "Add 0 clients"}
```

When a root is configured, the same function builds a real list, possibly empty, with one dictionary per immediate subfolder. A dictionary here is Python's key to value mapping, and each entry carries the folder's name and whether it is already a client. A set comprehension, the `{... for ...}` form that builds a set in one expression, collects the folder paths of existing clients first, so membership checks are fast and exact.

```
existing_paths = {client["FOLDER_PATH"] for client in queries.list_clients(conn)}
candidates = [
    {
        "name": folder.name,
        "existing": str(folder.resolve()) in existing_paths,
    }
    for folder in candidate_folders(root_folder)
]
new_count = sum(1 for candidate in candidates if not candidate["existing"])
```

So the guard reads as, show the panel only when a root is saved and a scan actually ran. An empty list passes this guard, because an empty scan is still information worth showing, and the inner branch handles it next.

The candidate list

```
{% if candidates %}
<h3>Select the folders that represent clients.</h3>
<p>Current root: {{ configured_root }}</p>
```

Inside the guard, a second `if` checks `candidates` for truthiness, and an empty list is falsy in Jinja just as in Python, so this branch runs only when the scan found at least one subfolder. The heading invites the selection, and the next paragraph echoes the root being scanned. `configured_root` comes from the routes as the saved root, in the GET handler it is the same value as `root_folder`, and in the failed save it is deliberately the old, still valid root rather than the rejected new path, so the panel below keeps describing the root that is actually in effect.

```
<p>
    We found {{ candidates | length }} folder{{ ' ' if candidates | length == 1 else 's' }}.
    {{ new_count }} new client folder{{ ' is' if new_count == 1 else 's are' }} selected.
</p>
```

Two new pieces of Jinja appear here. The pipe character applies a filter, a small function that transforms a value, and the `length` filter returns how many items a list holds, so `{{ candidates | length }}` prints the folder count. Right after it comes an inline conditional expression, `{{ ' ' if candidates | length == 1 else 's' }}`, which is Jinja's version of Python's one line if else. It prints nothing when the count is one and the letter s otherwise, turning folder into folders exactly when grammar demands it. The second sentence plays the same trick twice at once, printing `new_count` and then choosing between the words is and are along with the plural, so the page says 1 new client folder is selected or 3 new client folders are selected. `new_count` is computed in `discovery_context` as the number of candidates not already added, the `sum` line you saw above.

```

<form method="post" action="/onboarding/confirm">
  {% for candidate in candidates %}
  <p>
    <label>
      <input type="checkbox" name="folders" value="{{ candidate['name'] }}"
        {% if candidate['existing'] %}disabled{% else %}checked{% endif %}>
      {{ candidate['name'] }}
      {% if candidate['existing'] %}(already added){% endif %}
    </label>
  </p>
  {% endfor %}
  <button type="submit">{{ submit_label }}</button>
</form>

```

This second form posts to `/onboarding/confirm`, the route that actually creates clients. The `for` tag is Jinja's loop, it repeats everything down to `endfor` once per item, binding each candidate dictionary to the name `candidate` in turn, so one paragraph with one checkbox is emitted per folder.

Each checkbox deserves a careful read. All of them share `name="folders"`, and when several controls share a name the browser submits one value per ticked box under that key, which is exactly why the route reads them with `form.getlist("folders")`, the plural reader that returns every value for a repeated key. The `value` attribute is the folder's name, pulled from the dictionary with the subscript syntax `candidate['name']`, and escaped on output like every double brace.

The attribute area then splits on `candidate['existing']` with an `if` and `else` placed inside the tag itself, a legal and common Jinja move, the engine just decides which word lands in the HTML. A folder that is already a client gets the `disabled` attribute, and a new folder gets `checked`. The two attributes encode the page's whole policy. New folders arrive ticked, so the common case, add everything I just created, is one click. Already added folders arrive disabled, which does two things in the browser. The visitor cannot toggle the box, and, by a rule of HTML forms, a disabled control is never included in the data the browser submits. So when this form posts, the `folders` list contains only genuinely new, still ticked names, and an existing client can never even be proposed again from this page.

That said, the server does not trust the template to enforce this, because a POST can be crafted by hand without any browser. `confirm_clients` scans the real root again and accepts only names that match an actual current subfolder, ignoring everything else.

```

actual_folders = {
    folder.name: folder for folder in candidate_folders(root_folder)
}

form = await request.form()
for name in form.getlist("folders"):
    folder = actual_folders.get(str(name))
    if folder is None:
        continue

```

The disabled checkbox is the polite front door, the fresh scan and the database's UNIQUE rule are the locks. Belt and braces, and the belt lives in this template.

After the checkbox, `{{ candidate['name'] }}` prints the folder name as the visible label text, and a small `if` appends the marker (already added) next to disabled entries, so the visitor knows why a box cannot be ticked.

The submit button's caption is not fixed text this time, it is `{{ submit_label }}`, built in `discovery_context` with an f-string, Python's inline formatting syntax where expressions inside braces are evaluated into the string.

```
return {
    "candidates": candidates,
    "new_count": new_count,
    "submit_label": f"Add {new_count} client{' ' if new_count == 1 else 's'}",
}
```

So the button itself says Add 2 clients, a live restatement of what pressing it will do, and it agrees with the sentence above it because both derive from the same `new_count`.

The empty root message

```
{% else %}
<p>No folders found inside the root. Create your client folders on
disk, then come back here, this page never creates folders itself.</p>
{% endif %}
{% endif %}
{% endblock %}
```

The `else` pairs with the inner `{% if candidates %}`, so this paragraph renders when a root is configured and the scan ran but produced an empty list. That happens for a genuinely empty root, and also, if you recall `candidate_folders`, when the root has vanished or cannot be read, since that helper returns an empty list on those failures too. The message states the module's contract out loud, onboarding reads the disk and never writes it, so the fix belongs in the file manager, not here. The two `endif` lines close the inner branch and the outer guard, and `endblock` closes the `content` block that `extends` promised to fill, ending the file.

The four page states

Every combination of route values lands the visitor in one of four screens, all rendered from this one template.

- no root: the first visit, `get_root_folder` returns `None`, so `root_folder` and `configured_root` are `None` and `discovery_context` returns `candidates` as `None`. The guard fails on both sides and the page is just the heading and an empty form.
- error: `error` is a non empty string, so the bold paragraph appears above the form. From a failed save the box echoes the rejected `submitted` text and the response carries status 400, while `configured_root` stays the old saved root, so a previously working discovery panel remains visible beneath the complaint. The GET route can show the same paragraph from an `?error=` query parameter.
- candidates found: a root is saved and the scan found subfolders, so the guard and the inner `if` both pass. The panel shows the current root, the counts sentence, one checkbox per folder, new ones ticked, existing ones disabled and marked, and a button whose caption counts the additions.
- empty root: a root is saved but the scan returned an empty list, so the guard passes and the inner `if` falls to `else`, showing the no folders found paragraph in place of the list.

Where every value comes from

For quick reference, each name the template uses traces to one place in `app/routes/onboarding.py`.

- `error`: `request.query_params.get("error")` in `onboarding_page`, or the fixed rejection message in `save_root`
- `root_folder`: `queries.get_root_folder(conn)` in `onboarding_page`, or the echoed `submitted` text in `save_root`
- `configured_root`: the saved root in both routes, `root_folder` in the GET handler and the freshly fetched `configured_root` in the failed save
- `candidates`: built in `discovery_context`, `None` without a root, otherwise one dictionary per subfolder from `candidate_folders`
- `candidate['name']` and `candidate['existing']`: the two keys of each dictionary in that list, the folder's name and whether its resolved path already sits in the clients table
- `new_count`: the `sum` over candidates in `discovery_context`, counting entries where `existing` is false
- `submit_label`: the f-string in `discovery_context`, Add N clients with the same singular and plural logic the template uses for its own sentences

Notice the division of labor. The routes decide what is true, the template decides what is shown, and every branch in the HTML corresponds to a value the route computed on purpose. When you add the next page to TaxDesk, you will extend `base.html` the same way, fill the same `content` block, and hand it a dictionary whose every key you can point to in a route.

5.8 The Database Tests

TaxDesk keeps everything it knows in a single SQLite file, and the two test modules in this chapter are the safety net around that file. The tests in `tests/test_database.py` pin down the migration runner, the code that builds the schema from numbered SQL files and records what it has applied, so a database can be created once and upgraded forever without losing data. The tests in `tests/test_seed.py` pin down the development seed, the script that fills a fresh database with five sample clients and their filing tasks so a developer has something realistic to look at. Together they answer the two questions every storage layer must answer before anyone builds on top of it, does the schema come out right on any machine, and can the setup scripts run twice without wrecking anything. Every later feature, onboarding, task generation, document tracking, quietly assumes the guarantees proved here.

The File `tests/test_database.py`

We start with the migration tests. The file opens with a docstring and its imports.

```
"""Tests for the migration runner and the schema's rules."""

import sqlite3
from collections.abc import Iterator
from pathlib import Path

import pytest
```

```
from database import migrate
from database.migrate import connect, initialize
```

Why it exists. Every test in this file drives the real migration code, not a copy of it, so the imports pull in the actual `database.migrate` module. The docstring states the file's two subjects, the runner that applies migrations and the rules the finished schema enforces. Keeping both in one file makes sense because both are properties of the same `initialize()` call.

How it works. `import sqlite3` brings in Python's bundled SQLite driver, the tests need it for the connection type and for its exception classes such as `sqlite3.IntegrityError`. `Iterator` comes from `collections.abc` and is used purely as a type annotation, you will see it on the fixture below. `Path` from `pathlib` is Python's object for filesystem paths, it lets you join paths with the `/` operator instead of gluing strings. `pytest` is the test framework itself, imported so the file can use its decorators and helpers. The last two lines look redundant but are not. `from database import migrate` imports the module object as a whole, which two tests later need so they can swap out variables that live on the module, `test_failed_migration_leaves_no_trace` patches `migrate.MIGRATIONS_DIR` and `test_new_required_service_reaches_an_existing_database` patches `migrate.REQUIRED_SERVICES`, both through `monkeypatch.setattr` on that module object. `from database.migrate import connect, initialize` then imports the two functions every test calls by name, `connect()` opens a database file and `initialize()` runs any pending migrations against it.

The db Fixture

```
@pytest.fixture
def db(tmp_path: Path) -> Iterator[sqlite3.Connection]:
    """Give a test a fresh initialized database in a temp folder.

    In: pytest's tmp_path, a unique folder per test.
    Out: an open connection, closed automatically after the test.
    """
    conn = connect(tmp_path / "test.db")
    initialize(conn)
    yield conn
    conn.close()
```

Why it exists. Most tests here need the same starting point, a brand new database that has already been through the full migration run. Writing that setup inside every test would repeat four lines a dozen times and, worse, would make it easy for one test to forget the cleanup. The fixture centralizes both the setup and the teardown.

How it works. This is our first pytest fixture, so a quick definition. A fixture is a function marked with the `@pytest.fixture` decorator, a decorator being a line starting with `@` that wraps the function below it with extra behavior. When a test declares a parameter named `db`, pytest sees a fixture with that name exists, runs it, and passes the result into the test. The fixture itself asks for `tmp_path`, which is a fixture that ships with pytest. It hands you a `Path` to a freshly created temporary folder that is unique to each individual test, so no test can ever see another test's files, and pytest cleans old ones up on its own. The body opens a connection with `connect(tmp_path / "test.db")`, the `/` here is `pathlib` path joining, then calls `initialize(conn)` to apply every migration. The `yield conn` line is the interesting one. A function containing `yield` is a generator, a function that can pause and hand a value out mid execution. Pytest exploits that shape, everything before `yield` is setup, the yielded value is what the test receives, and

everything after `yield` runs as teardown once the test finishes, pass or fail. So `conn.close()` is guaranteed to run, which matters on Windows where an open handle can block the temp folder from being removed. The return annotation `Iterator[sqlite3.Connection]` is the honest type of a generator that yields connections.

test_fresh_database_gets_all_tables

```
def test_fresh_database_gets_all_tables(db: sqlite3.Connection) -> None:
    """Initialization on a new file must create every table."""
    rows = db.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
    names = {row[0] for row in rows}

    expected = {
        "CLIENTS",
        "SERVICES",
        "CLIENT_SERVICES",
        "PERIODS",
        "TASKS",
        "documents",
        "SETTINGS",
    }
    assert expected <= names
```

Why it exists. This is the smoke test for the whole migration pipeline. If `initialize()` fails to run a file, runs them out of order, or points at the wrong migrations folder, the very first symptom is a missing table, and this test catches that before any subtler test gets confused by it.

How it works. The test receives the `db` fixture, so by its first line a fully migrated database already exists. It queries `sqlite_master`, which is SQLite's own internal catalog table, every database carries one, and each row describes one object in the schema. Filtering on `type = 'table'` keeps only tables. The next line is a set comprehension, the `{expression for item in iterable}` syntax that builds a set in one line, here collecting `row[0]`, the name column, from every result row into a set of table names. The `expected` set lists the seven tables the application needs. Notice `documents` is lowercase while the rest are uppercase, SQLite stores each name exactly as its `CREATE TABLE` statement spelled it, so the test must match the schema files letter for letter. The final assertion uses `<=` between two sets, which for sets means subset, every expected name must appear among the actual names. Subset rather than equality is deliberate, the runner also creates its own bookkeeping table, `schema_applied`, and demanding exact equality would make the test fail every time housekeeping tables change.

What breaks it. A migration file that never gets picked up, a typo in a `CREATE TABLE` name, or a runner bug that stops after the first file would all leave a hole in `names` and fail the subset check.

test_second_initialize_keeps_existing_data

```
def test_second_initialize_keeps_existing_data(tmp_path: Path) -> None:
    """Running initialization again must be a no-op, never a reset."""
    conn = connect(tmp_path / "test.db")

    assert initialize(conn) is True
    conn.execute("INSERT INTO CLIENTS (NAME, FOLDER_PATH) VALUES ('Alpha', '/c/Alpha')")
    conn.commit()

    assert initialize(conn) is False
```

```
count = conn.execute("SELECT COUNT(*) FROM CLIENTS").fetchone()[0]
assert count == 1

conn.close()
```

Why it exists. TaxDesk runs `initialize()` on every application start, against a database that already holds real client data. If that call ever behaved like a reset instead of a check, a user would lose everything on a routine restart. This test is the contract that a second run touches nothing.

How it works. This test skips the `db` fixture on purpose and asks for `tmp_path` directly, because it needs to call `initialize()` itself, twice, and inspect the return value each time. The first call must return `True`, the runner's signal that it actually applied migrations. The test then inserts one client row and calls `conn.commit()`, which makes the insert permanent, SQLite groups writes into transactions and `commit` is the step that seals one. Now the crucial move, `initialize()` is called a second time on the same connection. The assertion `is False` pins the return value, nothing was pending, nothing was applied. Then `SELECT COUNT(*)` counts the client rows, and the chained `.fetchone()[0]` idiom appears for the first time, `fetchone()` returns the first result row, a `sqlite3.Row` here since `connect()` sets the row factory, and `[0]` takes its first column by position, the count. It must still be 1, the Alpha row survived. The connection is closed by hand at the end because no fixture is doing it for this test.

What breaks it. A runner that drops and recreates tables instead of checking its logbook, or one that reapplies `001_schema.sql` because it failed to record it the first time, would either wipe the Alpha row or crash on a duplicate table, and either way this test goes red.

test_settings_allows_exactly_one_row

```
def test_settings_allows_exactly_one_row(db: sqlite3.Connection) -> None:
    """The CHECK on ID must make a second configuration row impossible,
    and the root folder must never be saved empty."""
    db.execute("INSERT INTO SETTINGS (ID, ROOT_FOLDER) VALUES (1, '/clients')")

    with pytest.raises(sqlite3.IntegrityError):
        db.execute("INSERT INTO SETTINGS (ID, ROOT_FOLDER) VALUES (2, '/other')")

    with pytest.raises(sqlite3.IntegrityError):
        db.execute("UPDATE SETTINGS SET ROOT_FOLDER = NULL WHERE ID = 1")
```

Why it exists. The `SETTINGS` table is TaxDesk's configuration store and the design says it holds exactly one row, whose `ROOT_FOLDER` points at the folder full of client directories. Two rows would mean two competing configurations and a null root folder would mean the app has nowhere to scan. The schema enforces both rules with constraints, and this test proves those constraints are really in force after the runner builds the table.

How it works. The first `INSERT` stores the legitimate single row with `ID` 1. Then comes `pytest.raises` inside a `with` statement, our first context manager, an object the `with` statement drives so that code runs on entering the block and again on leaving it. `pytest.raises(sqlite3.IntegrityError)` inverts the usual meaning of an exception, the block passes only if that exact exception is raised inside it and fails if the code runs cleanly. `sqlite3.IntegrityError` is the error SQLite raises when a constraint is violated. The first guarded block tries to insert a row with `ID` 2, which the

schema's `CHECK` constraint on `ID` must reject, a `CHECK` constraint being a rule written in the table definition that every row must satisfy. The second guarded block attacks from a different angle, it keeps the one legal row but tries to `UPDATE` its `ROOT_FOLDER` to `NULL`, which the column's not null rule must refuse.

What breaks it. Someone loosening the schema during a migration edit, dropping the `CHECK` while reshaping the table or forgetting `NOT NULL` on the recreated column, would make one of these forbidden statements silently succeed, and the test would fail on the exception that never arrived.

test_migrations_recorded_in_order

```
def test_migrations_recorded_in_order(tmp_path: Path) -> None:
    """A fresh database must record both migration files, and a second
    initialize call must find nothing pending."""
    conn = connect(tmp_path / "test.db")

    assert initialize(conn) is True
    recorded = [
        row[0]
        for row in conn.execute("SELECT filename FROM schema_applied ORDER BY filename")
    ]
    assert recorded == ["001_schema.sql", "002_settings.sql"]

    assert initialize(conn) is False
    conn.close()
```

Why it exists. The migration runner's whole trick is a logbook table, `schema_applied`, that remembers which files have already run. If the logbook is wrong the runner either reapplies a file, crashing into tables that already exist, or skips a file forever. This test checks the logbook directly rather than trusting side effects.

How it works. Again the test manages its own connection because it needs the return values of `initialize()`. After the first call, asserted `True`, it reads every `filename` from `schema_applied`, ordered alphabetically so the comparison is stable. The square bracket expression is a list comprehension, the list building sibling of the set comprehension from earlier, and it collects the first column of each row into a plain list. The assertion pins the exact contents, the two real migration files of the project, `001_schema.sql` and `002_settings.sql`, and nothing else. The numeric prefixes are what give migrations their order, alphabetical order of the filenames is execution order. Finally `initialize()` runs once more and must return `False`, proving the runner consulted the logbook and found nothing left to do.

What breaks it. A runner that executes a file but forgets to insert its logbook row fails the list comparison. A runner that writes the row but keys it wrongly, or ignores the logbook when scanning for pending work, fails the final `is False`. Adding a third migration to the project also fails this test, which is intentional, the author must come here and extend the expected list, acknowledging the new file.

test_failed_migration_leaves_no_trace

```
def test_failed_migration_leaves_no_trace(
    tmp_path: Path,
    monkeypatch: pytest.MonkeyPatch,
) -> None:
    """A migration failing halfway must roll back completely, neither
    its tables nor its logbook record may survive, so the rerun starts
    clean instead of crashing into half applied structure."""
```

```

migrations = tmp_path / "migrations"
migrations.mkdir()
(migrations / "001_good.sql").write_text(
    "CREATE TABLE GOOD (X INTEGER);"
)
(migrations / "002_broken.sql").write_text(
    "CREATE TABLE PARTIAL (X INTEGER);"
    " INSERT INTO NO_SUCH_TABLE VALUES (1);"
)
monkeypatch.setattr(migrate, "MIGRATIONS_DIR", migrations)

conn = connect(tmp_path / "test.db")
with pytest.raises(sqlite3.OperationalError):
    initialize(conn)

tables = {
    row[0]
    for row in conn.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
}
assert "GOOD" in tables
assert "PARTIAL" not in tables

recorded = [
    row[0] for row in conn.execute("SELECT filename FROM schema_applied")
]
assert recorded == ["001_good.sql"]

# The connection must be usable afterwards, nothing left open.
assert conn.in_transaction is False
conn.close()

```

Why it exists. This is the most important test in the file. Migrations run against databases that hold real data, and a migration that dies halfway must not leave half of its work behind, because a later rerun would then collide with the leftovers and the database would be stuck, too broken to migrate and too migrated to start over. The test forces exactly that crash and demands a perfect rollback.

How it works. The project's real migration files are all correct, so the test has to manufacture a broken one. It builds its own migrations folder inside `tmp_path`, creating the directory with `mkdir()`, then writes two SQL files with `write_text()`. The first, `001_good.sql`, is a healthy single statement creating table `GOOD`. The second, `002_broken.sql`, is the trap, and notice how its content is written as two string literals sitting next to each other with no operator between them. That is implicit string concatenation, Python joins adjacent literals into one string at compile time, a common way to lay a long SQL string across lines. The resulting file holds two statements, a valid `CREATE TABLE PARTIAL` followed by an `INSERT` into `NO_SUCH_TABLE`, a table that does not exist. So when the runner executes this file, the first statement succeeds and the second explodes, which is precisely the halfway failure the docstring describes.

Then comes the redirection. `monkeypatch` is another fixture that ships with `pytest`, and its job is temporary substitution, `monkeypatch.setattr(migrate, "MIGRATIONS_DIR", migrations)` replaces the `MIGRATIONS_DIR` variable on the `migrate` module object with the temp folder, and `pytest` automatically restores the original value when the test ends, so no other test ever sees the fake folder. This is why the file imported `migrate` as a whole module at the top, you need the module object in hand to patch an attribute on it.

With the trap set, the test opens a connection and calls `initialize()` inside `pytest.raises(sqlite3.OperationalError)`, the error SQLite raises for a statement that references a missing table. The failure is required, swallowing it would itself be a bug. Now the forensics. The set comprehension over `sqlite_master` gathers the surviving tables, `GOOD` must be there because its file completed and was committed, and `PARTIAL` must not be, even though its `CREATE TABLE` statement succeeded before the crash. That pair of assertions is the rollback proof, the runner must wrap each migration file in a transaction so that a failure undoes every statement of that file. The logbook check makes the same demand on the bookkeeping, `schema_applied` must list `001_good.sql` alone, no record of the broken file may exist, otherwise a rerun would skip it forever and the fix would never apply. The last assertion reads `conn.in_transaction`, a property of the connection that is true while a transaction is open, it must be `False`, meaning the runner cleaned up after the exception instead of leaving the caller inside a dangling transaction. The comment above it earns its place by stating that intent, which the bare property read does not say on its own.

What breaks it. A runner that executes statements with autocommit and no transaction per file leaves `PARTIAL` behind. A runner that inserts the logbook row before executing the file leaves a lying record of `002_broken.sql`. A runner whose error handling forgets the rollback leaves `in_transaction` true. Each of those is a realistic one line mistake, and each one turns a recoverable crash into permanent corruption.

test_new_required_service_reaches_an_existing_database

```
def test_new_required_service_reaches_an_existing_database(
    tmp_path: Path,
    monkeypatch: pytest.MonkeyPatch,
) -> None:
    """A service appended to REQUIRED_SERVICES must appear on the next
    initialize call, without editing or re-running schema.sql."""
    conn = connect(tmp_path / "test.db")
    initialize(conn)

    monkeypatch.setattr(
        migrate,
        "REQUIRED_SERVICES",
        [*migrate.REQUIRED_SERVICES, "PT"],
    )

    assert initialize(conn) is False

    names = {row[0] for row in conn.execute("SELECT NAME FROM SERVICES")}
    assert "PT" in names
    assert len(names) == 5

    conn.close()
```

Why it exists. The four standard services live in a Python list, `REQUIRED_SERVICES`, and `initialize()` tops up the `SERVICES` table from that list on every run. The point of that design is that adding a fifth service one day should take one line of Python and no new migration file. This test proves the design actually delivers that, on a database that already exists.

How it works. The test builds and initializes a database the normal way, simulating an installation already out in the field. Then it uses `monkeypatch.setattr` again, this time to replace the `REQUIRED_SERVICES` list on the `migrate` module object with an extended copy, the second of the two patches that the whole module import exists to serve. The expression `[*migrate.REQUIRED_SERVICES, "PT"]` uses the star unpacking idiom, `*` spreads the existing list's items into a new

list literal, followed by the new entry "PT", so the original list object is never mutated, only shadowed for the duration of the test. The second `initialize()` call must still return `False`, because no migration files are pending, topping up services is not a migration. Yet the set comprehension over `SERVICES` must now contain "PT", and `len(names) == 5` confirms the arithmetic, the four originals plus the newcomer, with no duplicates of the existing four.

What breaks it. If the service top up only ran on a brand new database, existing installations would never receive the new service and "PT" would be missing. If the top up used a plain `INSERT` instead of `INSERT OR IGNORE`, the second `initialize()` would crash with an `IntegrityError` on the four existing names, since `SERVICES.NAME` is `UNIQUE`, and the test would fail before its assertions were even reached. And if someone moved service insertion into a migration file after all, the `is False` assertion would flag the change of contract.

test_foreign_keys_are_enforced

```
def test_foreign_keys_are_enforced(db: sqlite3.Connection) -> None:
    """connect() must switch the per connection foreign key pragma on."""
    assert db.execute("PRAGMA foreign_keys").fetchone()[0] == 1

    with pytest.raises(sqlite3.IntegrityError):
        db.execute(
            "INSERT INTO documents (client_id, filename, path)"
            " VALUES (99, 'x.pdf', '/x.pdf')"
        )
```

Why it exists. A foreign key is a column that must point at an existing row in another table, here `documents.client_id` must point at a real client. SQLite has a famous trap, it parses foreign key constraints but does not enforce them unless each connection switches enforcement on with a pragma, a pragma being SQLite's mechanism for connection and database settings. `connect()` is supposed to flip that switch every time, and this test makes sure it does.

How it works. The first assertion asks SQLite directly, `PRAGMA foreign_keys` with no argument reads the current setting, and `fetchone()[0]` must be `1`, meaning on. That alone could pass while enforcement subtly fails, so the second half stages a real violation, inserting a document row whose `client_id` is 99 in a database whose `CLIENTS` table is empty. With enforcement on, SQLite must refuse with `IntegrityError`, and `pytest.raises` demands exactly that. The SQL string is again split across two adjacent literals joined by implicit concatenation.

What breaks it. Removing the pragma line from `connect()` would make the orphan insert quietly succeed and turn this test red. A second code path that opens connections without going through `connect()` is the same danger in production, though this test alone would stay green, it drives the sanctioned door and only that one, which is exactly why the project bans every other way of opening the database.

test_schema_constraints_hold_through_the_runner

```
def test_schema_constraints_hold_through_the_runner(db: sqlite3.Connection) -> None:
    """The rules written in schema.sql must survive the runner path."""
    db.execute("INSERT INTO CLIENTS (NAME, FOLDER_PATH) VALUES ('Alpha', '/c/Alpha')")
    # Services come from initialization now, look one up instead of inserting.
    service = db.execute("SELECT ID FROM SERVICES WHERE NAME = 'GSTR-3B']").fetchone()[0]
    db.execute("INSERT INTO PERIODS (YEAR, MONTH) VALUES (2026, 8)")
    db.execute(
        "INSERT INTO TASKS (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH)"
        " VALUES (1, ?, 2026, 8)",
```

```

        (service,),
    )

    with pytest.raises(sqlite3.IntegrityError):
        db.execute(
            "INSERT INTO TASKS (CLIENT_ID, SERVICE_ID, PERIOD_YEAR, PERIOD_MONTH) "
            " VALUES (1, ?, 2026, 8)",
            (service,),
        )

    with pytest.raises(sqlite3.IntegrityError):
        db.execute("UPDATE TASKS SET STATUS = 'Done' WHERE ID = 1")

```

Why it exists. Constraints can exist in `schema.sql` and still be lost in transit, a migration edit or a table rebuild can drop them without any error. This test exercises two of the most valuable rules on `TASKS` end to end, through the same runner path production uses, so the schema file and the live database cannot silently disagree.

How it works. The first block assembles the minimum world a task needs, one client, one service, one period. The comment explains a piece of history that the code cannot, services are no longer inserted by hand because initialization now provides them, so the test looks up the `ID` of `GSTR-3B` instead of creating it. Then it inserts the period for August 2026 and one task tying client 1 to that service and period. Note the `?` in the task insert, that is a parameterized query, the value travels separately in the tuple `(service,)` instead of being pasted into the SQL string, which is the safe way to pass data into SQL. The one element tuple needs its trailing comma, `(service)` without it would just be parentheses around a variable.

The first `pytest.raises` block repeats the identical insert, same client, same service, same period, and demands rejection. That pins the uniqueness rule on `TASKS`, one task per client per service per period, the rule that makes the seed rerunnable and stops duplicate filing work from ever being created. The second block updates the task's `STATUS` to `'Done'` with a capital D and demands rejection too. The schema restricts `STATUS` to a fixed set of lowercase values, the seed tests in the next file use `'done'`, `'pending'`, and `'not_applicable'`, and the check is case sensitive, so `'Done'` is an illegal value.

What breaks it. Losing the unique constraint during a `TASKS` migration would let task generation insert duplicates every run, and losing or loosening the status check would let a typo like `'Done'` create a task that every status filter in the application ignores forever. Both failures are invisible at write time, which is exactly why the database, not the application code, must be the enforcer.

The File `tests/test_seed.py`

The second file tests the development seed. Same opening pattern, a docstring and imports.

```

"""Tests for the development seed and the services in initialization."""

import sqlite3
from collections.abc import Iterator
from pathlib import Path

import pytest

from database.migrate import connect, initialize
from database.seed import (

```

```

    mark_sample_statuses,
    seed_clients,
    seed_period_and_tasks,
)

```

Why it exists. The seed module fills a fresh database with believable sample data, five clients, their service subscriptions, one filing period, and the tasks that follow, plus a few tasks nudged into interesting statuses. These tests keep that sample data trustworthy, because developers eyeball it daily and other tests may lean on its shape.

How it works. The first four imports mirror the previous file, the SQLite driver, the `Iterator` and `Path` annotations, and `pytest`. `connect` and `initialize` come from the migration module because every seed test needs a migrated database first. The last import pulls the three seed steps from `database.seed`, written in the parenthesized multi line form with a trailing comma, one name per line, the style that keeps diffs small when a name is added. The three functions are `seed_clients`, which creates the sample clients and their subscriptions, `seed_period_and_tasks`, which creates the period and generates tasks from active subscriptions, and `mark_sample_statuses`, which flips a couple of tasks out of pending for realism. The list names the steps but do not read it as the pipeline, it is sorted alphabetically, which puts `mark_sample_statuses`, the last step, first. The real execution order lives in the `run_seed` helper just below.

The `run_seed` Helper

```

def run_seed(conn: sqlite3.Connection) -> None:
    """Run the full seed sequence on an initialized database.

    In: an open connection after initialize().
    Out: nothing, same steps as the command line entry point.
    """
    seed_clients(conn)
    seed_period_and_tasks(conn)
    mark_sample_statuses(conn)
    conn.commit()

```

Why it exists. Every test in this file wants the whole seed, not one step of it, and the steps must run in the same order the real command line entry point uses, otherwise the tests would be validating a sequence nobody actually runs. Wrapping the sequence in one helper keeps that order defined in exactly one place inside the test file.

How it works. This is a plain function, not a fixture, because tests want to choose when it runs, sometimes twice in one test. It calls the three seed steps in dependency order, clients must exist before tasks can reference them, and tasks must exist before statuses can be marked on them. The final `conn.commit()` seals all three steps into the database, matching what the real entry point does. The docstring's In and Out lines state the contract, an initialized connection in, nothing returned, side effects only.

The `db` Fixture in This File

```

@pytest.fixture
def db(tmp_path: Path) -> Iterator[sqlite3.Connection]:
    """Give a test a fresh initialized database in a temp folder.

    In: pytest's tmp_path, a unique folder per test.
    Out: an open connection, closed automatically after the test.
    """

```

```

conn = connect(tmp_path / "test.db")
initialize(conn)
yield conn
conn.close()

```

Why it exists. The seed tests need the identical starting point the migration tests used, a fresh migrated database, so the fixture is repeated here line for line. Fixtures are looked up per file unless placed in a shared `conftest.py`, and at two small copies the duplication is still cheaper than the indirection of a shared file.

How it works. Exactly as before. `tmp_path` supplies a private temp folder, `connect` opens `test.db` inside it, `initialize` migrates it, `yield` hands the connection to the test, and the line after `yield` closes the connection during teardown. One thing the fixture deliberately does not do is run the seed. The very first test below inspects the unseeded state, and every test that wants seed data calls `run_seed(db)` itself, so seeding stays an explicit, visible step in each test that uses it.

test_initialization_provides_the_four_services

```

def test_initialization_provides_the_four_services(db: sqlite3.Connection) -> None:
    """A fresh database must already know the four filing types."""
    rows = db.execute("SELECT NAME FROM SERVICES ORDER BY NAME")
    names = [row[0] for row in rows]

    assert names == ["EPF", "ESI", "GSTR-1", "GSTR-3B"]

```

Why it exists. The four filing types, the two GST returns plus the EPF and ESI payroll filings, are supposed to arrive with initialization itself, not with the seed. This test pins that ownership boundary, a production database that is never seeded must still know all four services.

How it works. Note that the test does not call `run_seed`, that omission is the whole point. It selects every service name, ordered alphabetically so the result is deterministic, gathers them with a list comprehension, and compares against the exact expected list. Using a list with `==` rather than a set checks completeness and absence of duplicates in one stroke, exactly four rows, exactly these names.

What breaks it. Moving service creation into the seed module would empty this list on unseeded databases and fail the test. A duplicate insert of a service during initialization would produce five rows and fail it too.

test_seed_creates_expected_rows

```

def test_seed_creates_expected_rows(db: sqlite3.Connection) -> None:
    """Five clients, twelve subscriptions, one inactive, eleven tasks."""
    run_seed(db)

    clients = db.execute("SELECT COUNT(*) FROM CLIENTS").fetchone()[0]
    subscriptions = db.execute("SELECT COUNT(*) FROM CLIENT_SERVICES").fetchone()[0]
    active = db.execute(
        "SELECT COUNT(*) FROM CLIENT_SERVICES WHERE ACTIVE = 1"
    ).fetchone()[0]
    tasks = db.execute("SELECT COUNT(*) FROM TASKS").fetchone()[0]

    assert clients == 5
    assert subscriptions == 12

```

```
assert active == 11
assert tasks == active
```

Why it exists. The seed's shape is part of its value, five clients, twelve subscriptions of which one is switched off, and one task per active subscription. Developers reason about the sample data by those numbers, so the test freezes them.

How it works. After running the full seed, the test takes four counts with the now familiar `.fetchone()[0]` idiom, total clients, total rows in `CLIENT_SERVICES`, the join table that says which client subscribes to which service, the subset of those with `ACTIVE = 1`, and total tasks. The first three assertions pin the literal numbers from the docstring. The fourth is the smartest line in the test, `tasks == active` asserts the relationship rather than the number eleven, task generation must produce exactly one task per active subscription, no more and no fewer.

What breaks it. A generation bug that produced a task for every subscription regardless of the active flag would make `tasks` twelve while `active` stays eleven. A generation loop that ran per service instead of per subscription would inflate the task count. Editing the sample data without updating this test fails it immediately, which forces the docstring numbers to stay honest.

test_generation_skips_inactive_subscriptions

```
def test_generation_skips_inactive_subscriptions(db: sqlite3.Connection) -> None:
    """The switched off subscription must produce no task."""
    run_seed(db)

    row = db.execute(
        "SELECT COUNT(*) FROM TASKS t"
        " JOIN CLIENTS c ON c.ID = t.CLIENT_ID"
        " JOIN SERVICES s ON s.ID = t.SERVICE_ID"
        " WHERE c.NAME = 'Bhima Textiles' AND s.NAME = 'GSTR-1'"
    ).fetchone()

    assert row[0] == 0
```

Why it exists. The previous test showed the totals line up, but totals can lie, eleven tasks could include one for the inactive subscription while missing one elsewhere and the counts would still match. This test names the specific subscription that is switched off in the sample data, Bhima Textiles on GSTR-1, and asserts that precisely that task does not exist.

How it works. The query is the file's first `JOIN`, a `JOIN` combines rows from two tables by a matching condition. Here `TASKS` is joined to `CLIENTS` on the client id and to `SERVICES` on the service id, with short aliases `t`, `c`, and `s` standing in for the table names, so the `WHERE` clause can filter by the human readable names instead of numeric ids. The SQL is one string built from four adjacent literals through implicit concatenation, mind the leading space on each continuation piece, without those spaces the words would fuse and the SQL would be garbage. The count of tasks for that exact client and service pair must be zero.

What breaks it. A `WHERE ACTIVE = 1` clause dropped from the generation query, or an active check written against the wrong column, would create the Bhima Textiles GSTR-1 task and turn this zero into a one. Because the test names the pair explicitly, the failure message points straight at the culprit.

The snapshot Helper

```
def snapshot(conn: sqlite3.Connection) -> dict[str, list[tuple]]:
    """Capture the complete rows of every seeded table.

    In: an open connection.
    Out: a dict of table name to all of its rows, deterministically
    ordered, so two snapshots compare as whole database states.
    """
    tables = ("CLIENTS", "SERVICES", "CLIENT_SERVICES", "PERIODS", "TASKS")
    return {
        table: conn.execute(f"SELECT * FROM {table} ORDER BY 1, 2").fetchall()
        for table in tables
    }
```

Why it exists. The idempotency test that follows needs to say the entire database is unchanged, and the only honest way to say that is to capture every row of every seeded table and compare the captures. This helper is that camera.

How it works. The five seeded tables are listed in a tuple, and the return value is built with a dict comprehension, the `{key: value for item in iterable}` syntax that constructs a dictionary in one expression, mapping each table name to all of its rows. The query uses an f-string, a string literal prefixed with `f` in which `{table}` is replaced by the variable's value, to splice the table name into `SELECT * FROM {table}`. Splicing values into SQL strings is normally forbidden, that is how injection happens, but here the spliced values come from a fixed tuple written in this very function, no outside input can ever reach it, so the shortcut is safe. Table names also cannot be passed as `?` parameters, which is why the f-string is used at all. `ORDER BY 1, 2` sorts by the first and second columns positionally, giving every capture a deterministic row order so two snapshots of equal states compare equal instead of differing by arbitrary row order. `fetchall()` materializes all the rows as a list, and each row is a `sqlite3.Row`, the project's `connect()` sets that row factory, so the annotation's `list[tuple]` describes how the rows behave, tuple like and indexable by position, rather than their exact class.

test_second_seed_run_changes_nothing

```
def test_second_seed_run_changes_nothing(db: sqlite3.Connection) -> None:
    """Rerunning the whole seed must leave the database state byte
    identical, every value of every row, not merely the row counts."""
    run_seed(db)
    before = snapshot(db)

    run_seed(db)

    assert snapshot(db) == before
```

Why it exists. The seed must be idempotent, a property meaning that running an operation twice leaves the same result as running it once. Developers rerun the seed casually, after a pull, after a schema change, out of habit, and a seed that mutates data on the second run would corrupt the very sample environment people debug against. This test is the guarantee that rerunning is always safe.

How it works. The body is three beats. Seed once and photograph the entire database with `snapshot`. Seed again. Photograph again and demand the two pictures are equal. Because `snapshot` returns plain dicts of row lists, and `sqlite3.Row` objects compare by their values, the single `==` compares every value of every column of every row of all five tables at once, and pytest's assertion rewriting will display the exact differing rows if the comparison fails.

The docstring insists on every value, not merely the row counts, and that distinction is the lesson of this test. A count based check, five clients before and five after, was the obvious first version of this assertion, and it is not enough, because a whole family of idempotency bugs preserves counts while changing data. A seed that deletes everything and reinserts it keeps every count identical while handing out fresh ids, silently breaking anything that stored the old ones. An upsert gone wrong, an upsert being an insert that updates the row instead when it already exists, could overwrite a column on every rerun without adding a single row. The sharpest example lives in this very seed, `mark_sample_statuses` stamps a completion timestamp on the done task, and a version of it that stamped the current time on every run would change `COMPLETED_AT` at each rerun while every count in the database stayed exactly the same. The full snapshot comparison catches all three, the count check catches none of them.

What breaks it. Any seed step missing its existence check, inserting duplicates, rewriting timestamps, or recreating rows under new ids, shows up here as two unequal snapshots with the offending rows printed in the diff.

test_sample_statuses_present

```
def test_sample_statuses_present(db: sqlite3.Connection) -> None:
    """One done task with a trace, one not applicable, rest pending."""
    run_seed(db)

    done = db.execute(
        "SELECT COMPLETED_AT, COMPLETION_METHOD FROM TASKS WHERE STATUS = 'done'"
    ).fetchall()
    assert len(done) == 1
    assert done[0][0] is not None
    assert done[0][1] == "manual"

    not_applicable = db.execute(
        "SELECT COUNT(*) FROM TASKS WHERE STATUS = 'not_applicable'"
    ).fetchone()[0]
    assert not_applicable == 1

    pending = db.execute(
        "SELECT COUNT(*) FROM TASKS WHERE STATUS = 'pending'"
    ).fetchone()[0]
    assert pending == 9
```

Why it exists. A sample database where every task is pending demonstrates nothing, so `mark_sample_statuses` deliberately makes the data interesting, one task done, one marked not applicable, the rest untouched. This test pins that arrangement, and it also pins a rule of the data model, a done task must carry the evidence of its completion.

How it works. After seeding, the first query selects two columns from every task whose status is `'done'`, and `fetchall()` returns them all so the test can count them. There must be exactly one such task. Then the test drills into that row, `done[0][0]` is its `COMPLETED_AT`, which must not be `None`, meaning a completion timestamp was recorded, and `done[0][1]` is its `COMPLETION_METHOD`, which must be the string `"manual"`, recording how the task was completed. That pair of assertions encodes the invariant that done is never a bare flag, it always comes with a when and a how. The second query counts tasks with status `'not_applicable'`, exactly one. The third counts `'pending'`, exactly nine, and the arithmetic closes the loop, one done plus one not applicable plus nine pending equals the eleven tasks the earlier count test established, so no task carries an unexpected status.

What breaks it. A `mark_sample_statuses` that set the status but forgot the timestamp or the method would fail the trace assertions, catching a completion path that loses its audit trail. Marking the wrong number of tasks, or a status string drifting out of sync with the schema's allowed values, breaks the counts. And if task generation ever changes how many tasks exist, the pending count of nine fails, forcing this test and the sample data to be reconciled on purpose rather than drifting apart.

That closes the storage layer's test suite. Between the two files, every promise the rest of TaxDesk depends on is written down as an executable check, the schema arrives complete, migrations apply once and fail clean, the constraints hold, foreign keys are enforced, and the seed can be run any number of times without changing a byte. When a later chapter mutates this database freely, this is the net underneath it.

5.9 The Application Tests

TaxDesk is a small FastAPI application that stores everything in a single SQLite file and, on first run, walks the user through onboarding, where a root folder on disk is scanned and its subfolders become client records after an explicit confirmation. This chapter reads the two test files that pin that behavior down. `tests/test_app.py` covers the application skeleton, the `/health` route, database startup, and the rollback guarantee of the request scoped connection. `tests/test_onboarding.py` covers the whole onboarding flow, from an empty database to saved clients, including its trust boundary against crafted folder names. Together they are the executable contract for everything the app does today, and every later chapter's code must keep these green.

The module docstring and imports of `test_app.py`

Why it exists. Every test module opens by declaring its subject and pulling in exactly the pieces it exercises. This one needs pytest, FastAPI's test machinery, and three things from the application itself, the dependency it will override, the factory that builds the app, and the low level database connector it uses to inspect results from outside the app.

```
"""Tests for the application skeleton, the health route and startup."""

from collections.abc import Iterator
from pathlib import Path
from sqlite3 import Connection

import pytest
from fastapi import Depends
from fastapi.testclient import TestClient

from app.deps import get_db
from app.main import create_app
from database.migrate import connect
```

How it works. The triple quoted string on the first line is a module docstring, a description Python attaches to the module itself. `Iterator` from `collections.abc` is a type used to annotate functions that yield values one at a time rather than returning once. `Path` from `pathlib` is Python's object for filesystem paths, and `Connection` is the `sqlite3` type representing an open database handle, both imported so the annotations below can name them. `pytest` is the test runner, `Depends` is FastAPI's marker for declaring that a route parameter should be supplied by a dependency function, and `TestClient` is FastAPI's fake HTTP client, which sends requests to the app entirely in memory, no network socket

involved, while still running the real routing, validation, and error handling. The last three imports come from the project under test. `get_db` is the dependency that hands each request a database connection, `create_app` builds a fresh application instance pointed at a given database file, and `connect` opens a raw SQLite connection so a test can look at the database directly, bypassing the app.

The client fixture

Why it exists. Almost every test here needs the same setup, a fresh app over a fresh database with migrations already applied. A fixture is pytest's named, reusable setup function, and any test that lists the fixture's name as a parameter receives its value automatically. Centralizing the setup here means each test body contains only the behavior it is actually about.

```
@pytest.fixture
def client(tmp_path: Path) -> Iterator[TestClient]:
    """Give a test a running app over a fresh temp database.

    In: pytest's tmp_path.
    Out: a TestClient whose startup has run, so migrations applied.
    """
    application = create_app(tmp_path / "test.db")
    with TestClient(application) as test_client:
        yield test_client
```

How it works. `@pytest.fixture` is a decorator, a function that wraps another function to change how it is used, and here it registers `client` as a fixture. The fixture itself asks for `tmp_path`, one of pytest's built in fixtures, which supplies a brand new temporary directory unique to each test, so no test can ever see another test's files or database. `create_app(tmp_path / "test.db")` builds the application configured to store its SQLite file inside that temporary directory, the `/` operator on `Path` objects joining path segments. The `with` statement uses `TestClient` as a context manager, an object that runs setup code on entry and cleanup code on exit, and for `TestClient` entry means running the app's startup events, which is exactly when migrations apply. The `yield` makes this fixture a generator, a function that pauses at `yield` and hands the value out. Pytest gives `test_client` to the test, runs the test, then resumes the fixture past the `yield`, which exits the `with` block and shuts the app down cleanly. The return annotation `Iterator[TestClient]` states that shape, a function that yields a `TestClient` rather than returning one.

test_health_returns_200

Why it exists. The most basic promise of the skeleton is that the health route exists and answers successfully. If this fails, nothing else in the system is worth testing.

```
def test_health_returns_200(client: TestClient) -> None:
    """The route must answer with HTTP 200."""
    assert client.get("/health").status_code == 200
```

How it works. The test asks for the `client` fixture by naming it as a parameter, so it starts with a running app. `client.get("/health")` performs an in memory GET request and returns a response object, and the test asserts its `status_code` is 200, the HTTP code meaning success. A broken route registration, a typo in the path, or a crash during the request would all surface here as a non 200 code.

test_health_body_is_exactly_status_ok

Why it exists. Checking the status code is not enough, the body is part of the contract too. This test uses exact equality so no field can quietly join the response later without a test noticing.

```
def test_health_body_is_exactly_status_ok(client: TestClient) -> None:
    """Key for key equality, so nothing can quietly join the response."""
    assert client.get("/health").json() == {"status": "ok"}
```

How it works. `.json()` parses the response body as JSON into a Python dictionary. Comparing it with `==` against `{"status": "ok"}` demands the whole dictionary match, every key and every value. A test that only asserted `body["status"] == "ok"` would keep passing if someone added a debug field carrying internal details, this one would fail immediately.

test_health_uses_the_injected_database

Why it exists. The health route is supposed to reach SQLite through the `get_db` dependency, not by opening its own connection on the side. You cannot see that from the response body, so the test proves it indirectly, by swapping the dependency for a spy and observing that the spy gets consumed.

```
def test_health_uses_the_injected_database(tmp_path: Path) -> None:
    """The route must reach SQLite through the dependency, proven by
    overriding it and observing the override being consumed."""
    application = create_app(tmp_path / "main.db")
    used = False

    def override() -> Iterator[Connection]:
        nonlocal used
        used = True
        conn = connect(tmp_path / "other.db")
        try:
            yield conn
        finally:
            conn.close()

    application.dependency_overrides[get_db] = override

    with TestClient(application) as test_client:
        response = test_client.get("/health")

    assert response.status_code == 200
    assert used is True
```

How it works. This test builds its own app instead of using the `client` fixture, because it needs the application object in hand to install the override on it. It sets a flag `used = False`, then defines `override`, an inner function that will stand in for `get_db`. Inside it, `nonlocal used` tells Python that assignments to `used` should change the variable in the enclosing test function rather than create a new local one, which is how the inner function reports back. The override flips the flag, opens a connection to a completely different database file, `other.db`, and yields it inside a `try/finally` so the connection is closed no matter what the route does with it, the same generator shape FastAPI expects from a real dependency. The key line is `application.dependency_overrides[get_db] = override`. Every FastAPI app carries a `dependency_overrides` dictionary mapping an original dependency function to a replacement, and FastAPI

consults it at request time, each time a route's dependencies are resolved, so installing the entry before the client starts is tidy ordering rather than a requirement. While the entry is present, any route that declared `Depends(get_db)` receives the replacement's value instead, and the dictionary exists precisely for tests like this one. The test then starts the client, hits `/health`, and asserts two things, the route still answered 200 even though its database was silently swapped, and `used` is now `True`, which can only happen if FastAPI actually called the override, which in turn can only happen if the route declares the dependency. A route that opened its own connection directly would leave `used` as `False` and fail the second assertion.

test_startup_initializes_a_fresh_database

Why it exists. The app promises that merely starting it against a fresh database file leaves that database migrated. This test enters the app context, does nothing else, and then inspects the file from outside to verify the promise.

```
def test_startup_initializes_a_fresh_database(tmp_path: Path) -> None:
    """Entering the app context must leave the database migrated."""
    db_path = tmp_path / "fresh.db"
    application = create_app(db_path)

    with TestClient(application):
        pass

    conn = connect(db_path)
    recorded = [
        row[0]
        for row in conn.execute("SELECT filename FROM schema_applied ORDER BY filename")
    ]
    conn.close()

    assert recorded == ["001_schema.sql", "002_settings.sql"]
```

How it works. The test builds the app over a database file that does not exist yet. `with TestClient(application):` `pass` enters and immediately exits the client's context, which runs startup and shutdown and nothing in between, no request is ever sent. That is the whole point, startup alone must be enough. Afterward the test opens the database file directly with the project's `connect` helper and reads the `schema_applied` table, the migration ledger where the migration runner records each SQL file it has applied. The square bracket expression is a list comprehension, a compact way to build a list by looping, here collecting `row[0]`, the filename column, from every row, ordered by filename so the comparison is deterministic. The final assertion demands exactly the two known migration files, in order. A broken startup hook, a migration that failed silently, or a missing recording step would each make this list wrong.

test_get_db_rolls_back_writes_when_the_route_raises

Why it exists. The `get_db` dependency's docstring promises that a request which fails midway leaves nothing committed. That is a transactional guarantee, and the only honest way to test it is to write something inside a route, blow up on purpose, and then check from outside that the write vanished.

```
def test_get_db_rolls_back_writes_when_the_route_raises(tmp_path: Path) -> None:
    """A route that writes and then raises must leave nothing
    committed, the actual guarantee get_db's docstring promises."""
    application = create_app(tmp_path / "test.db")
```

```

@application.get("/boom")
def boom(conn: Connection = Depends(get_db)) -> None:
    conn.execute("INSERT INTO SETTINGS (ID, ROOT_FOLDER) VALUES (1, 'x')")
    raise RuntimeError("simulated failure after a write")

with TestClient(application, raise_server_exceptions=False) as test_client:
    response = test_client.get("/boom")

assert response.status_code == 500

conn = connect(tmp_path / "test.db")
row = conn.execute("SELECT ROOT_FOLDER FROM SETTINGS WHERE ID = 1").fetchone()
conn.close()
assert row is None

```

How it works. The app has no real route that writes and then crashes, so the test manufactures one.

`@application.get("/boom")` is the same decorator the application uses to register its own routes, and here it bolts a synthetic route named `/boom` onto this test's private app instance, which is safe because the instance lives and dies inside this test. The route declares `conn: Connection = Depends(get_db)`, meaning FastAPI will call the real `get_db` dependency and inject the connection it yields. The body inserts a row into `SETTINGS` and then raises a `RuntimeError`, simulating a bug striking after a write has already happened. The client is built with `raise_server_exceptions=False`, and that flag matters. By default `TestClient` re raises any unhandled server exception straight into the test, which would abort the test with the `RuntimeError` before any assertion runs. Setting it to `False` makes the client behave like a real browser instead, the exception stays on the server side and the client receives the resulting 500 response. The test asserts that 500, then opens the database file independently and asks for the row the route inserted. `fetchone()` returns the first matching row or `None` when there is none, and the test demands `None`. If `get_db` committed as soon as the insert ran, or failed to roll back when the route raised, the row would survive and this assertion would catch it.

test_unknown_route_returns_plain_404

Why it exists. A request for a path that does not exist should get FastAPI's stock 404 and nothing more. A custom error page or handler that leaked internals would break this contract.

```

def test_unknown_route_returns_plain_404(client: TestClient) -> None:
    """A miss must return FastAPI's generic 404 with no internals."""
    response = client.get("/does-not-exist")

    assert response.status_code == 404
    assert response.json() == {"detail": "Not Found"}

```

How it works. The test requests a path no route claims and asserts two things, the 404 status code, and the exact body `{"detail": "Not Found"}`, which is the generic JSON FastAPI produces on its own for a routing miss. The exact equality again protects against anything extra creeping into the error body.

test_health_exposes_no_internal_information

Why it exists. A health endpoint is often reachable without authentication, so its body must not advertise anything about the machine it runs on. This test scans the response for a list of telltale substrings and also locks the response down to a single key.

```
def test_health_exposes_no_internal_information(client: TestClient) -> None:
    """The response must carry no paths, versions, or database hints."""
    response = client.get("/health")
    body = response.text.lower()

    for leak in ("taxdesk", ".db", "sqlite", "path", "version", "traceback"):
        assert leak not in body
    assert set(response.json().keys()) == {"status"}
```

How it works. The test fetches `/health` and lowercases the raw text body so the substring checks are not fooled by capitalization. It then loops over a tuple of forbidden fragments, the project name, the database file extension, the database engine name, and words that typically ride along with leaked paths, versions, or stack traces, asserting each is absent. The last line converts the JSON body's keys into a set, an unordered collection of unique values written with braces, and compares it against the set containing only `"status"`. That is a second, structural way of saying the body carries exactly one field. A future change that added a version field or an absolute path to the health response would fail one or both checks.

The module docstring and imports of `test_onboarding.py`

Why it exists. The onboarding tests exercise a flow that touches the filesystem and the database at once, so the docstring states the safety property up front, everything runs on temp folders and temp databases. The import list is shorter than the app test file's because these tests never override dependencies, they drive the app purely through HTTP and then verify through the database file.

```
"""Tests for onboarding, discovery, confirmation, and its trust
boundary. Everything runs on temp folders and temp databases."""

from collections.abc import Iterator
from pathlib import Path

import pytest
from fastapi.testclient import TestClient

from app.main import create_app
from database.migrate import connect
```

How it works. The docstring names the three phases under test, onboarding, discovery, and confirmation, plus the trust boundary, meaning the line between folder names the app found itself and names a request merely claims. The imports mirror the earlier file, `Iterator` and `Path` for annotations, `pytest` for fixtures, `TestClient` for in memory HTTP, `create_app` to build the app over a temp database, and `connect` to inspect that database from outside the app.

The client fixture in the onboarding file

Why it exists. This is the same setup pattern as in `test_app.py`, repeated locally so this file stands alone. Each test gets a live app over its own fresh database, with migrations already applied by startup.

```
@pytest.fixture
def client(tmp_path: Path) -> Iterator[TestClient]:
    """Give a test a running app over a fresh temp database.
```

```

In: pytest's tmp_path.
Out: a TestClient with startup done, migrations applied.
"""

application = create_app(tmp_path / "test.db")
with TestClient(application) as test_client:
    yield test_client

```

How it works. The fixture builds the app pointed at `test.db` inside the test's private temporary directory, enters the `TestClient` context manager so startup runs and migrations apply, and yields the client to the test. When the test finishes, pytest resumes the generator, the `with` block exits, and the app shuts down. Because both this fixture and the helpers below derive paths from the same `tmp_path`, a test that receives both `client` and `tmp_path` is always talking about one and the same database file.

The root fixture

Why it exists. Discovery is about reading a real folder on disk, so the tests need a realistic one. This fixture builds a root that contains exactly the mix of things discovery must handle, folders that should appear, and clutter that must not.

```

@pytest.fixture
def root(tmp_path: Path) -> Path:
    """Give a test a realistic root folder on disk.

    In: pytest's tmp_path.
    Out: a root containing three client folders, one hidden folder,
    and one plain file.
    """
    folder = tmp_path / "clients"
    folder.mkdir()
    (folder / "Aster Traders").mkdir()
    (folder / "Bhima Textiles").mkdir()
    (folder / "Cauvery Mills").mkdir()
    (folder / ".hidden").mkdir()
    (folder / "loose_file.txt").write_text("not a folder")
    return folder

```

How it works. The fixture creates a directory named `clients` inside the test's temporary directory, then populates it with five entries. Three are ordinary subfolders with client sounding names, Aster Traders, Bhima Textiles, and Cauvery Mills, and these are the only entries discovery should ever surface. One is `.hidden`, a folder whose name starts with a dot, the convention for hidden files on Unix systems, which discovery must skip. The last is `loose_file.txt`, a plain file rather than a folder, written with `write_text`, which discovery must also skip. The final layout on disk looks like this.

```

tmp_path/
  clients/
    Aster Traders/
    Bhima Textiles/
    Cauvery Mills/
    .hidden/
    loose_file.txt

```

The fixture returns the `clients` path itself, so a test can post it as the root folder and also build expected paths under it.

The saved_root helper

Why it exists. Several tests must know what root, if any, the app has actually persisted, and they must know it independently of what any page claims. This helper opens the database file directly and reads the single settings row.

```
def saved_root(client: TestClient, tmp_path: Path) -> str | None:
    """Read ROOT_FOLDER straight from the test database.

    In: the test client fixture's paired tmp_path.
    Out: the stored value, or None when no row exists.
    """
    conn = connect(tmp_path / "test.db")
    row = conn.execute("SELECT ROOT_FOLDER FROM SETTINGS WHERE ID = 1").fetchone()
    conn.close()
    return row["ROOT_FOLDER"] if row else None
```

How it works. This is a plain helper function, not a fixture, so tests call it explicitly. Note that the body never touches the `client` parameter, it only opens `tmp_path / "test.db"`, the same file the `client` fixture built its app on. Passing the client anyway makes the call sites read as a pair, this client and its database. The return annotation `str | None` is a union type, meaning the function returns either a string or `None`. It runs one query for the `ROOT_FOLDER` column of the settings row with `ID = 1`, and `fetchone()` yields that row or `None`. The last line is a conditional expression, when a row exists it returns `row["ROOT_FOLDER"]`, indexing the row by column name, which the project's `connect` helper makes possible by configuring name based row access, and otherwise it returns `None`, the state before any root was ever saved.

The `client_rows` helper

Why it exists. The other fact tests keep needing is which clients exist in the database, again read straight from the file so no page can lie about it. Returning name and path pairs in a fixed order makes whole list equality assertions possible.

```
def client_rows(tmp_path: Path) -> list[tuple[str, str]]:
    """Read all clients straight from the test database.

    In: the test's tmp_path.
    Out: (name, folder_path) pairs ordered by name.
    """
    conn = connect(tmp_path / "test.db")
    rows = conn.execute(
        "SELECT NAME, FOLDER_PATH FROM CLIENTS ORDER BY NAME"
    ).fetchall()
    conn.close()
    return [(row["NAME"], row["FOLDER_PATH"]) for row in rows]
```

How it works. The helper opens the test database, selects every row of the `CLIENTS` table ordered by name so results are deterministic, and closes the connection. `fetchall()` returns all rows at once. The final line is a list comprehension building a list of two element tuples, each pairing a client's name with its stored folder path. The annotation `list[tuple[str, str]]` states exactly that shape. An empty database yields an empty list, which several tests rely on.

`test_onboarding_without_root_asks_for_one`

Why it exists. On a fresh install nothing is configured, and the onboarding page must handle that state by asking for a root, not by crashing or by showing an empty candidate list. This pins down the very first screen a user ever sees.

```
def test_onboarding_without_root_asks_for_one(client: TestClient) -> None:
    """No configured root shows the form and no candidate list."""
    response = client.get("/onboarding")

    assert response.status_code == 200
    assert 'name="root_folder"' in response.text
    assert "Select the folders" not in response.text
```

How it works. With a fresh database and no root saved, the test fetches `/onboarding` and asserts the page renders successfully. It then checks the HTML text for `name="root_folder"`, the attribute of the form input where the user types a path, proving the ask for a root form is present. Finally it asserts the phrase "Select the folders" is absent, which is the heading of the discovery panel, proving the page does not show a folder selection step before there is anything to select. The real bug this catches is a page that assumes a root always exists and either raises when the settings row is missing or renders an empty, confusing selection list.

test_valid_root_is_saved_and_redirects_303

Why it exists. Submitting a real directory is the happy path of step one. Two things must happen, the path is persisted, and the response is a redirect so the browser lands back on the page in its new state.

```
def test_valid_root_is_saved_and_redirects_303(
    client: TestClient, root: Path, tmp_path: Path
) -> None:
    """A real directory is persisted and the POST answers 303."""
    response = client.post(
        "/onboarding/root", data={"root_folder": str(root)}, follow_redirects=False
    )

    assert response.status_code == 303
    assert saved_root(client, tmp_path) == str(root.resolve())
```

How it works. The test posts the fixture root's path as form data, `data={...}` sending it the way an HTML form does. `follow_redirects=False` tells the client not to chase the redirect automatically, otherwise the test would only see the final page and a 200, never the redirect itself. It then asserts the status is 303, the See Other code, which is the correct answer to a form POST because it tells the browser to fetch the next page with GET, so refreshing never resubmits the form. The second assertion calls `saved_root` and compares against `str(root.resolve())`, where `resolve()` turns a path into its absolute, symlink free canonical form. That comparison would fail if the app stored the raw text the user typed instead of a canonical absolute path, or if it redirected without saving at all.

test_invalid_root_rejected_and_settings_untouched

Why it exists. Validation is only half the job, the other half is that a failed submission must change nothing. This test posts a path that does not exist and verifies both the error response and the untouched database.

```
def test_invalid_root_rejected_and_settings_untouched(
    client: TestClient, tmp_path: Path
) -> None:
    """A nonexistent path renders an error and saves nothing."""
    response = client.post(
        "/onboarding/root",
```

```

        data={"root_folder": str(tmp_path / "nope")},
        follow_redirects=False,
    )

    assert response.status_code == 400
    assert "Nothing was saved" in response.text
    assert saved_root(client, tmp_path) is None

```

How it works. The posted path, `tmp_path / "nope"`, was never created, so it fails whatever directory check the route performs. The test asserts a 400 Bad Request rather than a redirect, asserts the page tells the user plainly that "Nothing was saved", and then goes around the app entirely with `saved_root` to confirm the settings row still does not exist. The bug this pins down is a route that writes the value first and validates second, which would leave a broken root configured after an error the user believed was harmless.

test_invalid_root_keeps_showing_the_still_valid_configuration

Why it exists. This is the subtlest test on the page. Once a working root is configured, a user might try to change it and typo the new path. Rejecting the typo is easy. The hard part is that the error page must not make the working setup look gone, and must not lose what the user typed. This test pins both at once.

```

def test_invalid_root_keeps_showing_the_still_valid_configuration(
    client: TestClient, root: Path, tmp_path: Path
) -> None:
    """A rejected second path must echo what was typed and must not
    hide the working root's own discovery panel, nothing was lost."""
    client.post("/onboarding/root", data={"root_folder": str(root)})

    bad_path = str(tmp_path / "nope")
    response = client.post(
        "/onboarding/root", data={"root_folder": bad_path}, follow_redirects=False
    )

    assert response.status_code == 400
    assert f'value="{bad_path}"' in response.text
    assert "Aster Traders" in response.text
    assert saved_root(client, tmp_path) == str(root.resolve())

```

How it works. The test first configures the valid fixture root, establishing a working state. It then posts a nonexistent path and inspects the 400 response with three assertions. The first uses an f-string, a string literal prefixed with `f` where expressions in braces are substituted in, to build `value="..."` containing the bad path, and asserts that exact attribute appears in the HTML, meaning the input field echoes back what the user typed so they can correct one character instead of retyping. The second asserts "Aster Traders" is still on the page, which can only be true if the discovery panel for the still saved, still valid root rendered alongside the error. The third confirms through the database that the saved root is still the old one, resolved and intact. Three distinct real bugs would each fail this test, a form that clears itself on error, an error page built from only the rejected input so the working root's panel disappears and the user thinks their setup was wiped, and worst of all a route that overwrote the good root with the bad one before validating.

test_discovery_lists_immediate_folders_only

Why it exists. Discovery has a precise scope, the immediate subfolders of the root and nothing else. Files, hidden folders, and anything nested deeper must stay out. This test plants one of each and checks all four rules on a single rendered page.

```
def test_discovery_lists_immediate_folders_only(
    client: TestClient, root: Path
) -> None:
    """Subfolders appear, files and hidden folders and nested
    directories do not."""
    (root / "Aster Traders" / "Nested").mkdir()
    client.post("/onboarding/root", data={"root_folder": str(root)})

    page = client.get("/onboarding").text

    for name in ("Aster Traders", "Bhima Textiles", "Cauvery Mills"):
        assert name in page
    assert "loose_file.txt" not in page
    assert ".hidden" not in page
    assert "Nested" not in page
```

How it works. The test first creates `Nested` inside `Aster Traders`, adding a second level the fixture does not have, then saves the root and fetches the onboarding page. The loop asserts each of the three real client folders appears in the HTML. The three negative assertions then check the exclusions, `loose_file.txt` because it is a file, `.hidden` because its name starts with a dot, and `Nested` because it lives one level too deep. The corresponding bugs are a discovery routine that lists every directory recursively, one that forgets to filter out files, or one that surfaces hidden folders the user never meant as clients.

test_new_candidates_are_checked_by_default

Why it exists. The onboarding flow is designed so the common case, add everything found, is one click. That means every newly discovered folder starts ticked, and the confirm button states how many clients it is about to add so the user confirms a number, not a vague action.

```
def test_new_candidates_are_checked_by_default(
    client: TestClient, root: Path
) -> None:
    """Every new folder's checkbox carries the checked attribute and
    the button says how many clients it will add."""
    client.post("/onboarding/root", data={"root_folder": str(root)})

    page = client.get("/onboarding").text

    assert page.count("checked") == 3
    assert "Add 3 clients" in page
```

How it works. After saving the root, the test fetches the page and uses `str.count` to count occurrences of the substring "checked" in the HTML, asserting exactly three, one per discovered folder's checkbox. Counting, rather than merely checking presence, catches both a page where nothing is preselected and a page where the attribute appears in the wrong number of places. The second assertion looks for the literal button text "Add 3 clients", tying the visible count to the actual number of candidates. A template that dropped the `checked` attribute, or a count computed from the wrong list, would fail here.

test_confirm_creates_exactly_the_selected_clients

Why it exists. Confirmation is the moment folders become database rows, and it must honor the user's selection exactly, no more and no less, storing absolute paths so the records stay meaningful regardless of where the app is later run from.

```
def test_confirm_creates_exactly_the_selected_clients(
    client: TestClient, root: Path, tmp_path: Path
) -> None:
    """Two ticked folders become two clients with absolute paths, the
    unticked one does not."""
    client.post("/onboarding/root", data={"root_folder": str(root)})

    response = client.post(
        "/onboarding/confirm",
        data={"folders": ["Aster Traders", "Cauvery Mills"]},
        follow_redirects=False,
    )

    assert response.status_code == 303
    rows = client_rows(tmp_path)
    assert rows == [
        ("Aster Traders", str((root / "Aster Traders").resolve())),
        ("Cauvery Mills", str((root / "Cauvery Mills").resolve())),
    ]
    assert all(Path(path).is_absolute() for _, path in rows)
```

How it works. After saving the root, the test posts to the confirm endpoint with `data={"folders": [...]}`, and because the value is a list, the client sends the `folders` field twice, exactly how an HTML form submits multiple ticked checkboxes sharing one name. Bhima Textiles is deliberately left out, playing the unticked folder. The test asserts the 303 redirect, then reads the database through `client_rows` and compares the whole list against exactly two expected tuples, each pairing a name with the resolved absolute path of its folder under the root. Whole list equality means an extra row for Bhima Textiles would fail just as loudly as a missing row. The last line asserts every stored path is absolute, using `all()` over a generator expression, a lazy comprehension that yields values one at a time, with tuple unpacking `for _, path in rows` discarding the name via the throwaway underscore. A confirm handler that added every candidate regardless of selection, or stored paths relative to the root, would fail here.

test_existing_clients_marked_and_not_duplicated

Why it exists. Onboarding can be revisited, so it must be safe to run twice. Folders already turned into clients should be labeled as such, the button count should reflect only what is genuinely new, and confirming the same folder again must not create a duplicate row.

```
def test_existing_clients_marked_and_not_duplicated(
    client: TestClient, root: Path, tmp_path: Path
) -> None:
    """After a confirmation, rerunning shows already added and a second
    confirm changes nothing."""
    client.post("/onboarding/root", data={"root_folder": str(root)})
    client.post("/onboarding/confirm", data={"folders": ["Aster Traders"]})

    page = client.get("/onboarding").text
    assert "(already added)" in page
    assert "Add 2 clients" in page

    before = client_rows(tmp_path)
    client.post(
```

```

        "/onboarding/confirm",
        data={"folders": ["Aster Traders"]},
    )
    assert client_rows(tmp_path) == before

```

How it works. The test saves the root and confirms a single folder, Aster Traders. Reloading the page, it asserts the marker text "(already added)" appears, so the user can see which folders are done, and that the button now reads "Add 2 clients", counting only the two remaining candidates. Then it snapshots the client rows, posts the very same confirmation a second time, and asserts the rows are unchanged, comparing the whole list before and after. The bugs this pins down are a page that cannot tell existing clients from new candidates, a count that includes folders already added, and a confirm handler whose insert is not idempotent, meaning safe to repeat, so a double click or a page refresh would mint duplicate clients.

test_traversal_names_cannot_create_clients

Why it exists. The confirm endpoint receives folder names from the request body, and a request can claim any name it likes, including names that escape the root. This is the trust boundary named in the module docstring, and this test attacks it three ways at once.

```

def test_traversal_names_cannot_create_clients(
    client: TestClient, root: Path, tmp_path: Path
) -> None:
    """Crafted names die silently, only real subfolders count."""
    outside = tmp_path / "outside"
    outside.mkdir()
    client.post("/onboarding/root", data={"root_folder": str(root)})

    client.post(
        "/onboarding/confirm",
        data={"folders": ["../outside", str(outside), "no_such_folder"]},
    )

    assert client_rows(tmp_path) == []

```

How it works. The test first creates a real directory named `outside` that sits next to the root, not inside it, so the attacks below point at something that genuinely exists. After saving the root, it posts three crafted names to the confirm endpoint. The first, `"../outside"`, is a path traversal, `..` means the parent directory, so naively joining it onto the root would land on the outside folder. The second is the absolute path of that same folder, testing whether an absolute name can override the root entirely, which is how `Path` joining behaves if the handler is careless. The third, `"no_such_folder"`, is inside the root in spirit but does not exist. The single assertion is total, `client_rows` must return an empty list, meaning not one of the three produced a client. The docstring's phrase, crafted names die silently, also matters, the endpoint filters them out rather than erroring, since only real immediate subfolders count. The real bug here would be a confirm handler that joins each submitted name onto the root and inserts without verifying the result is an existing immediate subfolder, which would let any request register arbitrary directories on the machine as clients, exactly the kind of hole that turns a local convenience app into a browsing tool for the whole disk.

test_empty_root_is_handled_cleanly

Why it exists. A brand new user might point the app at a folder that has nothing in it yet. That is a valid state, not an error, and the page should say so in words instead of showing a blank or broken panel.

```
def test_empty_root_is_handled_cleanly(
    client: TestClient, tmp_path: Path
) -> None:
    """An empty root saves fine and the page says so plainly."""
    empty = tmp_path / "empty"
    empty.mkdir()

    client.post("/onboarding/root", data={"root_folder": str(empty)})
    page = client.get("/onboarding").text

    assert "No folders found inside the root" in page
```

How it works. The test creates a directory with nothing in it, saves it as the root, which must succeed because the directory is real, and then loads the page. The single assertion looks for the explicit sentence "No folders found inside the root". A template that only handles the nonempty case, or discovery code that trips over an empty listing, would fail this test by rendering something else or by erroring.

test_unreadable_root_does_not_crash_the_page

Why it exists. A folder can pass the existence check when saved and later become unlistable, permissions can change, a removable drive can hiccup. Reading the directory is an operation that can fail at page render time, and the page must survive that instead of collapsing into a 500.

```
def test_unreadable_root_does_not_crash_the_page(
    client: TestClient, tmp_path: Path
) -> None:
    """A folder that stats fine but cannot be listed, permissions, a
    removable drive hiccup, must render, not raise a 500."""
    locked = tmp_path / "locked"
    locked.mkdir()
    client.post("/onboarding/root", data={"root_folder": str(locked)})

    locked.chmod(0o000)
    try:
        response = client.get("/onboarding")
    finally:
        locked.chmod(0o755)

    assert response.status_code == 200
```

How it works. The test creates a directory, saves it as the root while it is still readable so validation passes, and only then removes every permission with `chmod(0o000)`, the `0o` prefix marking an octal literal, zero rights for owner, group, and everyone. The directory still exists and still stats fine, but any attempt to list its contents now raises a permission error. The page fetch is wrapped in `try/finally` so that `chmod(0o755)`, restoring normal permissions, runs even if the request explodes, without that restore pytest could not delete the temporary directory during cleanup. The assertion is simply a 200. The real bug is discovery code that calls the directory listing with no error handling, so a permission error propagates up as an unhandled exception and the user staring at a 500 has no idea their configured root became unreadable.

test_changing_root_keeps_existing_client_paths

Why it exists. Clients store absolute paths, so they are facts about the disk, not about the current root setting. Changing the root must never rewrite or invalidate records created under a previous root. This test locks in that separation.

```
def test_changing_root_keeps_existing_client_paths(
    client: TestClient, root: Path, tmp_path: Path
) -> None:
    """A new root never rewrites paths recorded under the old one."""
    client.post("/onboarding/root", data={"root_folder": str(root)})
    client.post("/onboarding/confirm", data={"folders": ["Aster Traders"]})
    before = client_rows(tmp_path)

    other = tmp_path / "other_root"
    other.mkdir()
    (other / "Deccan Services").mkdir()
    client.post("/onboarding/root", data={"root_folder": str(other)})

    assert client_rows(tmp_path) == before
    assert saved_root(client, tmp_path) == str(other.resolve())
```

How it works. The test builds a full working state, root saved, one client confirmed, and snapshots the client rows. It then constructs a second, unrelated root containing its own folder, Deccan Services, and saves that as the new root. The two assertions check both sides of the contract, the client rows are byte for byte what they were before the switch, and the settings row now holds the new root's resolved path. A handler that treated a root change as a reason to remap or delete existing clients, or one that failed to actually update the setting, would each fail one of these two lines.

test_home_redirects_to_onboarding

Why it exists. At this stage of the project onboarding is the only page there is, so the root URL should send the user straight to it rather than answering with a 404 or an empty page.

```
def test_home_redirects_to_onboarding(client: TestClient) -> None:
    """The root URL points at the only page that exists."""
    response = client.get("/", follow_redirects=False)

    assert response.status_code == 302
    assert response.headers["location"] == "/onboarding"
```

How it works. The test requests `/` with `follow_redirects=False` so it can inspect the redirect itself. It asserts a 302 Found, the ordinary temporary redirect for a GET, temporary being the right choice because a future version of the app will surely put a real home page here. It then reads the `location` response header, the field a redirect uses to name its destination, and asserts it is exactly `/onboarding`. A missing route on `/`, or a redirect pointed anywhere else, fails immediately.

Part 6, What Actually Happens At Runtime

Starting the application

```
venv/bin/uvicorn app.main:app
```

1. uvicorn starts as an operating system process
2. it imports app.main, which imports the route modules, and the decorators register every route while importing
3. lifespan runs, opens the database, applies pending migrations, ensures the four services exist, closes that connection
4. uvicorn binds 127.0.0.1 port 8000 and waits

Binding to 127.0.0.1 means the operating system only accepts connections from this same machine. That is the security boundary of the product, and it is why no host flag is ever passed.

A read, GET /onboarding

1. browser sends GET /onboarding
2. uvicorn hands the request to FastAPI
3. FastAPI matches the path to onboarding_page
4. the signature asks for a connection, so get_db opens one
5. queries.get_root_folder reads the settings row
6. discovery_context lists the real subfolders and compares them against existing client paths
7. the route hands values to onboarding.html, Jinja2 fills and escapes them
8. the HTML travels back, get_db commits nothing of substance and closes the connection

A write, POST /onboarding/confirm

1. browser sends the ticked folder names
2. FastAPI calls confirm_clients, get_db opens a connection
3. the route reads the saved root, then rescans the disk to build the set of folder names that really exist
4. for each submitted name, if it is in that set, a client is created, otherwise it is silently ignored
5. the route returns 303 pointing at /onboarding
6. get_db commits, making every created client permanent together
7. the browser follows the redirect with a GET, so a refresh cannot resubmit

If step 4 had raised instead, `get_db` would have rolled back and no client would have been created at all, not even the ones already inserted in the loop.

Part 7, Decisions And Rejected Alternatives

Every entry here records a real choice, what it beat, and why. The running record with more detail is `docs/engineering-log.md`.

Services as a table, not fixed columns

Rejected. Four boolean columns on `CLIENTS`, or the four names baked into a CHECK constraint. Both make a fifth filing type a structural change. The lookup table makes it a row.

Required services live in the runner, not in the schema file

Rejected, and caught in review. The four service rows were first written as INSERT statements inside `001_schema.sql`. That file is applied once and then frozen forever, so a fifth service added later would never reach a database that already ran it. They moved into `REQUIRED_SERVICES` in the runner, ensured on every initialize call. The general lesson is that anything inside an applied migration is frozen the moment one real database runs it.

Settings as one typed row, not a key and value store

Rejected. A generic table of key and value pairs. There is exactly one known configuration value, so a generic store was flexibility nobody asked for, and it would have thrown away typed columns and constraints. A future second value is a migration adding a column.

Transactions inside the migration script

Rejected, and caught in review. The first version applied a migration and then recorded it, relying on one `.commit()` call. Verification showed `executescript` commits on its own, so the two were never atomic and a crash between them could leave a migration applied but unrecorded. The fix carries `BEGIN` and `COMMIT` inside the executed script. A test now breaks a migration halfway and proves neither its tables nor its record survive.

Confirm before create, in onboarding

Rejected. Automatically turning every subfolder into a client. In a system that deliberately never deletes, the moment before a row exists is the only cheap place for human judgment. Folders such as Backup and Old are not clients, and only the practitioner knows that.

Deterministic seed data

Caught in review. The seed set a completion timestamp to the current time, so a second run changed data even though the row counts matched, which contradicted the claim that it was repeatable. The timestamp became a fixed constant, and the test was strengthened from comparing counts to comparing every value of every row.

Raw SQL, no ORM

Rejected. An object relational mapper, a library that hides SQL behind objects. The project's second purpose is learning, and hidden SQL defeats that. The cost accepted is writing queries by hand, which at this size is small.

Part 8, What Is Not Built, And Why

Each of these is absent deliberately, not forgotten.

- assigning services to clients. The next feature. Until it exists, every onboarded client has zero services, so nothing downstream can produce work
 - periods and task generation. They depend on service assignment
 - the dashboard. It depends on tasks existing
 - documents and the proof scanner. They depend on knowing what real filenames look like, which requires seeing a practitioner's actual folders
 - real due dates. The due day for each filing type is a business fact the practitioner has to supply. Guessing would put invented rules into a compliance tool
 - authentication. The application listens only on localhost, so the operating system login is the front door. This must be revisited the moment anything is reachable from another machine
 - continuous integration. Nothing runs the 33 tests automatically on every change yet
 - backups. There is no backup mechanism for the database file
-

Part 9, Honest Limitations

Named plainly, because a document that only lists strengths is marketing.

- no backup exists. Once real data lives in `database/taxdesk.db`, a disk failure loses it. This is the most important gap before the product is used for real work
 - no automated test run on changes. The suite is only as useful as someone remembering to run it
 - the seed writes rows for subscribed services only, while the upcoming service assignment feature will write rows for every service including inactive ones. Two shapes for one table, worth reconciling
 - the application has no navigation. `base.html` is a heading and a content block, so features are reachable only by typing URLs
 - changing the root folder does not repair existing client paths. If a whole tree moves, stored paths go stale, and path repair is a named future feature rather than a silent side effect
 - there is no way to edit or archive a client once created
 - table naming is inconsistent, most tables are uppercase and `documents` is lowercase. Cosmetic, and frozen by applied migrations, so a rename would need a rebuild migration
-

Part 10, Running And Developing

Running the application

```
python3 -m venv venv
venv/bin/pip install fastapi uvicorn jinja2 python-multipart
venv/bin/uvicorn app.main:app
```

Then open `http://127.0.0.1:8000` in a browser. Migrations run automatically at startup. Never pass a host flag, listening on localhost only is the security boundary.

Development tasks

```
venv/bin/pip install pytest httpx2      # test tooling
venv/bin/pytest                          # run the 33 tests
venv/bin/pytest -W error                  # treat warnings as failures
python3 -m database.seed                  # fill a development database
python3 database/migrate.py               # apply migrations by hand
```

Dependencies, and why each exists

- fastapi, the web framework, routing and dependency injection
- uvicorn, the server that listens on the port
- jinja2, HTML templates with automatic escaping
- python-multipart, required by the framework to parse form posts
- pytest, the test runner, development only
- httpx2, the HTTP client the framework's test client needs, development only

Six pinned packages, no ORM, no frontend framework, no build step.

Adding a change

1. branch from main, named for the single intent
 2. write the code and the tests together
 3. run the full suite
 4. update `docs/engineering-log.md` with purpose, what changed, why this way, and how it works
 5. open a PR, get it reviewed, fix findings on the same branch
 6. merge, then delete the branch
-

Part 11, Glossary

- ACID: the four guarantees of a database transaction, atomicity, consistency, isolation, durability
- ASGI: the standard interface between a Python web framework and the server that runs it
- constraint: a rule attached to a table that the database enforces on every write
- decorator: a line above a function that hands that function to another function, used here to register routes
- dependency injection: the framework supplies what a function declares it needs, rather than the function building it
- fixture: reusable setup a test asks for by name
- foreign key: a column holding another table's primary key
- idempotent: doing it twice leaves the same result as doing it once
- junction table: a table whose rows are pairings, used for many to many relationships
- local first: the software and the data live on the user's own machine
- migration: one numbered, recorded change to database structure
- normalization: organising data so each fact lives in exactly one place
- ORM: a library that maps database rows to objects, deliberately not used here
- parameterized query: SQL sent separately from its values, so values can never become commands
- post redirect get: answering a form POST with a redirect so refresh cannot resubmit
- PRAGMA: a SQLite specific setting command, used here for foreign keys
- primary key: the column or columns identifying a row uniquely
- referential integrity: the guarantee that pointers between tables always point at real rows
- rollback: discarding a transaction's writes
- soft delete: marking a row inactive instead of removing it
- SQL injection: an attack where input is treated as command
- surrogate key: an invented id with no business meaning
- transaction: a group of writes that all succeed or all fail
- trust boundary: the line where outside data enters a program and must be validated
- upsert: an insert that becomes an update when the row already exists
- XSS: cross site scripting, an attack where input becomes executable page content